Министерство просвещения Республики Казахстан [ПРОВЕРИТЬ]

Колледж Astana IT University

КУРСОВОЙ ПРОЕКТ

по профессиональному модулю ПМ.07 «Разработка мобильных приложений»

на тему:

«Создание приложения для автоматического формирования персонального плана обучения»

Выполнил: студент 3 курса, группа ПО2410

Баринов Н. В.

Руководитель: Синтиков Е. К.

Оценка: _____

Астана 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....4
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ПРИЛОЖЕНИЯ ДЛЯ ФОРМИРОВАНИЯ ПЕРСОНАЛЬНОГО ПЛАНА ОБУЧЕНИЯ.....8
1.1 Анализ предметной области.....8
1.2 Особенности мобильных приложений.....11
1.3 Принципы разработки мобильных приложений.....13
1.4 Анализ аналогичных решений.....15
1.5 Обоснование выбора технологий.....16
Выводы по главе 1.....24
ГЛАВА 2. РАЗРАБОТКА ПРИЛОЖЕНИЯ «МАРШРУТ».....26
2.1 Требования к приложению.....26
2.2 Архитектура приложения.....27
2.3 Проектирование базы данных и API сервера.....30
2.4 Алгоритм формирования плана.....37
2.5 Интерфейс приложения.....42
2.6 Реализация основных модулей.....52
2.7 Тестирование.....56
2.8 Сборка и развёртывание.....57
Выводы по главе 2.....58
ЗАКЛЮЧЕНИЕ.....60
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....63
Книги и статьи.....63
Электронные ресурсы.....64
ПРИЛОЖЕНИЕ А. ТОПОЛОГИЧЕСКАЯ СОРТИРОВКА ТЕМ (АЛГОРИТМ КАНА).....69
ПРИЛОЖЕНИЕ Б. ХЭШИРОВАНИЕ ПАРОЛЕЙ PBKDF2-HMAC-SHA256 71
ПРИЛОЖЕНИЕ В. РАСКЛАДКА ТЕМ ПО ДНЯМ И ИНТЕРВАЛЬНЫЕ ПОВТОРЕНИЯ.....75

ПРИЛОЖЕНИЕ Г. ГЕНЕРАТОР ПЛАНА И ПРОВЕРКА ДЕДЛАЙНА.....79
ПРИЛОЖЕНИЕ Д. ПЕРЕПЛАНИРОВАНИЕ.....82
ПРИЛОЖЕНИЕ Е. ТАБЛИЦЫ БАЗЫ ДАННЫХ.....85
ПРИЛОЖЕНИЕ Ж. ПРАВИЛО ДОСТУПА К ЭКРАНАМ И МАРШРУТЫ. .90
ПРИЛОЖЕНИЕ И. МОДУЛЬНЫЕ ТЕСТЫ ТОПОЛОГИЧЕСКОЙ СОРТИРОВКИ.....94
ПРИЛОЖЕНИЕ К. СЕРВЕР API: АДРЕСА, МАРШРУТЫ, ПРОВЕРКА ТОКЕНА И ОБРАБОТКА ОШИБОК.....96
ПРИЛОЖЕНИЕ Л. КЛИЕНТ API В ПРИЛОЖЕНИИ.....102

ВВЕДЕНИЕ

Актуальность темы. Самостоятельное обучение стало обычной частью учёбы и работы. Студенты осваивают технологии сверх учебной программы, специалисты переходят в новую профессию, школьники готовятся к экзаменам. Учебных материалов стало больше, чем человек успевает изучить: документация, видеокурсы, статьи, онлайн-платформы [13; 46]. Найти материалы перестало быть главной трудностью. Теперь человеку трудно понять, в каком порядке изучать темы, сколько времени это займёт и успеет ли он к нужному сроку.

У того, кто учится самостоятельно, обычно возникают три проблемы.

1. Неясен порядок изучения. Темы зависят друг от друга: виджеты Flutter не понять без основ Dart, а Present Perfect в английском — без знания глагола *to be*. Если пропустить базовую тему, дальнейшее обучение замедляется.
2. Не учитываются уже имеющиеся знания и личное время. Готовые курсы предлагают один план для всех: опытный ученик тратит время на знакомые темы, а занятый человек не успевает за чужим темпом.
3. План разрушается при отставании. Стоит пропустить несколько занятий, и расписание перестаёт соответствовать действительности. Пересчитывать его вручную долго, и часто человек просто бросает обучение.

Существующие решения закрывают эти проблемы по отдельности. Карты технологий показывают зависимости между темами, но не строят расписание [43]. Языковые сервисы мотивируют ежедневными целями, но план в них задаёт сам сервис [16]. Планировщики задач заставляют расписывать всё вручную [34; 48]. Анализ аналогов приведён в п. 1.4. Мобильного приложения, которое автоматически строит персональный план по дням с учётом зависимостей тем, знаний пользователя, его времени и

дедлайна и само перестраивает план при отставании, среди рассмотренных решений не найдено.

Для такой задачи подходит именно мобильное приложение. Телефон всегда под рукой, и пользователь может отметить занятие сразу после него, получить напоминание и увидеть план на сегодня. Это объясняет актуальность темы курсового проекта.

Цель работы — разработать мобильное приложение «Маршрут», которое автоматически формирует персональный план обучения по выбранному направлению с учётом уровня знаний пользователя, доступного времени и срока, а также перестраивает план при отставании.

Задачи работы:

1. Проанализировать предметную область: способы организации самостоятельного обучения, методики распределённого повторения и тайм-менеджмента.
2. Рассмотреть особенности мобильных приложений и принципы их разработки.
3. Проанализировать аналогичные решения и выявить их недостатки.
4. Обосновать выбор технологий: платформы, архитектуры, способа управления состоянием, хранилища данных и способа доступа к ним через API.
5. Разработать алгоритм формирования плана: граф тем, исключение известных тем по результатам теста, топологическая сортировка, раскладка по дням с учётом лимита часов, проверка дедлайна и интервальные повторения.
6. Спроектировать интерфейс и реализовать экраны приложения: заставку, регистрацию, авторизацию, главную страницу, навигацию, профиль, основной раздел с планом и выход из системы.
7. Протестировать приложение модульными, виджет- и интеграционными тестами и исправить найденные ошибки.

8. Проанализировать первую версию приложения, устранить найденные недостатки и подготовить релизную сборку для Android, iPhone и Windows.
9. Разработать сервер REST API с базой данных и перевести приложение на работу с данными только через API с обработкой ответов и ошибок сервера.

Объект разработки — процесс планирования самостоятельного обучения.

Предмет разработки — мобильное приложение, которое автоматически формирует и сопровождает персональный план обучения.

Целевая аудитория:

- студенты колледжей и вузов, которые осваивают технологии сверх программы или готовятся к экзаменам;
- самоучки, изучающие программирование или иностранный язык;
- люди, которые меняют профессию и учатся параллельно с работой.

Общее у этих пользователей одно: они учатся без преподавателя, у них ограничено время и, как правило, есть срок, к которому нужно успеть.

Основной функционал приложения:

- регистрация и вход по email и паролю; данные хранятся в базе на собственном сервере API, пароль сохраняется в виде хэша с солью;
- выбор направления обучения из каталога: Flutter-разработка, Python, английский язык, алгоритмы и структуры данных;
- короткий диагностический тест, по результатам которого известные темы исключаются из плана;
- настройка расписания: часов в неделю, учебных дней и дедлайна;
- автоматическое построение плана по дням с учётом зависимостей тем и интервальными повторениями;
- проверка дедлайна и подсказка, сколько часов в неделю нужно, чтобы успеть;

- отметка выполненных занятий и перестройка плана одной кнопкой при отставании;
- главная страница с прогрессом и задачами на сегодня, статистика, профиль пользователя.

Методы разработки: анализ и сравнение аналогов, объектно-ориентированное проектирование, клиент-серверная архитектура и REST API, методы теории графов (топологическая сортировка), модульное, виджет- и интеграционное тестирование.

Практическая значимость работы. Приложение можно использовать для реальной самостоятельной подготовки: уже сейчас в каталоге четыре направления и 75 тем с оценкой трудоёмкости и зависимостями. Приложение не требует облачных сервисов, ключей и платных подписок: сервер API запускается на обычном ноутбуке, и телефон работает с ним по Wi-Fi без доступа в интернет. Каталог хранится в JSON-файлах, поэтому новое направление добавляется без изменения кода. Алгоритм планирования выделен в отдельный слой на чистом Dart и покрыт тестами, поэтому его можно повторно использовать в других проектах, например в веб-версии или на сервере.

Структура работы. Курсовая работа состоит из введения, двух глав, заключения, списка использованной литературы и приложений. В первой главе рассмотрены предметная область, особенности и принципы разработки мобильных приложений, проведён анализ аналогов и обоснован выбор технологий. Во второй главе описаны архитектура, база данных и API сервера, алгоритм, интерфейс, реализация и тестирование приложения.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ПРИЛОЖЕНИЯ ДЛЯ ФОРМИРОВАНИЯ ПЕРСОНАЛЬНОГО ПЛАНА ОБУЧЕНИЯ

1.1 Анализ предметной области

1.1.1 Персональный план обучения

Персональный план обучения — это расписание изучения тем, составленное под конкретного человека с учётом его цели, уровня подготовки, доступного времени и срока. В отличие от учебной программы курса, которая одинакова для всех слушателей, персональный план отвечает на три вопроса:

1. **Что изучать?** Набор тем по выбранному направлению без тех, которые человек уже знает.
2. **В каком порядке?** Последовательность, в которой каждая тема идёт после тем, на которые она опирается.
3. **Когда?** Распределение тем по дням в пределах времени, которое человек готов тратить, и с учётом дедлайна.

Чтобы автоматизировать составление плана, предметную область нужно описать формально. Направление обучения разбивается на темы. Для каждой темы известны:

- трудоёмкость, то есть примерное время на изучение в часах;
- уровень сложности;
- категория (например, «Основы», «Интерфейс», «Данные»);
- список тем-предпосылок, без которых тему не понять;
- учебные материалы;
- контрольный вопрос для диагностики (не у всех тем).

Темы вместе с зависимостями образуют **ориентированный граф**: вершины — темы, дуга A → B означает, что тему A нужно изучить раньше темы B. Порядок изучения должен соблюдать все дуги, поэтому в графе не может быть циклов. Если тема A требует B, а B требует A, изучить их

невозможно. Такой граф называется ориентированным ациклическим графом (DAG, Directed Acyclic Graph). Последовательность его вершин, в которой каждая дуга ведёт «вперёд», называется топологической сортировкой [1].

1.1.2 Топологическая сортировка: алгоритм Кана

Один из классических алгоритмов топологической сортировки предложил А. Кан в 1962 году [6]. Алгоритм работает так:

1. Для каждой вершины считается *входящая степень* — число дуг, которые в неё входят. Для темы это число ещё не изученных предпосылок.
2. Вершины с нулевой входящей степенью, то есть темы без предпосылок, помещаются в очередь.
3. Пока очередь не пуста, из неё извлекается вершина и добавляется в результат. У всех вершин, в которые из неё ведут дуги, входящая степень уменьшается на единицу. Вершины, у которых она стала нулевой, добавляются в очередь.
4. Если в результат попали не все вершины, в графе есть цикл, и упорядочить такие темы нельзя.

Каждая вершина и каждая дуга обрабатываются один раз, поэтому время работы алгоритма — O(V + E), где V — число тем, E — число зависимостей [1]. Для каталога из 15–25 тем это доли миллисекунды даже на слабом телефоне.

Алгоритм Кана удобен тем, что сразу обнаруживает циклы. Если в каталоге по ошибке появится взаимная зависимость, приложение не зависнет и не построит неверный план, а сообщит об ошибке данных. Кроме того, если вместо простой очереди использовать очередь с приоритетом, можно управлять порядком среди тем, которые одновременно доступны для изучения. Например, брать сначала более лёгкие темы. Это делает результат детерминированным: для одних и тех же данных план всегда одинаков, что важно для тестирования.

1.1.3 Распределённое повторение

Изученный материал постепенно забывается. Экспериментально это впервые исследовал Г. Эббингауз: он показал, что доля сохранённого материала быстро снижается в первые часы и дни после заучивания, а затем убывает медленнее (так называемая «кривая забывания») [5]. Повторение замедляет забывание. При этом важно не только сколько повторять, но и когда.

Метаанализ Н. Сепеды и соавторов, обобщивший сотни экспериментов, подтвердил *эффект распределённого повторения*: при одинаковом суммарном времени занятия, разнесённые по времени, дают лучшее долговременное запоминание, чем те же занятия подряд [3]. На этом принципе построены системы интервальных повторений: к теме возвращаются через возрастающие промежутки времени.

В приложении «Маршрут» после изучения каждой темы автоматически планируются короткие повторения через 1, 3 и 7 дней. Интервалы выбраны как простое и понятное пользователю приближение к принципу возрастающих промежутков. Подбор оптимальных интервалов для каждого пользователя выходит за рамки курсового проекта.

1.1.4 Тайм-менеджмент: техника Pomodoro

Составить план недостаточно: его нужно выполнять. Для концентрации во время занятия часто используют технику Pomodoro, которую предложил Ф. Чирилло в конце 1980-х годов [4]. Работа делится на интервалы фокуса, классически по 25 минут, между ними делаются короткие перерывы, а после четырёх интервалов — длинный перерыв. Таймер Pomodoro дополняет план: план говорит, *что* изучать сегодня, а таймер помогает сосредоточиться на этом.

1.1.5 Требования к приложению

Из анализа предметной области следуют основные требования к приложению (таблица 1.1).

Таблица 1.1 — Требования к приложению и их источники

№	Требование	Откуда следует
1	Хранить каталог направлений и тем с трудоёмкостью и зависимостями	Формальная модель предметной области (п. 1.1.1)
2	Определять уже известные темы с помощью короткого теста	Проблема «один план для всех»
3	Упорядочивать темы с учётом зависимостей и обнаруживать ошибки в данных	Граф тем, алгоритм Кана (п. 1.1.2)
4	Раскладывать темы по выбранным дням с лимитом времени в день	Учёт личного времени
5	Проверять дедлайн и подсказывать нужную нагрузку	Учёт срока
6	Планировать повторения	Эффект распределённого повторения (п. 1.1.3)
7	Перестраивать план при пропусках, сохраняя выполненное	Проблема «план разрушается при отставании»
8	Отмечать занятия, показывать прогресс, напоминать о занятиях, вести таймер	Выполнение плана, мотивация (п. 1.1.4)
9	Хранить данные на сервере и работать с ними только через API, защищать вход паролем, работать без облачных сервисов	Личные данные, требования задания к работе с API

1.2 Особенности мобильных приложений

Мобильные приложения разрабатываются с учётом ряда особенностей, которых нет у настольных программ и веб-сайтов.

Ограниченный экран и сенсорный ввод. На экране телефона помещается немного информации, а управление идёт пальцем, а не точным курсором мыши. Поэтому элементы управления должны быть достаточно крупными, на экране должно быть только главное, а второстепенное нужно выносить на отдельные экраны. В рекомендациях Material Design описаны минимальные размеры области касания и типовые компоненты навигации: нижняя панель, вкладки, плавающая кнопка действия [33].

Разные размеры экрана и ориентация. Одно и то же приложение запускается на компактных телефонах, планшетах, складных устройствах, а Flutter-приложение ещё и в окне настольной системы [25]. Интерфейс должен

приспосабливаться к ширине окна. В рекомендациях Android окна делятся на классы по ширине: компактный (до 600 dp), средний (от 600 до 840 dp) и расширенный (от 840 dp) [50]. На компактном экране навигацию принято размещать внизу, на расширенном — сбоку.

Жизненный цикл и прерывания. Пользователь может в любой момент свернуть приложение, получить звонок или переключиться на другую программу. Операционная система вправе завершить процесс свёрнутого приложения, если ей не хватает памяти [36]. Поэтому важные данные нужно сохранять сразу, а не «при выходе», а после повторного запуска восстанавливать состояние, например автоматически входить в аккаунт.

Работа без сети. Мобильный интернет бывает медленным или недоступным: в транспорте, в метро, в поездке. Приложение, которое без сети перестаёт работать, теряет пользователя. Подход offline-first, при котором данные хранятся на устройстве, делает приложение доступным всегда.

Уведомления. Мобильные платформы позволяют приложению напомнить о себе, даже когда оно закрыто. Для приложения, где важна регулярность, это ключевая функция. На Android 13 и новее для показа уведомлений нужно отдельное разрешение пользователя, поэтому его нужно запрашивать в момент, когда пользователь понимает, зачем оно нужно [27].

Ограниченные ресурсы. Аккумулятор, память и процессор телефона ограничены. Тяжёлые вычисления и постоянная работа в фоне быстро разряжают батарею. Алгоритмы приложения должны быть эффективными, а фоновые задачи нужно поручать системе, например планировщику уведомлений.

Безопасность данных. Телефон легко потерять, им могут пользоваться несколько человек. Пароли нельзя хранить в открытом виде. Минимальная мера — хранить хэш пароля с уникальной случайной солью, чтобы

одинаковые пароли давали разные хэши и нельзя было воспользоваться заранее вычисленными таблицами [35].

Несколько платформ. Основные мобильные платформы — Android и iOS, у каждой свой язык разработки, набор инструментов и правила оформления. Разработка двух отдельных приложений удваивает затраты, поэтому популярны кроссплатформенные фреймворки, позволяющие писать один код для нескольких платформ (п. 1.5.1).

1.3 Принципы разработки мобильных приложений

С учётом перечисленных особенностей при разработке используются следующие принципы.

1. Разделение ответственности (слоистая архитектура). Код приложения делится на слои, каждый из которых отвечает за своё. В подходе «чистой архитектуры» Р. Мартина бизнес-правила находятся в центре и не зависят от интерфейса, базы данных и фреймворков; зависимости направлены только внутрь [2]. Для мобильного приложения обычно выделяют три слоя:

- *presentation* — экраны и управление их состоянием;
- *domain* — сущности, бизнес-логика и сценарии использования;
- *data* — работа с базой данных, файлами и сетью.

Документация Flutter также рекомендует разделять приложение на слой интерфейса и слой данных, а сложную логику выносить в отдельные классы [22]. Такое разделение позволяет тестировать логику без запуска интерфейса и заменять хранилище, не переписывая экраны.

2. Однонаправленный поток данных и явное управление состоянием. Состояние приложения (текущий пользователь, план, список занятий) хранится в одном месте. Интерфейс строится по состоянию и меняет его только через определённые методы. В Flutter для этого используются библиотеки управления состоянием: Provider, Riverpod, BLoC

и другие [24]. Явное состояние делает поведение предсказуемым и упрощает поиск ошибок.

3. Декларативный интерфейс. Во Flutter интерфейс описывается как функция от состояния: при изменении данных фреймворк сам перестраивает нужные виджеты [9]. Разработчику не нужно вручную синхронизировать экран с данными, что снижает количество ошибок.

4. Единая система дизайна. Все экраны оформляются по одним правилам: цвета, шрифты, отступы, формы кнопок. Material Design 3 задаёт цветовые схемы, типографику и компоненты для светлой и тёмной темы [33]. Во Flutter это реализуется через объект `ThemeData`, поэтому стиль задаётся один раз для всего приложения.

5. Адаптивность. Вёрстка строится от доступной ширины, а не от фиксированных размеров: на широком экране навигация переносится вбок, а содержимое ограничивается по ширине, чтобы строки не растягивались [50].

6. Обработка ошибок и пустых состояний. Каждый экран, загружающий данные, должен показывать три состояния: загрузку, ошибку с понятным сообщением и пустое состояние с подсказкой, что делать дальше. Ошибки ввода показываются рядом с полем, а не в общем окне.

7. Защита закрытых разделов. Экраны, доступные только после входа, защищаются на уровне навигации: при попытке открыть их без авторизации пользователь перенаправляется на экран входа. Проверка в одном месте, в маршрутизаторе, надёжнее, чем проверки на каждом экране.

8. Тестирование. Различают три уровня автоматических тестов: модульные (проверяют функцию или класс), виджет-тесты (проверяют экран без устройства) и интеграционные (проверяют приложение целиком на устройстве или эмуляторе) [9]. Чем больше логики вынесено в чистые функции, тем больше её покрывается быстрыми модульными тестами.

1.4 Анализ аналогичных решений

Для анализа выбраны решения, которыми пользуются при самостоятельном обучении: сайт карт изучения технологий roadmap.sh [43], сервис изучения языков Duolingo [16], образовательные платформы Stepik [46] и Coursera [13], а также планировщики задач Notion [34] и Todoist [48].

roadmap.sh — открытый проект с картами изучения технологий: frontend, backend, Python, Flutter и других. Карта показывает темы и связи между ними. Сильная сторона — наглядный граф тем, составленный сообществом. Недостаток — карта общая для всех. С 2025 года её можно персонализировать с помощью ИИ с учётом опыта пользователя [43], но расписания по дням, учёта срока и перестройки плана при отставании нет.

Duolingo — популярный сервис для изучения иностранных языков; позднее в нём появились курсы математики и музыки. Сильные стороны — ежедневная цель, серия дней подряд и напоминания. Порядок уроков задаёт сам сервис; есть входной тест уровня. Расписание под дедлайн пользователя не строится, набор направлений ограничен.

Stepik и Coursera — платформы онлайн-курсов. Внутри курса темы упорядочены автором, у некоторых курсов есть сроки сдачи заданий. На Coursera при отставании сроки заданий можно перенести вперёд [13]. Платформы дают сами материалы, но не составляют личный план: темп и порядок определяет курс, а не пользователь.

Notion и Todoist — универсальные инструменты для заметок и задач. В них можно составить любой план, но вручную. В Notion задачи можно связать зависимостями, и при переносе одной сдвигаются сроки связанных [34], однако порядок тем, расписание и нагрузку пользователь задаёт сам. В Todoist зависимостей между задачами нет, просроченные задачи переносятся вручную [48].

Результаты сравнения сведены в таблицу 1.2.

Таблица 1.2 — Сравнение аналогов

Возможность	**roadmap.sh**	**Duolingo**	**Stepik / Coursera**	**Notion / Todoist**	**«Маршрут»**
Порядок тем с учётом зависимостей	да, статичная схема	внутри курса	внутри курса	вручную (Notion)	да, автоматически
Учёт уже имеющихся знаний	частично, ИИ-персонализация	частично, тест уровня	нет	нет	да, диагностический тест
План по дням под личное время	нет	ежедневная цель	сроки курса	вручную	да
Проверка дедлайна и подсказка нагрузки	нет	нет	нет	нет	да
Перестройка плана при отставании	нет	нет	частично, перенос сроков (Coursera)	частично, сдвиг связанных сроков (Notion)	да, одной кнопкой
Интервальные повторения	нет	да	нет	нет	да
Работа без облачной учётной записи (свой сервер в локальной сети)	нет	нет	нет	частично	да

Сведения об аналогах сверены с сайтами и справкой сервисов 11.10.2026 [13; 16; 34; 43; 46; 48].

Вывод по анализу аналогов. Каждое из рассмотренных решений закрывает часть задачи. roadmap.sh даёт граф тем, Duolingo мотивирует регулярностью, платформы курсов дают материалы, планировщики позволяют вести расписание. Ни одно из них не объединяет все шаги: граф тем, учёт знаний, расписание под время и срок пользователя и автоматическую перестройку. Эти возможности и составляют основу разрабатываемого приложения «Маршрут».

1.5 Обоснование выбора технологий

1.5.1 Платформа разработки: Flutter и Dart

Для разработки мобильного приложения рассматривались три подхода:

1. **Нативная разработка** — Kotlin для Android и Swift для iOS. Даёт полный доступ к возможностям платформы, но требует двух отдельных приложений с двумя кодовыми базами.
2. **React Native** (Meta) — кроссплатформенный фреймворк на JavaScript/TypeScript, использующий нативные компоненты платформы [37].
3. **Flutter** (Google) — кроссплатформенный фреймворк на языке Dart с собственным движком отрисовки интерфейса [9; 25].

Также существует Kotlin Multiplatform от JetBrains, позволяющий выносить общую логику в общий код [32]; интерфейс при этом часто пишется отдельно для каждой платформы или с помощью Compose Multiplatform.

Выбран Flutter по следующим причинам:

- **Одна кодовая база для нескольких платформ.** Из одного проекта собираются приложения для Android и iOS, а также веб-версия и настольные приложения для Windows, macOS и Linux [25]. В рамках проекта из одного кода собираются релизный APK для Android, версия для Windows и версия для Linux; сборка и тесты выполняются автоматически (п. 2.8).
- **Собственная отрисовка.** Flutter рисует интерфейс сам, а не через нативные компоненты, поэтому приложение выглядит одинаково на всех устройствах [7; 9].
- **Быстрая разработка.** Горячая перезагрузка (hot reload) применяет изменения кода за секунды без перезапуска приложения [7].
- **Производительность.** В режиме разработки код Dart выполняется JIT-компилятором, а в релизной сборке компилируется заранее (AOT) в машинный код [14].
- **Богатая библиотека виджетов Material 3** и большое число пакетов на pub.dev.

- Технология изучалась в рамках дисциплины ПМ07, что позволило сосредоточиться на алгоритме и качестве приложения.

Язык Dart — объектно-ориентированный язык со строгой статической типизацией. С версии Dart 3 обязательна *звуковая null-безопасность*: компилятор гарантирует, что переменная без знака `?` не может быть `null` [45]. Это исключает целый класс ошибок времени выполнения. Dart поддерживает асинхронное программирование через `Future`, `async/await` и `Stream`, что используется для запросов к серверу и реактивного обновления экранов [14]. На Dart написан и сервер API приложения, поэтому клиент и сервер используют общий код: сущности, правила проверки данных и формат JSON.

1.5.2 Архитектура: Clean Architecture

Приложение построено по принципам чистой архитектуры с тремя слоями [2; 22]:

- **domain** — сущности (тема, план, занятие), алгоритм формирования плана, интерфейсы репозиториев. Слой написан на чистом Dart и не зависит от Flutter, поэтому алгоритм тестируется быстрыми модульными тестами;
- **data** — реализации репозиториев: в приложении это репозитории, которые обращаются к серверу API, на сервере — репозитории базы данных SQLite; также JSON-каталог тем, настройки, уведомления;
- **presentation** — экраны, виджеты и провайдеры состояния.

Альтернатива — писать логику прямо в виджетах. Для небольших учебных примеров это допустимо, но в приложении с нетривиальным алгоритмом такой код трудно тестировать и развивать. Например, алгоритм пришлось бы проверять через нажатия на кнопки.

1.5.3 Управление состоянием: Riverpod

Рассматривались три распространённых варианта [24].

Таблица 1.3 — Сравнение способов управления состоянием

Критерий	**Provider**	**BLoC**	**Riverpod**
Объём шаблонного кода	малый	большой: события, состояния, блоки	малый
Зависимость от `BuildContext`	да	да (через `BlocProvider`)	нет
Обнаружение ошибок	часть ошибок только во время выполнения	на этапе компиляции	на этапе компиляции
Асинхронные данные	вручную	через состояния блока	встроенный тип `AsyncValue` (загрузка, данные, ошибка)
Подмена зависимостей в тестах	ограниченно	да	да, через `overrides`

Provider прост, но провайдеры ищутся по дереву виджетов, и ошибка «провайдер не найден» проявляется только при запуске. **BLoC** — архитектурный паттерн, который Google представил на конференции DartConf в 2018 году, и реализующая его библиотека flutter_bloc [12]. Он хорошо подходит большим командам, но требует много кода: для каждого действия нужно описать событие и состояние.

Выбран **Riverpod** [42] — библиотека того же автора, что и Provider, созданная для устранения его ограничений:

- провайдеры объявляются как глобальные константы и не зависят от дерева виджетов, поэтому ошибки обнаруживаются при компиляции;
- `AsyncValue` единообразно описывает загрузку, данные и ошибку, что упрощает показ трёх состояний экрана (п. 1.3);
- любой провайдер подменяется в тестах через `ProviderScope(overrides: …)`, например HTTP-клиент, который передаёт запросы серверу API в памяти вместо сети, и фиксированная дата вместо текущей;
- провайдеры-потоки (`StreamProvider`) получают новые данные после каждого изменения на сервере, и экраны обновляются автоматически.

1.5.4 Хранение данных: собственный сервер API и drift (SQLite)

Требования к хранилищу: связанные данные (пользователь → планы → темы плана → занятия), выборки с фильтрами и сортировкой, целостность данных при удалении. Задание к курсовой работе добавляет ещё одно требование: приложение должно работать с базой данных исключительно через API, то есть получать, добавлять, изменять и удалять данные запросами к серверу и обрабатывать его ответы и ошибки. Локальная база на устройстве, которая использовалась в первой версии приложения, этому требованию не отвечает.

Таблица 1.4 — Сравнение способов хранения данных

Критерий	**Локальная база на устройстве (drift)**	**Firebase**	**Supabase**	**Свой сервер API (shelf + drift)**
Доступ к данным через API	нет, прямые запросы к базе	да, через SDK	да, REST API	да, REST API
Модель данных	реляционные таблицы	документы в облаке	реляционные таблицы (PostgreSQL)	реляционные таблицы (SQLite)
Связи и каскадное удаление	внешние ключи	вручную	внешние ключи	внешние ключи, `ON DELETE CASCADE`
Нужны интернет, учётная запись и ключи	нет	да	да	нет: сервер работает в локальной сети
Где хранятся данные	на телефоне	на серверах Google	на серверах Supabase	на компьютере разработчика
Язык серверной логики	—	правила безопасности, функции на JavaScript	SQL, функции на TypeScript	Dart, как и приложение

Firebase — облачная платформа Google [18], **Supabase** — облачная платформа с базой PostgreSQL и готовым REST API [47]. Обе дают API без написания сервера, но требуют интернета, учётной записи и ключей доступа, а данные пользователей хранятся у стороннего сервиса. Для учебного проекта, который должен запускаться без настройки облака, и для защиты в аудитории, где может не быть стабильного интернета, это неудобно.

Выбран **собственный сервер API** на Dart. Сервер принимает запросы приложения по протоколу HTTP, проверяет их и работает с базой данных SQLite. SQLite — встраиваемая реляционная СУБД, которая хранит базу в одном файле [10]; на сервере база открывается через библиотеку **drift** [15],

которая генерирует типобезопасный код по описанию таблиц и поддерживает транзакции и внешние ключи. Репозитории drift, написанные для первой версии приложения, перенесены на сервер без изменений, а приложение получило новые реализации тех же интерфейсов, которые отправляют запросы к API. Сервер запускается на ноутбуке, и телефон подключается к нему по Wi-Fi без доступа в интернет.

shared_preferences подходит только для простых настроек; в приложении он хранит адрес сервера, идентификатор пользователя и токен входа (если включено «Запомнить меня»), а также выбранную тему оформления. Hive [30] — быстрое хранилище «ключ–значение» на чистом Dart — не рассматривался для основных данных, так как связи между объектами и каскадное удаление пришлось бы реализовывать вручную.

1.5.5 Клиент-серверная архитектура и REST API

В клиент-серверной архитектуре приложение (клиент) не хранит общие данные само, а запрашивает их у сервера. Сервер — единственное место, где находится база данных: он проверяет, кто отправил запрос и имеет ли право на эти данные, проверяет корректность данных и только после этого выполняет действие.

Для обмена данными выбран архитектурный стиль **REST** [17]. Его основные принципы:

- каждая сущность доступна по своему адресу (ресурс): `/api/notes` — заметки пользователя, `/api/notes/3` — заметка с номером 3;
- действие с ресурсом задаётся методом HTTP [41]; методы соответствуют операциям CRUD (таблица 1.5);
- сервер не хранит состояние диалога с клиентом: каждый запрос содержит всё нужное для его выполнения, в том числе токен входа;
- результат запроса сообщается кодом ответа HTTP: 2xx — успех, 4xx — ошибка в запросе, 5xx — ошибка сервера [41].

Таблица 1.5 — Методы HTTP и операции с данными

Метод HTTP	**Операция CRUD**	**Пример в приложении**
GET	Получение (Read)	`GET /api/notes` — список заметок
POST	Добавление (Create)	`POST /api/notes` — новая заметка, ответ `201 Created` с её номером
PUT, PATCH	Изменение (Update)	`PUT /api/notes/3` — изменение заметки, `PATCH /api/plans/2` — переименование плана
DELETE	Удаление (Delete)	`DELETE /api/notes/3` — удаление заметки, ответ `204 No Content`

Данные передаются в формате **JSON** [40] — текстовом формате, который одинаково просто читается в Dart и человеком при отладке. Ошибка также возвращается в JSON: `{"error": "Заметка не найдена"}` с подходящим кодом ответа, поэтому приложение может показать пользователю понятный текст.

Аутентификация по токену. После успешного входа сервер выдаёт случайный токен, и приложение отправляет его с каждым запросом в заголовке `Authorization: Bearer <токен>` [38]. Сервер по токену определяет пользователя и отдаёт только его данные. В базе хранится не сам токен, а его хэш SHA-256 [8], поэтому копия файла базы не позволит войти от имени пользователя.

Сервер написан на Dart с библиотекой **shelf** [44] — стандартной библиотекой команды Dart для HTTP-серверов — и пакетом shelf_router, который сопоставляет адрес и метод запроса с функцией-обработчиком. В приложении запросы отправляет пакет **http** [31]. Сервер собирается в самостоятельную программу командой `dart build cli`, и для его запуска на Windows не нужно устанавливать Dart или Flutter.

1.5.6 Навигация: go_router

go_router — официальный пакет команды Flutter для декларативной навигации [29]. Маршруты описываются как адреса (`/home`, `/plan`, `/session/:id`), что упрощает переходы и передачу параметров. Важная возможность — функция `redirect`: перед каждым переходом проверяется

состояние входа, и неавторизованный пользователь перенаправляется на экран входа. Так реализуется требование «для доступа к закрытым разделам требуется авторизация» в одном месте приложения. `StatefulShellRoute` сохраняет состояние каждой вкладки нижней навигации при переключении.

1.5.7 Авторизация: хэш пароля с солью и токен входа

Учётные записи хранятся в базе данных на сервере. Пароль не сохраняется в открытом виде: при регистрации генерируется случайная соль длиной 16 байт, вычисляется хэш SHA-256 [8] от соли и пароля, в базу записываются соль и хэш. При входе хэш вычисляется заново и сравнивается с сохранённым. Соль делает хэши одинаковых паролей разными.

Так была устроена первая версия приложения (1 семестр). Однако SHA-256 вычисляется очень быстро, и если файл базы скопируют, простые пароли можно подобрать перебором. Для хранения паролей OWASP рекомендует медленные функции: Argon2id, scrypt, bcrypt, PBKDF2 [35]. Во второй версии приложения выбран PBKDF2-HMAC-SHA256 [39]: функция HMAC-SHA256 применяется к паролю и соли 100 000 раз. PBKDF2 реализуется на основе того же пакета crypto, поэтому дополнительные зависимости не нужны. Хэши первой версии пересчитываются автоматически при входе пользователя (п. 2.6.1). Пароль проверяет сервер; после успешного входа приложение получает токен (п. 1.5.5) и больше не передаёт пароль до следующего входа.

1.5.8 Дополнительные пакеты

- **fl_chart** [19] — построение столбчатых и круговых диаграмм для экрана статистики.
- **flutter_local_notifications** [27] — локальные уведомления: ежедневное напоминание о занятиях и сигнал окончания интервала Pomodoro. Уведомления формируются на устройстве и не требуют сервера.

- **http** [31], **shelf** и **shelf_router** [44] — отправка запросов из приложения и HTTP-сервер API (п. 1.5.5).
- **Material 3** [33] — система дизайна с поддержкой светлой и тёмной темы. Для приложения выбрана фирменная цветовая схема: тёмно-синий основной цвет и оранжевый акцентный.

Таблица 1.6 — Итоговый стек технологий

Задача	**Технология**
Язык и фреймворк	Dart, Flutter
Архитектура	Clean Architecture: domain / data / presentation
Управление состоянием	Riverpod
Хранение данных	собственный сервер API на Dart (shelf, shelf_router), база drift (SQLite) на сервере
Обмен данными	REST API, JSON, пакет http, вход по токену (Bearer)
Простые настройки	shared_preferences
Навигация	go_router с redirect
Хэширование паролей	PBKDF2-HMAC-SHA256 с солью (пакет crypto)
Графики	fl_chart
Уведомления	flutter_local_notifications
Дизайн	Material 3, светлая и тёмная темы
Тестирование	flutter_test, integration_test

Выводы по главе 1

1. Задача составления персонального плана формализована как работа с ориентированным ациклическим графом тем. Порядок изучения находится топологической сортировкой алгоритмом Кана за время O(V + E). Алгоритм позволяет также обнаружить ошибки в данных (циклы).
2. Эффект распределённого повторения обосновывает автоматическое планирование повторений, а техника Pomodoro — встроенный таймер концентрации.
3. Рассмотрены особенности мобильных приложений: ограниченный экран, разные размеры и ориентации, прерывания, работа без сети, уведомления, ограниченные ресурсы, безопасность данных. Сформулированы принципы разработки, которые учитывают эти особенности.

4. Анализ аналогов показал, что существующие решения закрывают задачу частично. Ни одно не объединяет граф тем, учёт знаний, расписание под время и срок пользователя и автоматическую перестройку плана.
5. Обоснован выбор технологий: Flutter и Dart, Clean Architecture, Riverpod, собственный сервер REST API на Dart с базой drift (SQLite), go_router, хэширование паролей PBKDF2 с солью, вход по токену, fl_chart, flutter_local_notifications, Material 3. Стек позволяет создать приложение, которое работает с данными только через API, не зависит от облачных сервисов и хорошо тестируется.

ГЛАВА 2. РАЗРАБОТКА ПРИЛОЖЕНИЯ «МАРШРУТ»

2.1 Требования к приложению

На основе анализа предметной области и аналогов (глава 1) сформулированы требования к приложению.

Функциональные требования приведены в таблице 2.1.

Таблица 2.1 — Функциональные требования

№	Требование	Где реализовано
Ф1	Регистрация по имени, email и паролю с проверкой полей; вход; автовход при «Запомнить меня»; выход	Экраны входа и регистрации, `AuthController`
Ф2	Закрытые разделы доступны только после входа	Правило `authRedirect` в `app_router.dart`
Ф3	Выбор направления обучения из каталога	Шаг 1 онбординга
Ф4	Диагностический тест: известные темы исключаются из плана	Шаг 2 онбординга, `KnowledgeEstimator`
Ф5	Настройка расписания: часы в неделю, учебные дни, дедлайн	Шаг 3 онбординга, экран настроек плана
Ф6	Автоматическое построение плана по дням с учётом зависимостей тем, лимита часов и интервальных повторений	`PlanGenerator`, `Scheduler`
Ф7	Проверка дедлайна и подсказка о нужном числе часов	`PlanGenerator`, метод `requiredHoursPerWeek`
Ф8	Отметка занятий, перестройка плана при отставании	Главная, «План», `Replanner`
Ф9	Несколько планов, выбор активного, переименование, удаление	«Мои планы»
Ф10	Свои темы, избранное, поиск и фильтр по категориям	Вкладка «Темы»
Ф11	Заметки к темам с поиском	«Заметки»
Ф12	Статистика: часы по неделям, распределение по категориям, история	Вкладка «Статистика»
Ф13	Таймер Pomodoro с учётом выполненных интервалов	Экран Pomodoro
Ф14	Ежедневное напоминание о занятиях	Профиль, `ReminderService`
Ф15	Профиль: редактирование данных, смена пароля, тема оформления, удаление аккаунта	Раздел «Профиль»
Ф16	Работа с данными только через API сервера: получение, добавление, изменение и удаление; обработка ответов и ошибок; настройка адреса сервера	Сервер `lib/server`, клиент и репозитории `lib/data/api`, кнопка «Сервер» на экране входа

Нефункциональные требования:

- данные хранятся в базе SQLite на собственном сервере API, приложение обращается к ним только через REST API; сервер работает в локальной сети без облачных сервисов и ключей;
- хранение пароля только в виде хэша с солью, доступ к данным — по токену входа;
- понятные сообщения при ошибках сервера и отсутствии связи;
- платформы: Android и iPhone (основные), Windows и Linux (настольная версия); сервер API — Windows и Linux;
- адаптивный интерфейс: телефон в обеих ориентациях и окно компьютера;
- интерфейс на русском языке в стиле Material 3, фирменные цвета — тёмно-синий и оранжевый;
- покрытие алгоритма и данных автоматическими тестами.

2.2 Архитектура приложения

Система состоит из двух программ: мобильного приложения и сервера API (рисунок 2.1). Приложение построено по принципам чистой архитектуры [2] и рекомендациям команды Flutter по архитектуре приложений [22] и разделено на три слоя. Сервер написан на том же языке Dart и использует общие с приложением сущности, правила проверки данных и формат JSON.

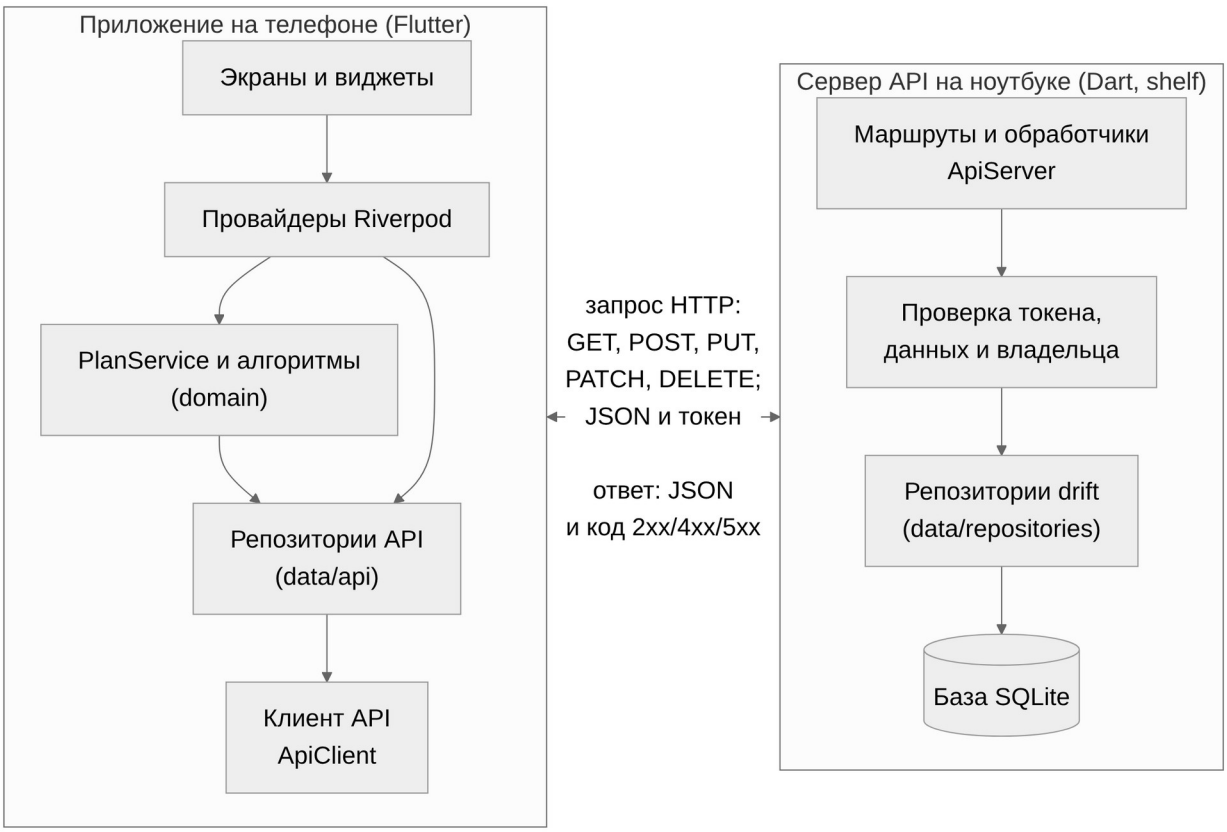


Рисунок 2.1 — Архитектура: приложение, сервер API и база данных

1. **Слой предметной области** (`lib/domain`) написан на чистом Dart и не зависит от Flutter и базы данных. В нём находятся сущности (тема, план, занятие, настройки расписания), алгоритм построения плана и интерфейсы репозиториев, то есть описание того, какие данные нужны алгоритмам, без указания, где они хранятся. Поэтому алгоритм проверяется обычными модульными тестами за доли секунды.
2. **Слой данных** (`lib/data`) реализует интерфейсы репозиториев. В приложении используются репозитории из папки `lib/data/api`: каждый метод отправляет запрос к серверу через общий клиент `ApiClient`. Таблицы drift и репозитории для работы с базой (`lib/data/db`, `lib/data/repositories`) используются на сервере. Здесь же чтение каталога, уведомления и хранение сессии в `shared_preferences`.
3. **Слой представления** (`lib/presentation`) содержит экраны и провайдеры Riverpod, которые связывают экраны с данными.

4. **Сервер API** (`lib/server/api_server.dart`, программа запуска `bin/marshrut_server.dart`) принимает запросы, проверяет токен и данные и работает с базой SQLite (п. 2.3.2).

Зависимости направлены внутрь: экраны знают о предметной области, слой данных реализует её интерфейсы, а предметная область ни от кого не зависит. Это подтвердилось на практике: при переходе от локальной базы на телефоне к серверу API в приложении заменились только реализации репозиториев, а алгоритм и экраны остались прежними.

Структура проекта приведена в таблице 2.2.

Таблица 2.2 — Структура проекта

Папка	**Назначение**
`lib/core/`	Тема оформления (`theme`), маршрутизация (`router`), валидаторы, форматирование дат и чисел, хэширование паролей (`utils`)
`lib/domain/ entities/`	Сущности: `Topic`, `Direction`, `StudyPlan`, `StudySession`, `ScheduleSettings`, `AppUser`, `Note`
`lib/domain/planner/`	Алгоритм: топологическая сортировка, раскладка по дням, генератор плана, перепланирование, оценка знаний по тесту
`lib/domain/ repositories/`	Интерфейсы репозиториев и сервисов
`lib/domain/ usecases/`	`PlanService`: сценарии создания плана, отметки занятий и перепланирования
`lib/data/api/`	Клиент API (`ApiClient`), репозитории приложения, которые обращаются к серверу, формат JSON
`lib/data/db/`	Описание таблиц drift и сгенерированный код запросов (используется сервером)
`lib/data/ repositories/`	Репозитории базы данных: пользователи, планы, темы, заметки (используются сервером); хранение сессии в приложении
`lib/server/`	Сервер API: маршруты, проверка токена и данных, обработка ошибок
`bin/`	Программа запуска сервера `marshrut_server.dart`
`lib/data/ datasources/`	Чтение каталога направлений из JSON
`lib/data/services/`	Уведомления (flutter_local_notifications)
`lib/presentation/ providers/`	Провайдеры Riverpod и контроллеры состояния
`lib/presentation/ screens/`	Экраны, сгруппированные по разделам
`lib/presentation/ widgets/`	Общие виджеты: логотип, кольцо прогресса, пустые состояния, карточка занятия
`assets/catalog/`	Каталог: 4 направления, 75 тем, 32 вопроса диагностического теста
`test/`, `integration_ test/`	Автоматические тесты

Папка	Назначение
`tool/`	Генераторы скриншотов и иконок, пример работы алгоритма, файл запуска сервера для Windows

Объём исходного кода без сгенерированных файлов — около 15 200 строк на Dart вместе с подробными комментариями на русском языке (из них сервер и клиент API — около 2 000 строк), тестов и вспомогательных программ — около 4 100 строк.

Управление состоянием и внедрение зависимостей. Все зависимости объявлены провайдерами Riverpod [42] в файле `lib/presentation/providers/providers.dart`. Внешние ресурсы (HTTP-клиент, настройки, уведомления, часы) объявлены как провайдеры, которые переопределяются при запуске: в `main()` — настоящими объектами, в тестах — HTTP-клиентом, который передаёт запросы серверу API в памяти, заглушкой уведомлений и фиксированной датой. Данные с сервера передаются в интерфейс потоками (`StreamProvider`): когда пользователь отмечает занятие, клиент API после успешного ответа сервера сообщает об изменении данных, и главная, план и статистика загружают данные заново без ручного обновления экранов (п. 2.3.3).

2.3 Проектирование базы данных и API сервера

2.3.1 База данных

Данные хранятся в базе SQLite на сервере API, работа с ней ведётся через библиотеку drift [15]. Приложение к базе напрямую не обращается: все чтения и изменения выполняет сервер по запросам API (п. 2.3.2). Структура базы показана на рисунке 2.2: на диаграмме оставлены ключи и основные поля, чтобы её можно было прочитать на странице. Назначение таблиц описано в таблице 2.3, полное описание всех полей — в приложении Е.

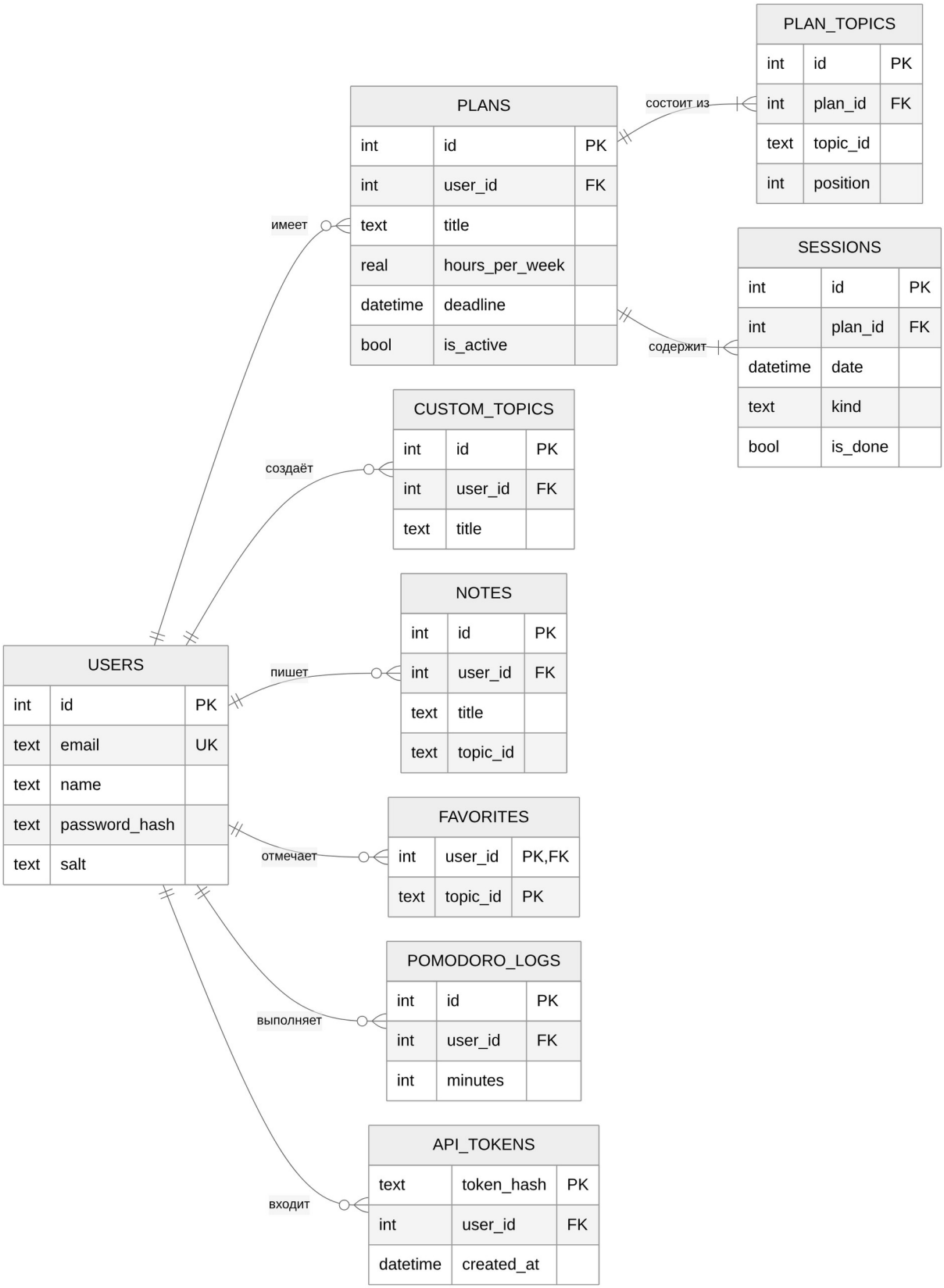


Рисунок 2.2 — ER-диаграмма базы данных (ключи и основные поля)

Таблица 2.3 — Таблицы базы данных

Таблица	**Назначение**
`users`	Учётные записи. Email уникален; пароль хранится как хэш PBKDF2 и соль; настройки напоминания
`plans`	Планы обучения. У пользователя может быть несколько планов, активен один (`is_active`). Хранятся настройки расписания и темы, исключённые по тесту
`plan_topics`	Темы плана в порядке изучения (`position`). Это снимок темы на момент создания плана, поэтому изменение каталога в новой версии приложения не ломает уже созданные планы
`sessions`	Занятия: дата, длительность, вид (`study` — изучение, `review` — повторение), номер части темы, отметка о выполнении и её время
`custom_topics`	Собственные темы пользователя с длительностью, категорией и зависимостями
`notes`	Заметки; могут быть привязаны к теме
`favorites`	Избранные темы; составной первичный ключ (пользователь, тема) исключает повторы
`pomodoro_logs`	Завершённые интервалы Pomodoro для статистики
`api_tokens`	Токены входа в API: хранится только хэш SHA-256 токена, пользователь и время выдачи

Все внешние ключи объявлены с правилом `ON DELETE CASCADE`, а при открытии базы включается проверка внешних ключей (`PRAGMA foreign_keys = ON`). Поэтому удаление плана удаляет его темы и занятия, а удаление аккаунта — все данные пользователя и его токены одной командой. Это проверено тестами (раздел 2.7).

Таблица `api_tokens` появилась во второй версии схемы. Номер версии хранится в самой базе SQLite, и при открытии базы версии 1 drift выполняет миграцию: создаёт новую таблицу, не затрагивая существующие данные.

Каталог направлений не хранится в базе: это JSON-файлы в папке `assets/catalog/`. Каждая тема описана идентификатором, названием, описанием, трудоёмкостью в часах, уровнем сложности (1–6), категорией, списком зависимостей, ссылками на материалы и, для части тем, вопросом диагностического теста. Такой формат позволяет добавить новое направление без изменения кода. В каталоге 4 направления: «Flutter-разработчик» (20 тем), «Python с нуля» (18), «Английский A1–B1» (19) и «Алгоритмы и структуры данных» (18); всего 75 тем, 113 зависимостей и 32 вопроса.

Каталог читает сервер при запуске и отдаёт приложению по адресу `GET /api/directions`. В приложении в `shared_preferences` хранятся только адрес сервера, идентификатор пользователя и токен входа (если включено «Запомнить меня») и выбранная тема оформления.

2.3.2 API сервера

Сервер API — программа на Dart (`bin/marshrut_server.dart`), которая запускается на компьютере и принимает запросы по протоколу HTTP на порту 8080. Обработчики запросов собраны в классе `ApiServer` (`lib/server/api_server.dart`), построенном на библиотеке shelf [44]. Каждый запрос проходит цепочку обработки:

1. **журнал** — после ответа в окно сервера записывается строка: время, метод, адрес, код ответа и время обработки;
2. **перехват ошибок** — любое исключение превращается в ответ JSON с кодом ошибки (коды перечислены ниже), подробности непредвиденных ошибок попадают только в журнал сервера;
3. **маршрутизатор** (shelf_router) — по методу и адресу выбирает обработчик;
4. **обработчик** — определяет пользователя по токену, читает и проверяет JSON-тело запроса, проверяет, что запись принадлежит этому пользователю, вызывает репозиторий drift и формирует ответ.

Адреса API организованы по ресурсам (таблица 2.4), полный список из 39 адресов приведён в приложении К.

Таблица 2.4 — Ресурсы API сервера

Ресурс	**Методы и адреса**	**Назначение**
Служебные	`GET /api/health`, `GET /api/directions`	Проверка связи, каталог направлений и тем
Пользователь	`POST /api/auth/register`, `/login`, `/logout`; `GET`, `PUT`, `DELETE /api/me`; `PUT /api/me/password`, `/api/me/reminder`	Регистрация, вход с выдачей токена, выход, профиль, пароль, напоминание, удаление аккаунта
Планы	`GET`, `POST /api/plans`; `GET /api/plans/active`; `GET`, `PATCH`, `DELETE /api/plans/{id}`; `POST …/activate`; `PUT …/settings`; `GET`, `PUT …/topics`; `GET …/sessions`; `PUT …/sessions/open`	Полный CRUD планов, выбор активного, настройки, темы плана, перепланирование
Занятия	`GET /api/sessions/completed`, `GET /api/sessions/{id}`, `PUT /api/sessions/{id}/done`	История, карточка занятия, отметка о выполнении
Свои темы	`GET`, `POST /api/custom-topics`; `GET`, `PUT`, `DELETE /api/custom-topics/{id}`	Полный CRUD своих тем
Избранное	`GET /api/favorites`, `POST /api/favorites/{id}/toggle`	Список и переключение избранного
Заметки	`GET`, `POST /api/notes`; `GET`, `PUT`, `DELETE /api/notes/{id}`	Полный CRUD заметок, фильтр по теме
Pomodoro	`GET`, `POST /api/pomodoros`	Журнал завершённых интервалов

Пример запроса на создание заметки и ответа сервера:

```
POST /api/notes
Authorization: Bearer <токен>
Content-Type: application/json

{"title": "Списки в Python", "body": "append добавляет элемент в конец"}

HTTP/1.1 201 Created
{"id": 1}
```

Сервер отвечает кодами HTTP [41], а текст ошибки передаёт в поле `error`:

- `200 OK` — данные получены или изменены; `201 Created` — запись создана, в ответе её номер; `204 No Content` — действие выполнено, тело ответа пустое;
- `400 Bad Request` — неверные данные: тело не в формате JSON, нет обязательного поля, поле другого типа, нарушено правило проверки («Введите корректный email…»), занятый email, неверный пароль;

- `401 Unauthorized` — нет токена, токен неизвестен, устарел (действует 30 дней) или удалён при выходе;
- `404 Not Found` — запись не найдена или принадлежит другому пользователю: сервер не сообщает, что чужая запись существует;
- `500 Internal Server Error` — непредвиденная ошибка, например сбой базы; приложение получает только общий текст.

Меры защиты на сервере:

- пароль хранится как хэш PBKDF2 с солью (п. 2.6.1), токен — как хэш SHA-256 [8], поэтому копия файла базы не даёт ни паролей, ни действующих токенов;
- токен — 32 случайных байта из криптостойкого генератора `Random.secure()`, при выходе он удаляется из базы;
- сервер повторно проверяет все поля теми же функциями `Validators`, что и формы приложения, поэтому неверные данные не попадут в базу, даже если запрос отправлен в обход приложения;
- каждый обработчик проверяет владельца записи, поэтому пользователь не может прочитать, изменить или удалить чужой план, заметку или тему, даже зная её номер.

Ниже приведён фрагмент журнала сервера при работе с заметками: ошибочный вход, вход, получение активного плана, создание, изменение и удаление заметки, а также ответы на запрос без обязательного поля, запрос без токена и запрос к удалённой заметке.

```
06:01:45  POST   /api/auth/login  →  400  (491 мс)
06:01:46  POST   /api/auth/login  →  200  (671 мс)
06:01:46  GET    /api/plans/active  →  200  (0 мс)
06:01:46  POST   /api/notes  →  201  (3 мс)
06:01:46  PUT    /api/notes/1  →  204  (2 мс)
06:01:46  DELETE /api/notes/1  →  204  (2 мс)
06:01:46  POST   /api/notes  →  400  (0 мс)
06:01:46  GET    /api/notes  →  401  (0 мс)
06:01:46  GET    /api/notes/1  →  404  (0 мс)
```

Вход занимает около половины секунды: столько длится расчёт PBKDF2 со 100 000 итераций, и это замедляет подбор паролей. Остальные запросы выполняются за единицы миллисекунд.

Текст ключевых частей сервера (маршруты, проверка токена, перехват ошибок) приведён в приложении К.

2.3.3 Работа приложения с API и обработка ошибок

Все запросы приложения проходят через класс `ApiClient` (`lib/data/api/api_client.dart`), построенный на пакете http [31]. Он приводит адрес сервера к виду `http://хост:порт`, добавляет к запросу заголовок `Authorization: Bearer <токен>`, переводит тело в JSON, ждёт ответ не дольше 10 секунд и разбирает ответ в кодировке UTF-8. Любая неудача превращается в исключение с понятным русским текстом:

- сервер выключен, нет сети или неверный адрес — «Нет связи с сервером … Проверьте, что сервер запущен и телефон в той же сети Wi-Fi»;
- сервер не ответил за 10 секунд — «Сервер … не отвечает»;
- ответ с кодом 4xx или 5xx — текст из поля `error` ответа сервера;
- ответ не в формате JSON — сообщение с кодом ответа;
- ответ `401` — дополнительно вызывается обработчик `onUnauthorized`: контроллер авторизации выполняет выход, и роутер открывает экран входа.

Репозитории приложения (`ApiPlanRepository`, `ApiNoteRepository` и другие) реализуют те же интерфейсы, что и прежние репозитории базы данных. Ответ `404` там, где интерфейс допускает отсутствие записи, превращается в `null`, поэтому экраны не потребовали изменений.

Ошибки показываются пользователю двумя способами. Если не загрузились данные экрана, вместо них выводится сообщение об ошибке с кнопкой «Повторить». Если не удалось действие (сохранение, удаление, отметка занятия, избранное), общая функция `runAction` показывает красное всплывающее сообщение с причиной, а данные остаются прежними. Встроенный в Riverpod автоматический повтор упавших запросов отключён:

иначе при выключенном сервере экран около 40 секунд показывал бы загрузку вместо понятной ошибки.

Чтобы экраны обновлялись сами, клиент API после каждого успешного запроса на изменение (`POST`, `PUT`, `PATCH`, `DELETE`) отправляет событие «данные изменились». Потоки данных, созданные методом `ApiClient.watch`, по этому событию загружают данные заново, а после ошибки сами повторяют запрос через 5 секунд. Когда экран закрывается, поток отписывается от событий и отменяет повтор.

Адрес сервера вводится на экране входа кнопкой «Сервер: …» (рисунок 2.30). Перед сохранением приложение отправляет запрос `GET /api/health` и сохраняет адрес, только если ответил именно сервер «Маршрута». Для работы по протоколу HTTP в локальной сети в Android-приложении объявлено разрешение `INTERNET` и разрешён нешифрованный трафик, а в iOS-приложении — подключения в локальной сети с пояснением для запроса разрешения. Для учебного проекта в локальной сети этого достаточно; при размещении сервера в интернете запросы нужно передавать по HTTPS.

Текст класса `ApiClient` приведён в приложении Л.

2.4 Алгоритм формирования плана

2.4.1 Общая схема

Алгоритм реализован в слое предметной области (`lib/domain/planner/`) и состоит из пяти шагов (рисунок 2.3).

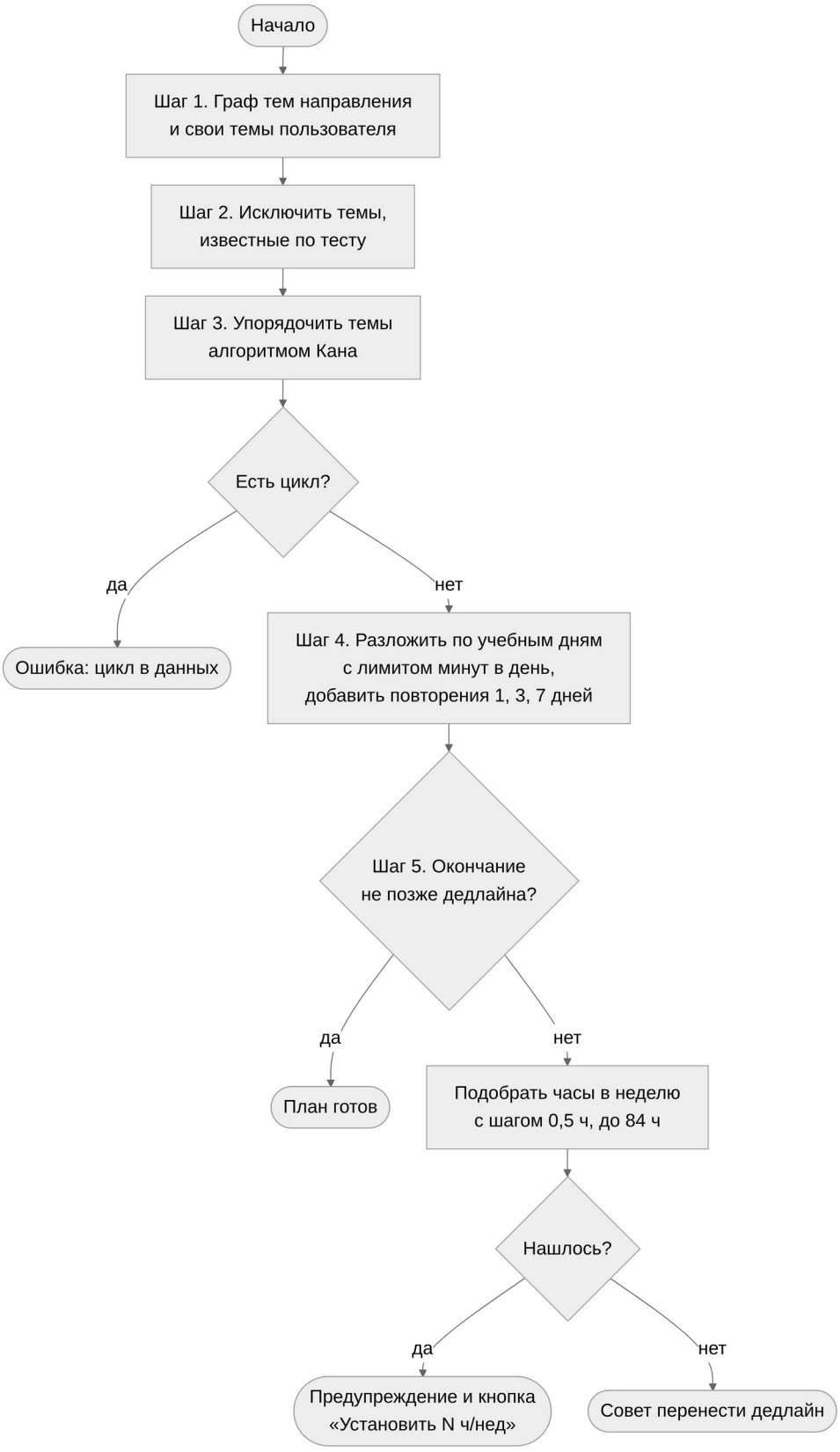


Рисунок 2.3 — Схема алгоритма формирования плана

2.4.2 Исключение известных тем

По ответам диагностического теста класс `KnowledgeEstimator` определяет известные темы. Верный ответ по теме означает, что пользователь знает и саму тему, и все её предшествующие темы: кто правильно ответил о Riverpod, знает и основы Dart. Предшественники находятся обходом графа в глубину. Неверный ответ или «Не знаю» исключает тему из известных, даже если её «покрыл» верный ответ по более сложной теме: явный провал важнее косвенного вывода.

2.4.3 Топологическая сортировка

Темы образуют ориентированный граф: ребро «A → B» означает, что тему B нельзя изучать раньше темы A. Порядок изучения — это топологическая сортировка графа, которая выполняется алгоритмом Кана [1; 6]:

1. Для каждой темы считается входящая степень — число её зависимостей внутри плана. Зависимость, которой нет в плане (тема уже известна), считается выполненной.
2. Темы с нулевой степенью помещаются в очередь.
3. Из очереди извлекается тема и добавляется в результат; у всех зависящих от неё тем степень уменьшается на единицу, и темы с нулевой степенью попадают в очередь.
4. Если в результат попали не все темы, в графе есть цикл, и алгоритм сообщает, какие темы в него входят.

Чтобы результат был одинаковым при одинаковых данных, очередь упорядочена по уровню сложности темы, а при равном уровне — по позиции темы в каталоге. Для этого используется упорядоченное множество `SplayTreeSet`, поэтому сложность алгоритма — $O((V + E) \cdot \log V)$, где V — число тем, E — число зависимостей. Для каталога из 20 тем это мгновенно. Фрагмент реализации:

```
final queue = SplayTreeSet<int>(compare);
for (var i = 0; i < topics.length; i++) {
```

```
  if (inDegree[i] == 0) queue.add(i);
}
final result = <Topic>[];
while (queue.isNotEmpty) {
  final current = queue.first;
  queue.remove(current);
  result.add(topics[current]);
  for (final next in dependents[current]) {
    inDegree[next]--;
    if (inDegree[next] == 0) queue.add(next);
  }
}
if (result.length != topics.length) throw CycleDetectedException(...);
```

Полный текст функции приведён в приложении А.

2.4.4 Раскладка по дням и интервальные повторения

Класс `Scheduler` моделирует календарь день за днём, начиная с даты старта:

1. Дни недели, не выбранные пользователем, пропускаются.
2. Дневной лимит равен часам в неделю, делённым на число учебных дней. Он округляется до 15 минут и не бывает меньше 30 минут.
3. В начале дня ставятся повторения, срок которых наступил, по 15 минут каждое.
4. Оставшееся время заполняется текущей темой. Тема может делиться на части, но часть не бывает короче 30 минут, если тема на ней не заканчивается: короткие обрывки неудобны для занятия.
5. Когда тема изучена полностью, планируются её повторения через 1, 3 и 7 календарных дней. Интервалы выбраны по кривой забывания Эббингауза [5] и исследованиям распределённого повторения [3]. Повторение, выпавшее на неучебный день, переносится на ближайший учебный.

2.4.5 Проверка дедлайна

Датой окончания плана считается день последнего занятия по новой теме (повторения после него не учитываются). Если дата окончания позже дедлайна, генератор подбирает минимальное число часов в неделю, при котором план укладывается в срок: значения перебираются с шагом 0,5 часа

до 84 часов в неделю (12 часов × 7 дней). Если не хватает даже 84 часов, пользователю предлагается перенести дедлайн.

2.4.6 Пример работы алгоритма

Пусть в плане три темы по 60 минут: «Основы Dart», «Виджеты» (зависит от «Основ Dart») и «Состояние» (зависит от «Виджетов»). Пользователь выбрал 7 часов в неделю, все дни недели, старт 12.10.2026 и дедлайн 14.10.2026. Дневной лимит: 7 · 60 / 7 = 60 минут. Результат работы алгоритма приведён в таблице 2.5 (получен программой `tool/print_schedule_example.dart`).

Таблица 2.5 — Пример расписания

Дата	**Занятие**	**Минут**
12.10	Основы Dart	60
13.10	Повторение: Основы Dart	15
13.10	Виджеты, часть 1 из 2	45
14.10	Виджеты, часть 2 из 2	15
14.10	Состояние, часть 1 из 2	45
15.10	Повторение: Основы Dart	15
15.10	Повторение: Виджеты	15
15.10	Состояние, часть 2 из 2	15
16.10	Повторение: Состояние	15
17.10	Повторение: Виджеты	15
18.10	Повторение: Состояние	15
19.10	Повторение: Основы Dart	15
21.10	Повторение: Виджеты	15
22.10	Повторение: Состояние	15

Повторения занимают часть дневного лимита, поэтому изучение заканчивается 15.10, на день позже дедлайна. Алгоритм сообщает, что план не укладывается в срок, и предлагает 8 часов в неделю: тогда лимит становится 75 минут, и последняя тема изучается 14.10.

2.4.7 Перепланирование

Если пользователь пропустил занятия или изменил расписание, класс `Replanner` перестраивает план. Выполненные занятия остаются на своих местах. Для каждой темы считается, сколько минут уже изучено, и остаток заново раскладывается начиная с сегодняшнего дня по тем же правилам.

Невыполненные повторения уже изученных тем переносятся на ближайшие учебные дни. Номера частей продолжают уже выполненные части. После перестройки снова проверяется дедлайн.

2.5 Интерфейс приложения

2.5.1 Навигация

Карта экранов показана на рисунке 2.4. Основные разделы доступны через пять вкладок: «Главная», «План», «Темы», «Статистика», «Профиль». Вкладки построены на `StatefulShellRoute` из go_router [29], поэтому каждая вкладка сохраняет своё состояние (прокрутку, фильтры) при переключении.

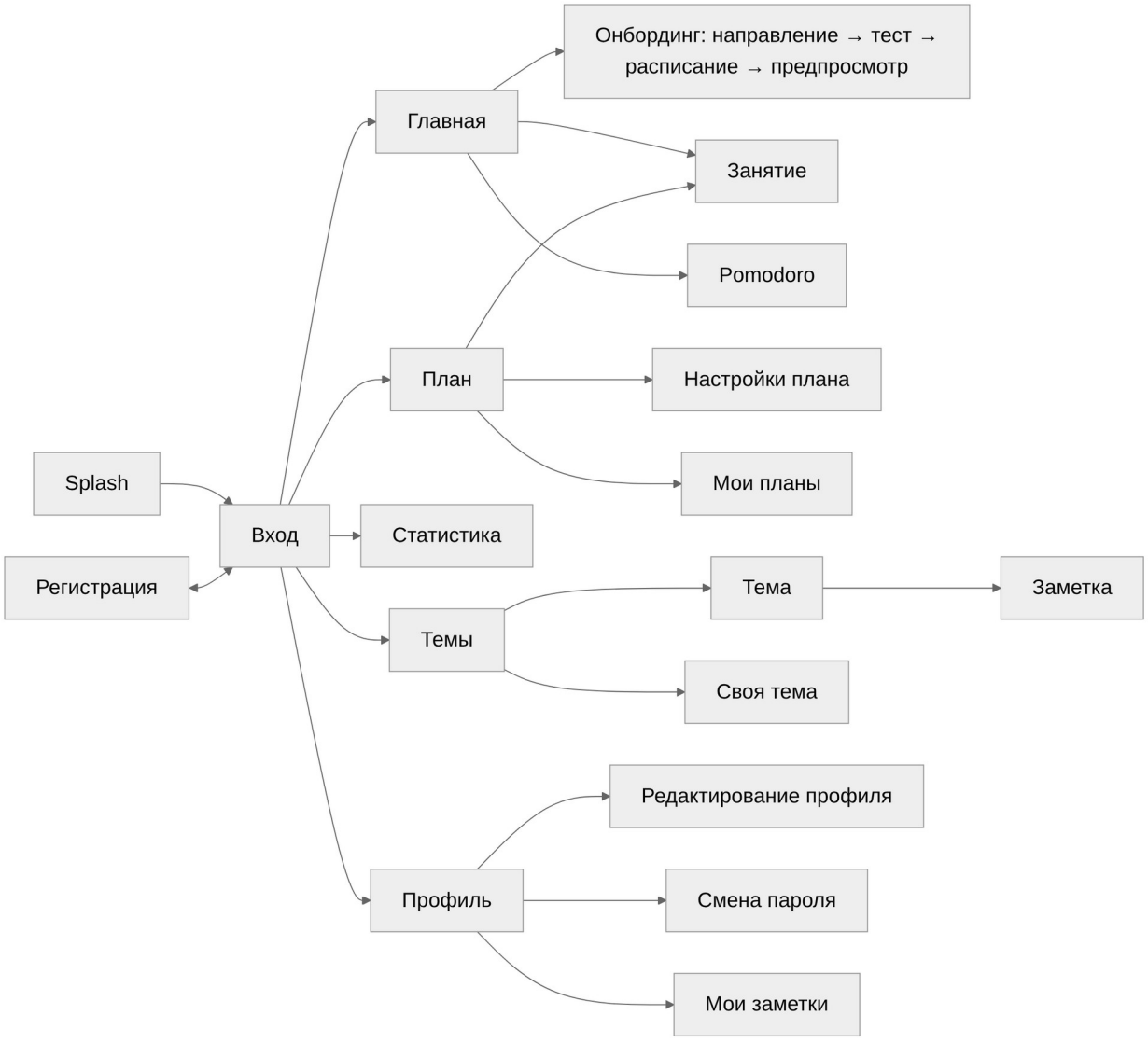


Рисунок 2.4 — Карта экранов

Доступ к экранам проверяет одна функция `authRedirect`, которую go_router вызывает перед каждым переходом:

```
String? authRedirect(AuthStatus status, String location) {
  if (status == AuthStatus.unknown) {
    return location == '/splash' ? null : '/splash';
  }
  final loggedIn = status == AuthStatus.authenticated;
  final isPublic = publicRoutes.contains(location);
  if (!loggedIn) return isPublic ? null : '/login';
  if (isPublic || location == '/splash') return '/home';
  return null;
}
```

Пока сессия проверяется, виден только Splash; без входа доступны только вход и регистрация; после входа экраны входа ведут на главную. Роутер подписан на состояние авторизации, поэтому выход из аккаунта или его удаление сразу открывают экран входа.

2.5.2 Оформление

Интерфейс построен на Material 3 [33]. Цветовая схема создаётся из фирменного тёмно-синего цвета (#1A3A8F), акцентный цвет — оранжевый (#F57C00). Есть светлая и тёмная темы; пользователь выбирает светлую, тёмную или системную тему в профиле. Стили текста строятся от типографики темы, поэтому шрифт и размеры одинаковы на всех экранах. Логотип «М» в виде линии маршрута нарисован кодом (`AppLogo`), иконки приложения для всех платформ генерируются из того же рисунка (`tool/icons/generate_icons.py`).

Пустые состояния (нет плана, нет заметок, ничего не найдено) показывают пояснение и кнопку следующего действия. Ошибки ввода выводятся под полями форм, ошибки операций — во всплывающих сообщениях. Необратимые действия (удаление плана, темы, заметки, выход, удаление аккаунта) требуют подтверждения.

2.5.3 Адаптивность

На широком экране (от 840 логических пикселей, класс «расширенное окно» по рекомендациям Android [50]) нижняя панель вкладок заменяется

боковой (`NavigationRail`), а содержимое экранов ограничивается колонкой шириной 720 пикселей по центру, чтобы строки текста не растягивались на весь монитор. Работа на телефоне в обеих ориентациях и в окне 1280×800 проверяется автоматическим тестом (раздел 2.7). Подписи вкладок не увеличиваются вместе с системным шрифтом, чтобы «Статистика» не переносилась по буквам, а остальной текст масштабируется. Перед сдачей все экраны дополнительно сняты в семи вариантах: светлая и тёмная тема, маленький телефон 360×640, крупный шрифт 130 %, горизонтальная ориентация и окно компьютера; по этим снимкам исправлено десять недочётов (ошибки № 13–22 в разделе о тестировании).

2.5.4 Экраны приложения

Экраны приложения перечислены в таблице 2.6 и показаны на рисунках 2.5–2.30. Скриншоты получены программой `tool/screenshots/screenshots_test.dart` на тестовых данных: пользователь Алия, план «Flutter-разработчик», начатый за четыре недели до текущей даты.

Таблица 2.6 — Экраны приложения

Экран	**Назначение**	**Рисунок**
Splash	Логотип и название, проверка сохранённой сессии	2.5
Вход	Email, пароль, «Запомнить меня», сообщение о неверных данных, кнопка адреса сервера API	2.6, 2.8
Регистрация	Имя, email, пароль и повтор с проверкой каждого поля	2.7
Главная	Приветствие, кольцо прогресса, часы, серия дней, дни до дедлайна, задачи на сегодня, баннер о пропущенных занятиях	2.9
План	Занятия по дням с фильтрами и поиском	2.10
Темы	Каталог направления, поиск, категории, избранное, свои темы	2.11
Тема	Описание, материалы, заметки, избранное	2.12
Статистика	Итоги, график часов по неделям, распределение по категориям, история	2.13
Профиль	Данные пользователя, показатели, адрес сервера API, настройки, выход	2.14
Мои планы	Список планов, выбор активного, переименование, перестройка, удаление	2.15

Экран	Назначение	Рисунок
Pomodoro	Таймер 25/5 или 50/10 минут, число интервалов за день	2.16
Заметки	Список заметок с поиском	2.17
Занятие	Описание темы, материалы, отметка выполнения, запуск Pomodoro	2.18
Настройки плана	Изменение расписания с перестройкой плана	2.19
Онбординг	Выбор направления, диагностический тест, расписание, предпросмотр	2.20–2.23
Тёмная тема	Главная и статистика в тёмной теме	2.24, 2.25
Окно Windows	Главная и план с боковой панелью вкладок	2.26, 2.27
Горизонтальная ориентация	Статистика на телефоне в горизонтальной ориентации	2.28
Удаление аккаунта	Подтверждение паролем	2.29
Адрес сервера API	Ввод адреса с проверкой связи перед сохранением	2.30

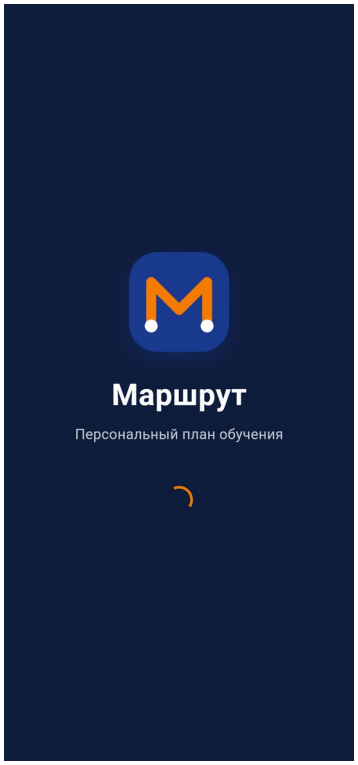


Рисунок 2.5 — Заставка

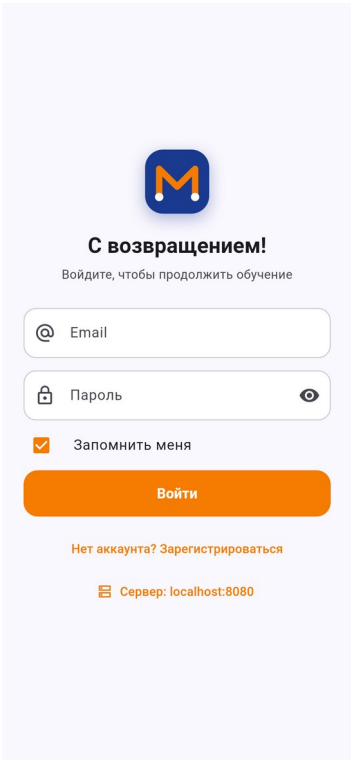


Рисунок 2.6 — Экран входа

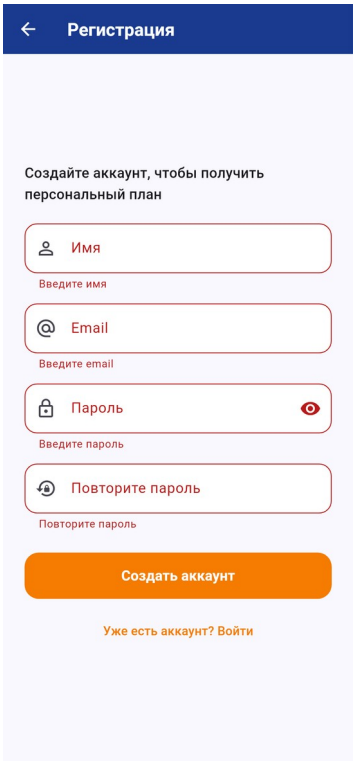


Рисунок 2.7 — Регистрация: сообщения об ошибках под полями

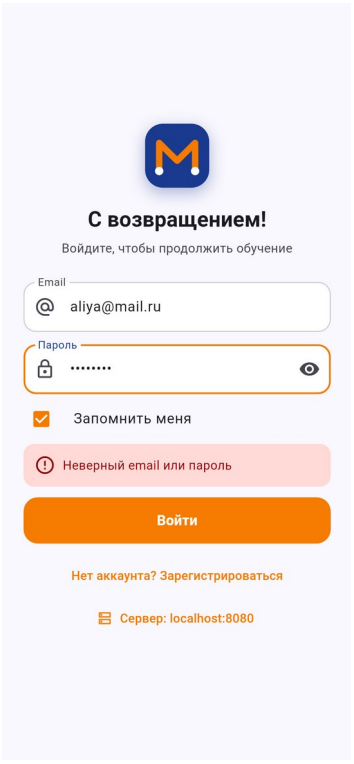


Рисунок 2.8 — Вход с неверным паролем

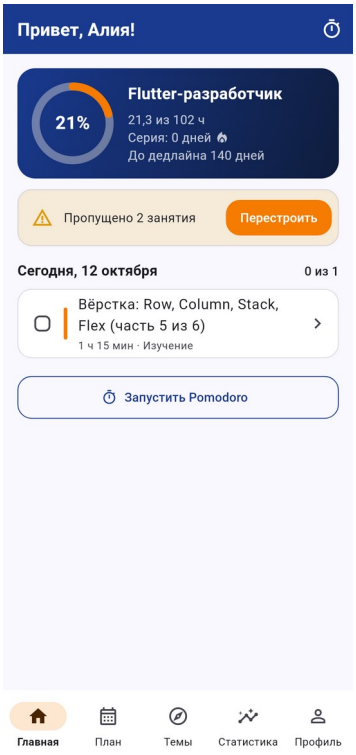


Рисунок 2.9 — Главная страница

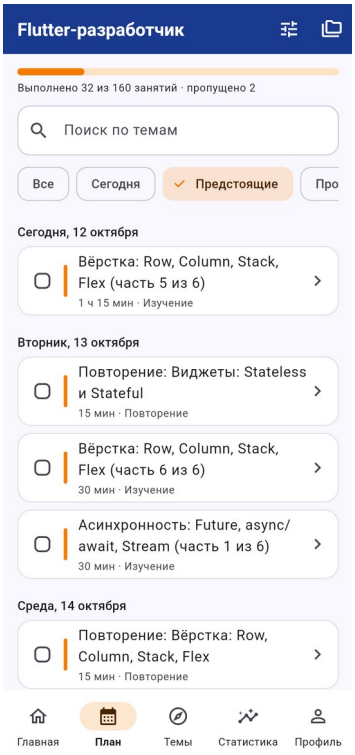


Рисунок 2.10 — План по дням

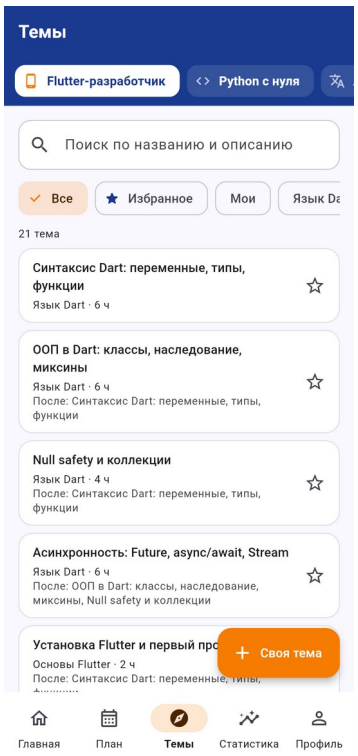


Рисунок 2.11 — Вкладка «Темы»

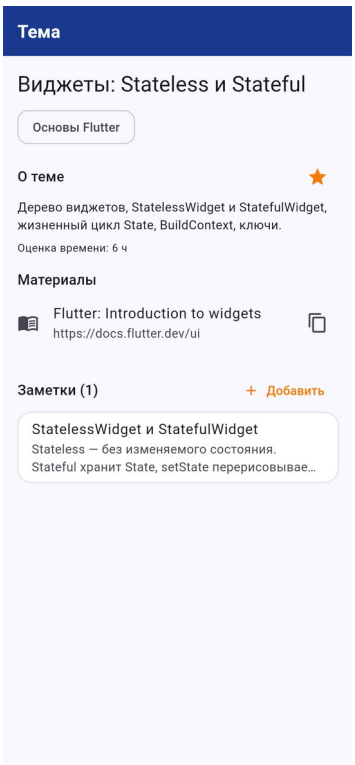


Рисунок 2.12 — Экран темы

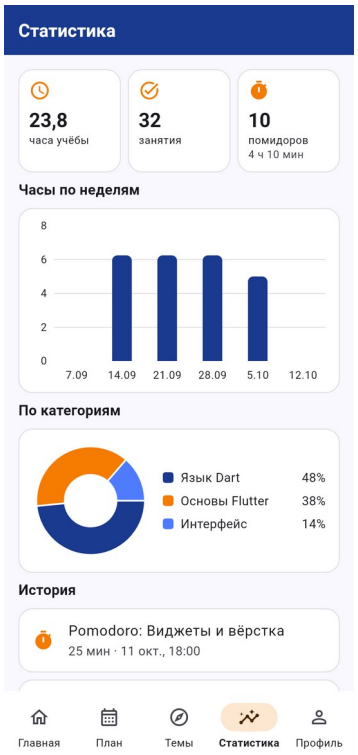


Рисунок 2.13 — Статистика

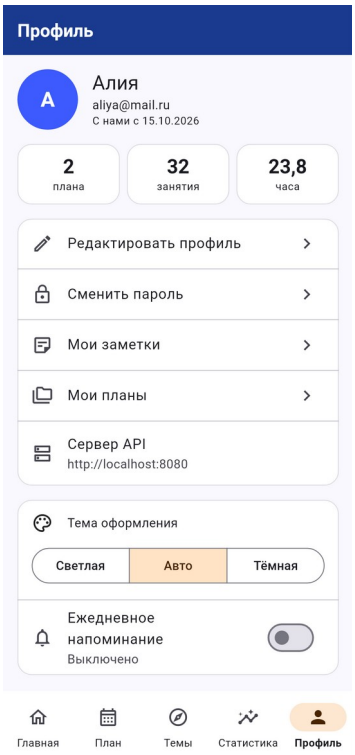


Рисунок 2.14 — Профиль

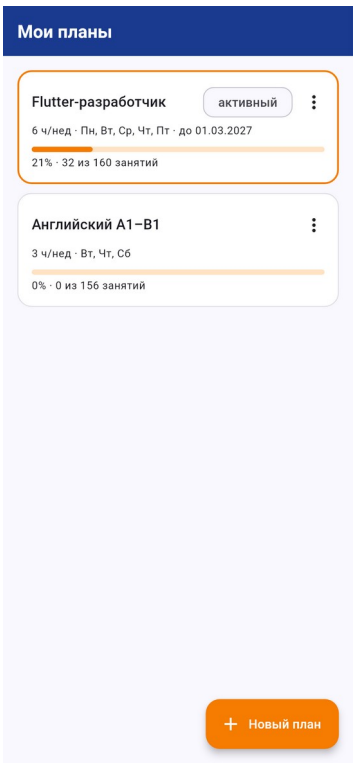


Рисунок 2.15 — «Мои планы»

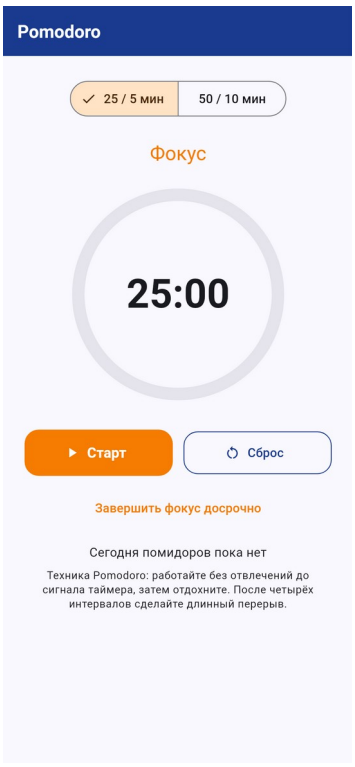


Рисунок 2.16 — Таймер Pomodoro

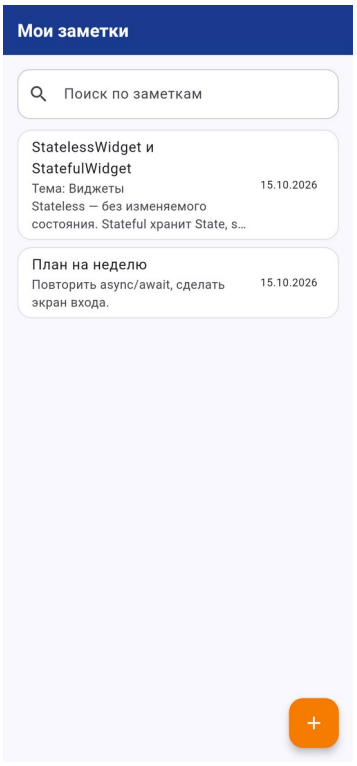


Рисунок 2.17 — Заметки

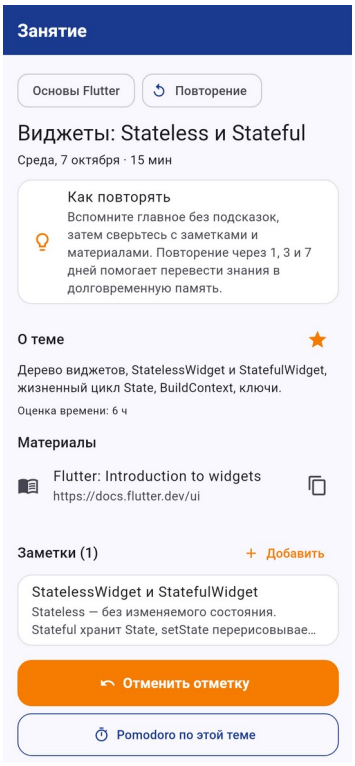


Рисунок 2.18 — Карточка занятия

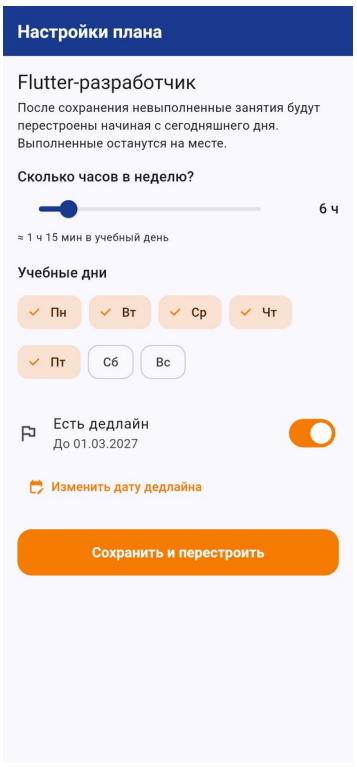


Рисунок 2.19 — Настройки плана

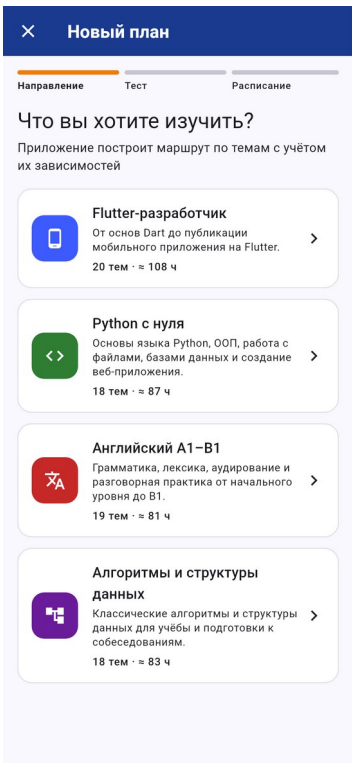


Рисунок 2.20 — Онбординг: выбор направления

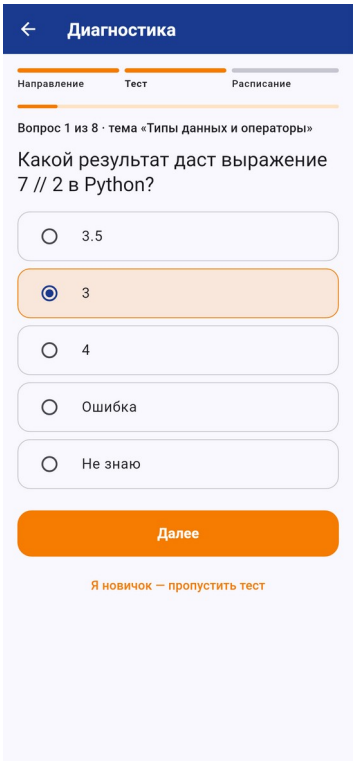


Рисунок 2.21 — Онбординг: диагностический тест

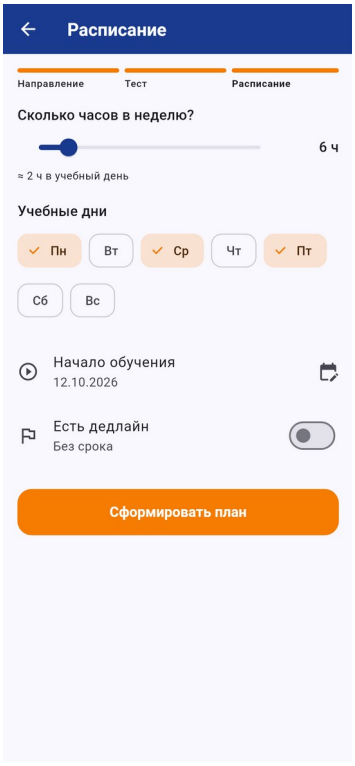


Рисунок 2.22 — Онбординг: расписание

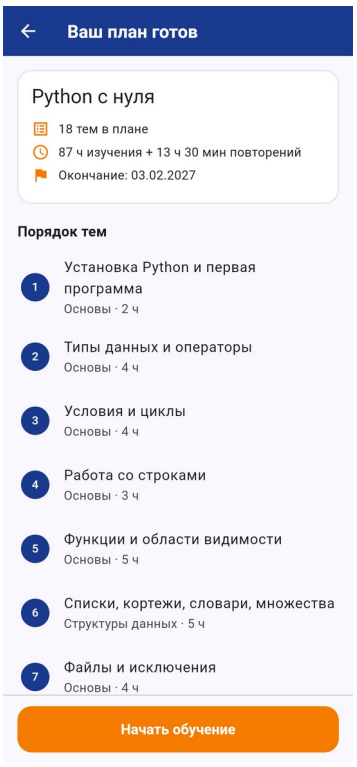


Рисунок 2.23 — Онбординг: предпросмотр плана

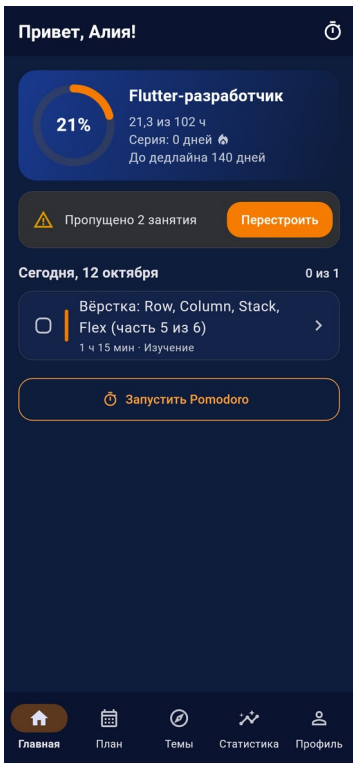


Рисунок 2.24 — Главная в тёмной теме

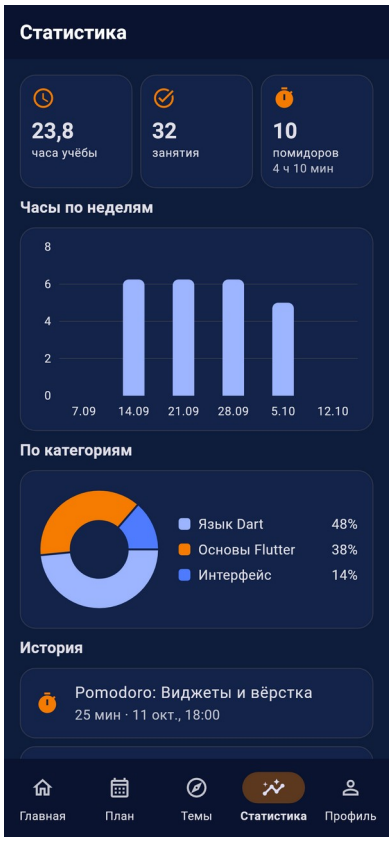


Рисунок 2.25 — Статистика в тёмной теме

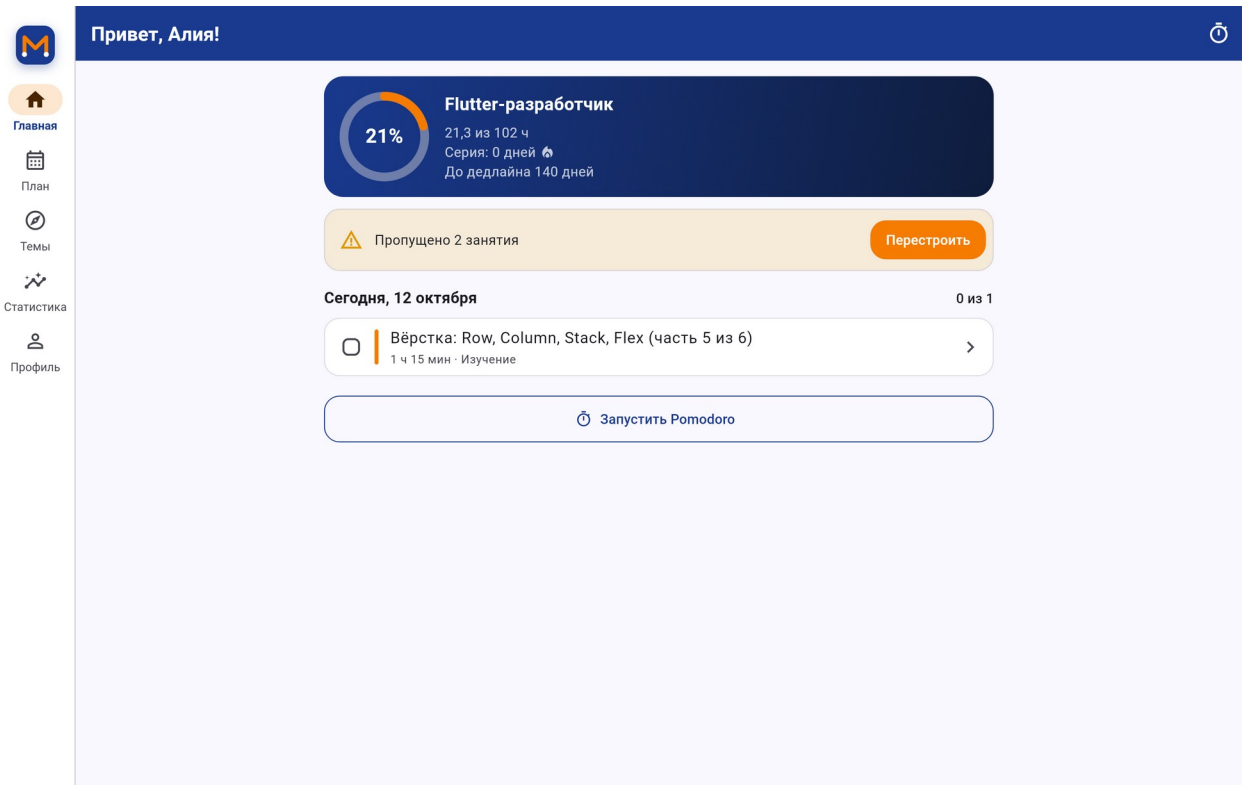


Рисунок 2.26 — Главная в окне компьютера

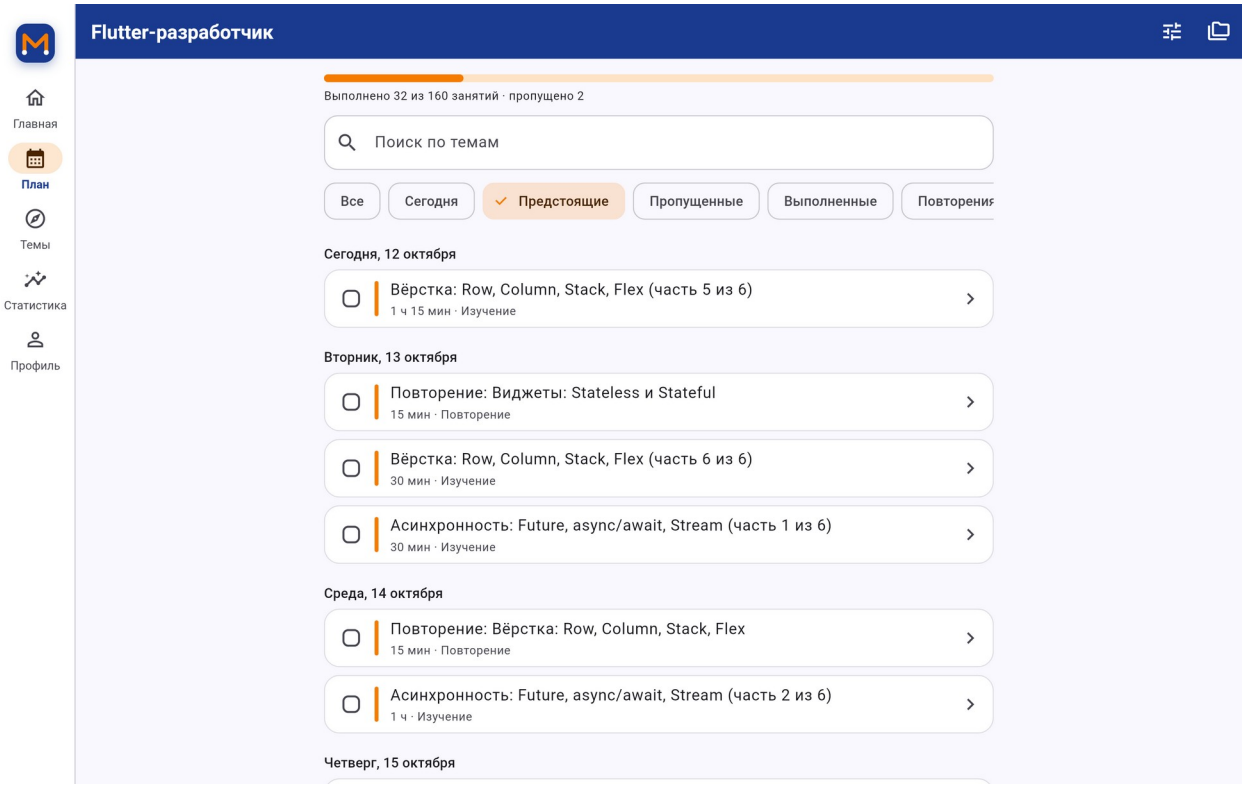


Рисунок 2.27 — План в окне компьютера

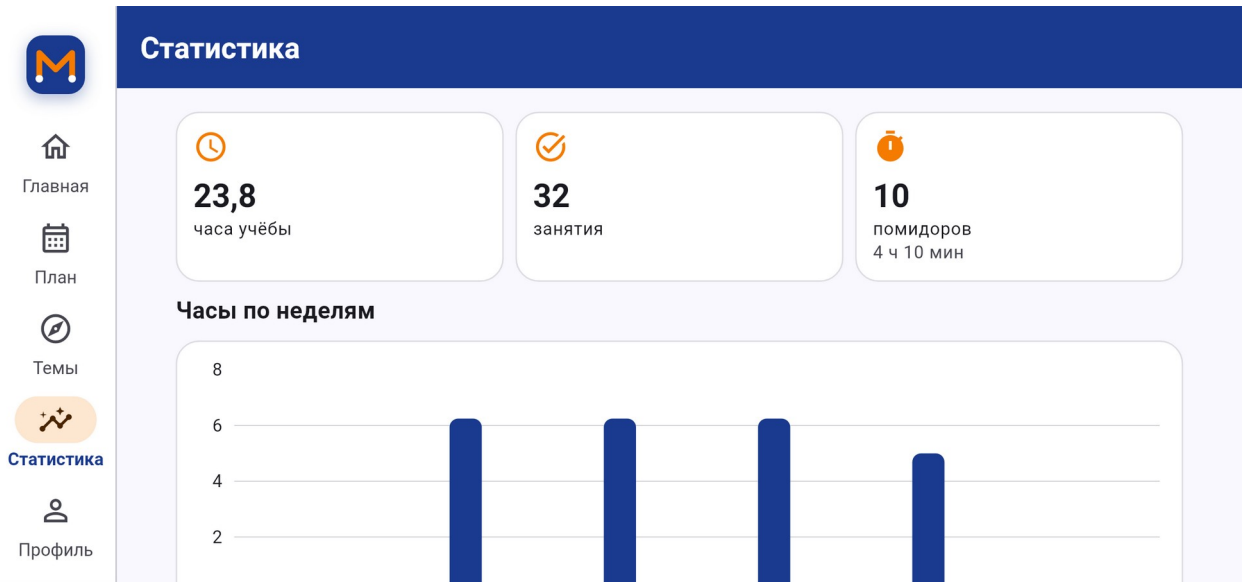


Рисунок 2.28 — Статистика в горизонтальной ориентации

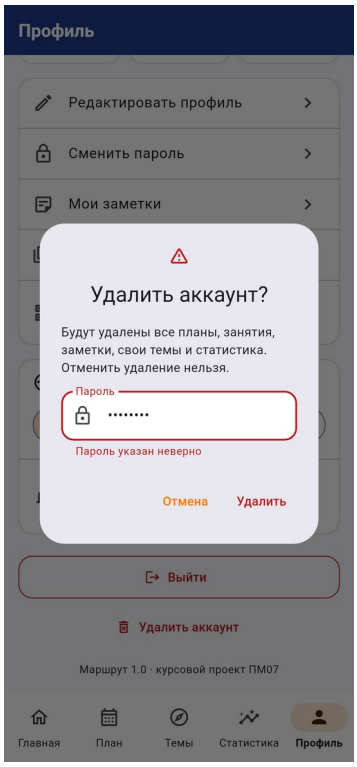


Рисунок 2.29 — Удаление аккаунта

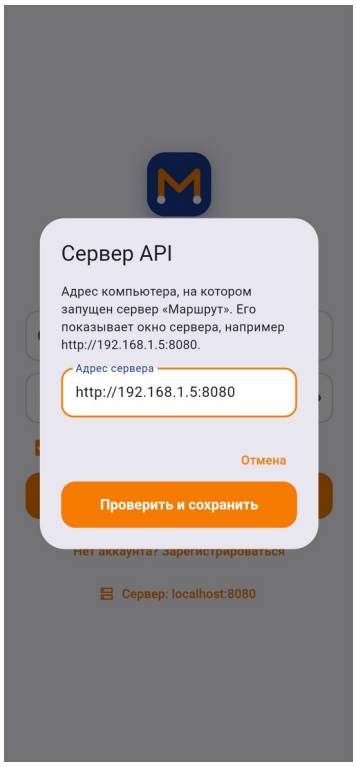


Рисунок 2.30 — Окно адреса сервера API

2.6 Реализация основных модулей

2.6.1 Регистрация, вход и хранение паролей

Учётные записи хранятся в таблице `users` на сервере. Регистрация и вход — запросы `POST /api/auth/register` и `POST /api/auth/login`; пароль проверяет и хэширует только сервер, приложение его не хранит. При регистрации имя очищается от лишних пробелов, email приводится к

нижнему регистру и проверяется на уникальность. Пароль должен быть не короче 8 символов и содержать буквы и цифры.

В версии 1 семестра пароль хранился как SHA-256 от соли и пароля. SHA-256 вычисляется очень быстро, поэтому при копировании файла базы простые пароли можно подобрать перебором. Во 2 семестре хранение переведено на PBKDF2-HMAC-SHA256 [39], одну из функций, которые для паролей рекомендует OWASP [35]:

- соль — 16 случайных байт из криптостойкого генератора `Random.secure()`;
- функция HMAC-SHA256 применяется 100 000 раз, что во столько же раз замедляет перебор;
- результат хранится в виде `pbkdf2_sha256$100000$<хэш>`: число итераций записано в самом хэше, поэтому его можно увеличить в следующей версии, и старые записи продолжат проверяться;
- сравнение хэшей выполняется за постоянное время, чтобы время проверки не подсказывало, сколько символов совпало;
- расчёт выполняется в отдельном изоляте (`Isolate.run`), поэтому сервер продолжает отвечать на другие запросы, пока считается хэш.

Пользователи, зарегистрированные в версии 1 семестра, не теряют доступ: при входе сервер определяет формат хэша, проверяет пароль старым способом и сразу сохраняет новый хэш PBKDF2 с новой солью. Пароль известен серверу только в момент входа, поэтому пересчитать хэши заранее нельзя.

Корректность реализации PBKDF2 проверена по эталонным значениям, которые независимо посчитаны функцией `hashlib.pbkdf2_hmac` языка Python, в том числе для пароля на кириллице. Полный текст класса приведён в приложении Б.

При неверном пароле и при неизвестном email сервер возвращает одно и то же сообщение «Неверный email или пароль», чтобы по ответу нельзя

было узнать, зарегистрирован ли адрес. После успешного входа сервер выдаёт токен (п. 2.3.2), и дальше приложение отправляет его вместо пароля.

2.6.2 Сессия и выход

Если при входе отмечено «Запомнить меня», идентификатор пользователя и токен сохраняются в `shared_preferences`. При следующем запуске Splash запрашивает `GET /api/me`: если сервер принял токен, открывается главная; если ответил `401` (токен устарел или аккаунт удалён), сохранённая сессия стирается и открывается экран входа. Если сервер недоступен, тоже открывается экран входа, где можно проверить адрес сервера, но сессия не стирается: после восстановления связи вход выполнится без пароля. При выходе приложение отправляет `POST /api/auth/logout`, и сервер удаляет токен; затем стираются сохранённые данные сессии, отменяется ежедневное напоминание, а роутер открывает экран входа. Выход выполняется даже без связи с сервером.

2.6.3 Удаление аккаунта

Удаление аккаунта необратимо, поэтому требует повторного ввода пароля: приложение отправляет `DELETE /api/me` с паролем в теле запроса. Сервер проверяет пароль и удаляет строку пользователя, а планы, занятия, темы, заметки, избранное, история Pomodoro и токены входа удаляются каскадно правилами внешних ключей. Затем выполняется выход.

2.6.4 Создание плана и работа с ним

Онбординг ведёт контроллер `OnboardingController`: он хранит выбранное направление, ответы теста и настройки расписания между шагами. На экране предпросмотра `PlanService.preview` рассчитывает план без сохранения. `PlanService.createPlan` рассчитывает план в приложении и отправляет его одним запросом `POST /api/plans`; сервер сохраняет темы и занятия в одной транзакции и делает новый план активным. Отметка занятия

— запрос `PUT /api/sessions/{id}/done`; после ответа сервера главная, план и статистика загружают данные заново (п. 2.3.3).

2.6.5 Напоминания и уведомления

Ежедневное напоминание реализовано пакетом flutter_local_notifications [27]. Время напоминания хранится в профиле пользователя и пересчитывается в UTC, поэтому дополнительный пакет определения часового пояса не нужен. При окончании интервала Pomodoro показывается мгновенное уведомление. В Android-проекте объявлены приёмники плагина для показа запланированных уведомлений и их восстановления после перезагрузки, разрешение `RECEIVE_BOOT_COMPLETED` и белый значок уведомлений. На Windows плагин не поддерживает ежедневное повторение без установщика MSIX, поэтому переключатель напоминания в настольной версии недоступен и подписан «Доступно в версии для Android и iOS»; мгновенные уведомления работают.

Сервис уведомлений скрыт за интерфейсом `ReminderService`. В тестах вместо него подставляется заглушка, которая записывает вызовы в журнал, поэтому тесты проверяют, что напоминание запланировано, не показывая настоящих уведомлений.

2.6.6 Обработка ошибок

Ошибки, понятные пользователю (занятый email, неверный пароль, пустое название), передаются исключением `AppException` с русским текстом и выводятся под полем формы или во всплывающем сообщении. Сервер проверяет те же правила и возвращает тот же текст в ответе `400`, а клиент API превращает его в `AppException`, поэтому для пользователя не важно, где именно найдена ошибка. Ошибки связи с сервером обрабатываются отдельно (п. 2.3.3). Повреждённый файл каталога превращается в понятное сообщение вместо падения приложения. Ошибки платформы при работе с уведомлениями не прерывают выход из аккаунта.

2.7 Тестирование

Тестирование подробно описано в отдельном разделе (`docs/coursework/testing.md`): тест-кейсы, состав тестов и журнал найденных ошибок. Итоги приведены в таблице 2.7.

Таблица 2.7 — Итоги тестирования

Вид тестирования	**Объём**	**Результат**
Модульные тесты: алгоритм (`test/domain`)	32 теста	Пройдены
Модульные тесты: валидаторы, пароли, маршрутизация, каталог, форматирование	29 тестов	Пройдены
Тесты репозиториев базы данных на SQLite в памяти	18 тестов	Пройдены
Тесты сервера API: адреса, коды ответов, токены, проверка данных, чужие записи, CRUD	23 теста	Пройдены
Тесты клиента API: ошибки сети и сервера, таймаут, 401, обновление данных	14 тестов	Пройдены
Виджет-тесты экранов и сценариев (приложение работает с сервером API в памяти)	21 тест	Пройдены
Интеграционный тест настоящего приложения (окно Linux, настоящий HTTP-сервер API)	1 сценарий	Пройден
Статический анализ `flutter analyze`	Весь проект	Замечаний нет
Ручная проверка скриншотов, настольной версии, настроек Android	25 скриншотов, 3 платформы	Найдено и исправлено 10 ошибок
Матрица снимков перед сдачей	280 снимков в 7 вариантах	Найдено и исправлено 10 ошибок
Переход на работу через API	Тесты и анализ кода	Найдено и исправлено 3 ошибки
Тест-кейсы	45 автоматизированных, 8 ручных	См. `testing.md`

Всего найдено и исправлено 27 ошибок. Шесть из них нашли автоматические тесты: ошибку освобождения контроллера в диалоге переименования плана, слишком раннее ожидание экрана входа в интеграционном тесте, нажатие по старым координатам после прокрутки в самих виджет-тестах, сообщение «Аккаунт создан», которое после входа закрывало кнопку «Начать обучение» (его поймал интеграционный тест в

GitHub Actions), повторный запрос после закрытия экрана в потоках данных API и 40-секундное ожидание на заставке при выключенном сервере.

Виджет-тесты запускают приложение вместе с настоящим сервером API, у которого база SQLite находится в оперативной памяти: HTTP-клиент передаёт запросы серверу напрямую, без сети. Так проверяется весь путь данных — от экрана через API до базы. Дата зафиксирована (понедельник 12.10.2026), поэтому результат не зависит от дня запуска и не затрагивает данные пользователя. Отдельный тест «выключает» связь с сервером посреди работы и проверяет, что пользователь видит причину ошибки, а после восстановления связи действие выполняется. Тест адаптивности открывает все экраны на трёх размерах окна; ошибка вёрстки (переполнение) автоматически роняет тест.

2.8 Сборка и развёртывание

Сборка автоматизирована в GitHub Actions [28] (файл `.github/workflows/build.yml`). При каждой отправке кода в репозиторий выполняются:

1. проверка форматирования кода, статический анализ и все модульные и виджет-тесты;
2. интеграционный тест настоящего приложения в Linux на виртуальном экране;
3. сборка релизного APK для Android (`flutter build apk --release`) [20];
4. сборка релизной версии для Windows (`flutter build windows --release`) [21];
5. сборка для iPhone на компьютере macOS без подписи (`flutter build ios --release --no-codesign`) и упаковка в файл IPA;
6. сборка программы-сервера API для Windows (`dart build cli`) вместе с каталогом тем и файлом «Запустить сервер.bat».

Готовые файлы сохраняются как артефакты сборки. Когда на странице Releases репозитория публикуется новый релиз (например, `v1.0.0`), сборка запускается для него и прикладывает APK, архив для Windows, IPA для iPhone и архив сервера к релизу, откуда их можно скачать без входа в GitHub.

Для установки на телефон нужно скачать файл `marshrut-release.apk`, открыть его и разрешить установку из этого источника. Релизный APK подписан отладочным ключом: для установки на свой телефон этого достаточно, для публикации в Google Play нужен собственный ключ подписи. Версия для Windows распространяется как архив: его нужно распаковать и запустить `marshrut.exe`. На iPhone приложение устанавливается с компьютера на Windows программой Sideloadly, которая подписывает IPA бесплатным Apple ID пользователя; такая подпись действует 7 дней, после чего установку повторяют. Публикация в App Store требует платного аккаунта разработчика Apple и в учебном проекте не использовалась.

Сервер API распространяется как архив: его нужно распаковать и запустить «Запустить сервер.bat», разрешить доступ в брандмауэре Windows и ввести на телефоне адрес, который показывает окно сервера. Пошаговые инструкции для пользователя — в файлах `docs/Сервер_API` и `docs/Установка_на_iPhone` репозитория. Для самостоятельной сборки достаточно установленного Flutter SDK: команды приведены в README.

Выводы по главе 2

1. Сформулированы функциональные и нефункциональные требования, все они реализованы в приложении.
2. Разработана клиент-серверная архитектура: приложение из трёх слоёв и сервер API. Алгоритм построения плана выделен в слой предметной области на чистом Dart и не зависит от интерфейса и способа хранения данных, поэтому переход на сервер API не потребовал его изменения.
3. Спроектирована база данных из девяти таблиц со связями и каскадным удалением. Разработан сервер REST API из 39 адресов: приложение

получает, добавляет, изменяет и удаляет данные только через API, сервер проверяет токен, данные и владельца записи и сообщает об ошибках кодами HTTP с понятным текстом, а приложение показывает их пользователю.

4. Реализован алгоритм формирования персонального плана: исключение известных тем, топологическая сортировка Кана, раскладка по дням с лимитом времени, интервальные повторения, проверка дедлайна с подбором нужного числа часов и перепланирование.
5. Реализованы экраны приложения в стиле Material 3 с фирменными цветами, светлой и тёмной темами и адаптивной вёрсткой для телефона и компьютера.
6. По итогам анализа версии 1 семестра усилена защита паролей (PBKDF2), добавлено удаление аккаунта, настроены уведомления Android, сделаны иконка и релизная сборка.
7. Приложение проверено 137 модульными и виджет-тестами, интеграционным тестом с настоящим HTTP-сервером и матрицей из 280 снимков экранов; найдено и исправлено 27 ошибок. Сборка APK, версий для Windows и iPhone и программы-сервера автоматизирована.

ЗАКЛЮЧЕНИЕ

В ходе курсового проекта разработано мобильное приложение «Маршрут», которое автоматически формирует персональный план обучения по выбранному направлению с учётом знаний пользователя, его времени и срока и перестраивает план при отставании. Цель работы достигнута, все поставленные задачи решены.

1. **Проанализирована предметная область.** Рассмотрены трудности самостоятельного обучения, методика распределённого повторения по кривой забывания Эббингауза и техника Pomodoro. Показано, что план обучения удобно описывать графом тем с зависимостями, а порядок изучения находить топологической сортировкой.
2. **Рассмотрены особенности мобильных приложений и принципы их разработки:** ограниченные ресурсы устройства, работа без сети, безопасность данных, адаптивность интерфейса, разделение кода на слои.
3. **Проведён анализ аналогов.** roadmap.sh, Duolingo, Stepik, Coursera, Notion и Todoist закрывают задачу частично; ни одно решение не объединяет граф тем, учёт знаний, расписание под время и срок пользователя и автоматическую перестройку плана.
4. **Обоснован выбор технологий:** Flutter и Dart, чистая архитектура, Riverpod, собственный сервер REST API на Dart (shelf) с базой drift (SQLite), go_router, fl_chart, flutter_local_notifications, Material 3. Приложение работает с данными только через API и не зависит от облачных сервисов.
5. **Разработан алгоритм формирования плана.** Известные по тесту темы исключаются, остальные упорядочиваются алгоритмом Кана, раскладываются по учебным дням с дневным лимитом и дополняются повторениями через 1, 3 и 7 дней. Затем проверяется дедлайн, и при необходимости алгоритм подсказывает нужное число часов в неделю.

Перепланирование сохраняет выполненное и раскладывает остаток заново. Алгоритм выделен в слой на чистом Dart и покрыт 32 модульными тестами.

6. **Спроектирован интерфейс и реализованы экраны:** заставка, регистрация, вход, главная, пять вкладок навигации, профиль, онбординг с диагностическим тестом, план, темы, статистика, заметки, Pomodoro, несколько планов. Интерфейс выполнен в стиле Material 3 в тёмно-синих и оранжевых цветах, есть тёмная тема, вёрстка адаптирована для телефона в обеих ориентациях и для окна компьютера.
7. **Приложение протестировано:** 137 модульных и виджет-тестов (в том числе тесты сервера и клиента API), интеграционный тест настоящего приложения с настоящим HTTP-сервером, 45 тест-кейсов, ручные проверки и матрица из 280 снимков экранов в светлой и тёмной теме, на разных размерах экрана и с крупным шрифтом. Найдено и исправлено 27 ошибок.
8. **Первая версия приложения проанализирована и доработана.** Хранение паролей переведено с однократного SHA-256 на PBKDF2-HMAC-SHA256 со 100 000 итераций с автоматическим пересчётом старых хэшей, добавлено удаление аккаунта, исправлена настройка уведомлений Android, сделаны иконка и название приложения. Сборка релизного APK для Android, версий для Windows и iPhone автоматизирована в GitHub Actions вместе с запуском всех тестов.
9. **Разработан сервер REST API, приложение переведено на работу с данными через API.** Сервер на Dart хранит данные в базе SQLite и предоставляет 39 адресов для получения, добавления, изменения и удаления данных. Вход выполняется по токену, сервер повторно проверяет данные и не даёт доступа к чужим записям, ошибки возвращаются кодами HTTP с понятным текстом. Приложение показывает пользователю ошибки сервера и связи и обновляет экраны

после каждого изменения. Сервер запускается на ноутбуке, телефон подключается к нему по Wi-Fi.

Практическая значимость. Приложение можно использовать для реальной подготовки: в каталоге четыре направления (Flutter, Python, английский язык, алгоритмы), 75 тем с оценкой трудоёмкости, зависимостями и ссылками на материалы, а также 32 вопроса диагностического теста. Новое направление добавляется JSON-файлом без изменения кода.

Направления развития:

- резервное копирование базы сервера и экспорт плана в файл;
- адаптация интервалов повторения к результатам пользователя, например по алгоритму SuperMemo;
- редактор зависимостей тем и импорт собственных направлений;
- публикация в Google Play с собственным ключом подписи;
- размещение сервера API в интернете с шифрованием HTTPS, чтобы приложение работало вне локальной сети;
- публикация версии для iPhone в App Store с платным аккаунтом разработчика Apple.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

Список оформлен по ГОСТ 7.1-2003 «Библиографическая запись. Библиографическое описание», электронные ресурсы описаны по ГОСТ 7.82-2001. [ПРОВЕРИТЬ] Уточнить в методичке колледжа, какой стандарт требуется. Возможно, нужен ГОСТ Р 7.0.100-2018: тогда меняется только расстановка знаков, состав сведений остаётся тем же. Дата обращения к электронным ресурсам: 11.10.2026, к источникам о сервере API (№ 17, 31, 38, 40, 41, 44, 47) — 15.10.2026. Число страниц книг сверено с библиотечными каталогами и сайтами издательств.

Книги и статьи

1. Кормен, Т. Х. Алгоритмы: построение и анализ / Т. Х. Кормен, Ч. И. Лейзерсон, Р. Л. Ривест, К. Штайн ; пер. с англ. – 3-е изд. – М. : Вильямс, 2013. – 1328 с.
2. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин ; пер. с англ. – СПб. : Питер, 2018. – 352 с.
3. Cepeda, N. J. Distributed practice in verbal recall tasks: A review and quantitative synthesis / N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, D. Rohrer // Psychological Bulletin. – 2006. – Vol. 132, № 3. – P. 354–380.
4. Cirillo, F. The Pomodoro Technique: The Acclaimed Time-Management System That Has Transformed How We Work / F. Cirillo. – New York : Currency, 2018. – 152 p.
5. Ebbinghaus, H. Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie / H. Ebbinghaus. – Leipzig : Duncker & Humblot, 1885. – IX, 169 S.
6. Kahn, A. B. Topological sorting of large networks / A. B. Kahn // Communications of the ACM. – 1962. – Vol. 5, № 11. – P. 558–562.
7. Napoli, M. L. Beginning Flutter: A Hands On Guide to App Development / M. L. Napoli. – Indianapolis : Wiley, 2019. – XXVIII, 500 p.

8. Secure Hash Standard (SHS) : Federal Information Processing Standards Publication 180-4. – Gaithersburg : National Institute of Standards and Technology, 2015. – 36 p. – DOI 10.6028/NIST.FIPS.180-4.
9. Windmill, E. Flutter in Action / E. Windmill. – Shelter Island : Manning, 2020. – 368 p.

Электронные ресурсы

10. About SQLite [Электронный ресурс]. – Режим доступа: https://www.sqlite.org/about.html (дата обращения: 11.10.2026).
11. Adaptive icons [Электронный ресурс] // Android Developers. – Режим доступа: https://developer.android.com/develop/ui/views/launch/icon_design_adaptive (дата обращения: 11.10.2026).
12. Bloc library: documentation [Электронный ресурс]. – Режим доступа: https://bloclibrary.dev (дата обращения: 11.10.2026).
13. Coursera [Электронный ресурс] : онлайн-платформа курсов. – Режим доступа: https://www.coursera.org (дата обращения: 11.10.2026).
14. Dart documentation [Электронный ресурс] / Google. – Режим доступа: https://dart.dev/guides (дата обращения: 11.10.2026).
15. Drift: documentation [Электронный ресурс]. – Режим доступа: https://drift.simonbinder.eu (дата обращения: 11.10.2026).
16. Duolingo [Электронный ресурс] : сервис для изучения языков. – Режим доступа: https://www.duolingo.com (дата обращения: 11.10.2026).
17. Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures [Электронный ресурс] : dissertation for the degree of Doctor of Philosophy / R. T. Fielding. – Irvine : University of California, 2000. – Режим доступа: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (дата обращения: 15.10.2026).

18. Firebase documentation [Электронный ресурс] / Google. – Режим доступа: https://firebase.google.com/docs (дата обращения: 11.10.2026).
19. fl_chart [Электронный ресурс] : пакет для Flutter // pub.dev. – Режим доступа: https://pub.dev/packages/fl_chart (дата обращения: 11.10.2026).
20. Build and release an Android app [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/deployment/android (дата обращения: 11.10.2026).
21. Build and release a Windows desktop app [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/deployment/windows (дата обращения: 11.10.2026).
22. Guide to app architecture [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/app-architecture/guide (дата обращения: 11.10.2026).
23. Integration testing [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/testing/integration-tests (дата обращения: 11.10.2026).
24. List of state management approaches [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/data-and-backend/state-mgmt/options (дата обращения: 11.10.2026).
25. Supported deployment platforms [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/reference/supported-platforms (дата обращения: 11.10.2026).
26. Testing Flutter apps [Электронный ресурс] // Flutter documentation. – Режим доступа: https://docs.flutter.dev/testing/overview (дата обращения: 11.10.2026).
27. flutter_local_notifications [Электронный ресурс] : пакет для Flutter // pub.dev. – Режим доступа:

https://pub.dev/packages/flutter_local_notifications (дата обращения: 11.10.2026).
28. GitHub Actions documentation [Электронный ресурс] / GitHub. – Режим доступа: https://docs.github.com/actions (дата обращения: 11.10.2026).
29. go_router [Электронный ресурс] : пакет для Flutter // pub.dev. – Режим доступа: https://pub.dev/packages/go_router (дата обращения: 11.10.2026).
30. Hive [Электронный ресурс] : пакет для Flutter // pub.dev. – Режим доступа: https://pub.dev/packages/hive (дата обращения: 11.10.2026).
31. http [Электронный ресурс] : пакет для Dart // pub.dev. – Режим доступа: https://pub.dev/packages/http (дата обращения: 15.10.2026).
32. Kotlin Multiplatform [Электронный ресурс] / JetBrains. – Режим доступа: https://www.jetbrains.com/kotlin-multiplatform/ (дата обращения: 11.10.2026).
33. Material Design 3 [Электронный ресурс] / Google. – Режим доступа: https://m3.material.io (дата обращения: 11.10.2026).
34. Notion [Электронный ресурс] : сервис заметок и задач. – Режим доступа: https://www.notion.so (дата обращения: 11.10.2026).
35. Password Storage Cheat Sheet [Электронный ресурс] // OWASP Cheat Sheet Series. – Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата обращения: 11.10.2026).
36. Processes and app lifecycle [Электронный ресурс] // Android Developers. – Режим доступа: https://developer.android.com/guide/components/activities/process-lifecycle (дата обращения: 11.10.2026).
37. React Native [Электронный ресурс] / Meta. – Режим доступа: https://reactnative.dev (дата обращения: 11.10.2026).
38. RFC 6750. The OAuth 2.0 Authorization Framework: Bearer Token Usage [Электронный ресурс] / M. Jones, D. Hardt. – IETF, 2012. – Режим

доступа: https://www.rfc-editor.org/rfc/rfc6750 (дата обращения: 15.10.2026).
39. RFC 8018. PKCS #5: Password-Based Cryptography Specification. Version 2.1 [Электронный ресурс] / K. Moriarty, B. Kaliski, A. Rusch. – IETF, 2017. – Режим доступа: https://www.rfc-editor.org/rfc/rfc8018 (дата обращения: 11.10.2026).
40. RFC 8259. The JavaScript Object Notation (JSON) Data Interchange Format [Электронный ресурс] / T. Bray. – IETF, 2017. – Режим доступа: https://www.rfc-editor.org/rfc/rfc8259 (дата обращения: 15.10.2026).
41. RFC 9110. HTTP Semantics [Электронный ресурс] / R. Fielding, M. Nottingham, J. Reschke. – IETF, 2022. – Режим доступа: https://www.rfc-editor.org/rfc/rfc9110 (дата обращения: 15.10.2026).
42. Riverpod: documentation [Электронный ресурс]. – Режим доступа: https://riverpod.dev (дата обращения: 11.10.2026).
43. roadmap.sh [Электронный ресурс] : карты изучения технологий. – Режим доступа: https://roadmap.sh (дата обращения: 11.10.2026).
44. shelf [Электронный ресурс] : пакет для Dart // pub.dev. – Режим доступа: https://pub.dev/packages/shelf (дата обращения: 15.10.2026).
45. Sound null safety [Электронный ресурс] // Dart documentation. – Режим доступа: https://dart.dev/null-safety (дата обращения: 11.10.2026).
46. Stepik [Электронный ресурс] : образовательная платформа. – Режим доступа: https://stepik.org (дата обращения: 11.10.2026).
47. Supabase documentation [Электронный ресурс] / Supabase. – Режим доступа: https://supabase.com/docs (дата обращения: 15.10.2026).
48. Todoist [Электронный ресурс] : менеджер задач. – Режим доступа: https://todoist.com (дата обращения: 11.10.2026).
49. Understanding Google Play’s app account deletion requirements [Электронный ресурс] // Play Console Help. – Режим доступа: https://support.google.com/googleplay/android-developer/answer/13327111 (дата обращения: 11.10.2026).

50. Use window size classes [Электронный ресурс] // Android Developers. – Режим доступа: https://developer.android.com/develop/ui/compose/layouts/adaptive/window-size-classes (дата обращения: 11.10.2026).

ПРИЛОЖЕНИЕ А. ТОПОЛОГИЧЕСКАЯ СОРТИРОВКА ТЕМ (АЛГОРИТМ КАНА)

Файл `lib/domain/planner/topological_sort.dart`.

```
// Топологическая сортировка тем алгоритмом Кана (слой domain, planner).
// Топологическая сортировка — такой порядок вершин графа, при котором каждая
// тема стоит после всех тем, от которых она зависит. Используется в
// PlanGenerator (шаг 3 построения плана) и в PlanService.addTopic().

import 'dart:collection';

import '../entities/topic.dart';

/// В графе тем обнаружен цикл, поэтому порядок изучения построить нельзя.
///
/// Цикл — это ситуация, когда темы зависят друг от друга по кругу
/// (A требует B, а B требует A), и ни одну из них нельзя начать первой.
class CycleDetectedException implements Exception {
  /// Создаёт исключение со списком тем [topicIds], которые не удалось
  /// упорядочить.
  CycleDetectedException(this.topicIds);

  /// Темы, которые остались неупорядоченными (входят в цикл или зависят
  /// от него).
  final List<String> topicIds;

  /// Текст ошибки: перечисляет проблемные темы через запятую.
  @override
  String toString() => 'В зависимостях тем есть цикл: ${topicIds.join(', ')}';
}

/// Топологическая сортировка тем алгоритмом Кана.
///
/// 1. Для каждой темы считаем входящую степень — число её зависимостей
///    внутри переданного набора. Зависимости, которых нет в наборе
///    (например, уже изученные темы), считаются выполненными.
/// 2. Темы с нулевой степенью помещаем в очередь с приоритетом.
/// 3. Извлекаем тему, добавляем в результат и уменьшаем степень у всех тем,
///    которые от неё зависят. Если степень стала нулевой — тема в очередь.
/// 4. Если в результат попали не все темы, в графе есть цикл.
///
/// Очередь упорядочена по уровню сложности, затем по исходной позиции темы,
/// поэтому при одинаковых входных данных результат всегда одинаковый.
/// Сложность — O((V + E) · log V) из-за очереди с приоритетом.
/// (V — число тем, E — число связей «тема → зависимость».)
List<Topic> topologicalSort(List<Topic> topics) {
  // Словарь «id темы → её позиция в списке». Дальше алгоритм работает
  // с номерами тем (индексами), а не с самими объектами — так проще.
  final index = <String, int>{
    for (var i = 0; i < topics.length; i++) topics[i].id: i,
  };
  // inDegree[i] — входящая степень темы i: сколько её зависимостей ещё
  // не поставлено в результат. Сначала у всех тем 0.
  final inDegree = List<int>.filled(topics.length, 0);
  // dependents[i] — номера тем, которые зависят от темы i
  // (рёбра графа в направлении «предшественник → зависимая тема»).
  final dependents = List.generate(topics.length, (_) => <int>[]);

  // Шаг 1 алгоритма: строим рёбра графа и считаем входящие степени.
  for (var i = 0; i < topics.length; i++) {
    // toSet() убирает повторы, если одна зависимость указана дважды.
    for (final prerequisite in topics[i].prerequisites.toSet()) {
      // Ищем номер темы-предшественника в переданном наборе.
      final from = index[prerequisite];
      if (from == null) continue; // зависимость уже изучена или вне плана
      // Запоминаем ребро «предшественник → тема i».
```

```
      dependents[from].add(i);
      // У темы i стало на одну невыполненную зависимость больше.
      inDegree[i]++;
    }
  }

  // Правило сравнения для очереди: раньше идёт тема с меньшим уровнем
  // сложности, а при равном уровне — та, что раньше стояла в списке.
  int compare(int a, int b) {
    // Сравниваем уровни сложности двух тем.
    final byLevel = topics[a].level.compareTo(topics[b].level);
    // Уровни разные — решает уровень, одинаковые — исходная позиция.
    return byLevel != 0 ? byLevel : a.compareTo(b);
  }

  // Шаг 2: очередь с приоритетом. SplayTreeSet — множество, которое само
  // держит элементы отсортированными по compare, поэтому первый элемент
  // всегда самый приоритетный.
  final queue = SplayTreeSet<int>(compare);
  // Кладём в очередь все темы без зависимостей — их можно учить сразу.
  for (var i = 0; i < topics.length; i++) {
    if (inDegree[i] == 0) queue.add(i);
  }

  // Шаг 3: сюда складываем темы в итоговом порядке изучения.
  final result = <Topic>[];
  // Повторяем, пока есть темы, готовые к изучению.
  while (queue.isNotEmpty) {
    // Берём самую приоритетную тему и убираем её из очереди.
    final current = queue.first;
    queue.remove(current);
    // Ставим её в результат: все её зависимости уже стоят раньше.
    result.add(topics[current]);
    // «Удаляем» исходящие рёбра: каждая зависимая тема теперь ждёт
    // на одну тему меньше.
    for (final next in dependents[current]) {
      inDegree[next]--;
      // Все зависимости темы выполнены — она готова, кладём в очередь.
      if (inDegree[next] == 0) queue.add(next);
    }
  }

  // Шаг 4: если в результат попали не все темы, какие-то темы ждут друг
  // друга по кругу — значит, в графе есть цикл.
  if (result.length != topics.length) {
    // Множество id тем, которые удалось упорядочить.
    final sorted = result.map((t) => t.id).toSet();
    // Сообщаем об ошибке и перечисляем темы, оставшиеся без места.
    throw CycleDetectedException(
      topics.where((t) => !sorted.contains(t.id)).map((t) => t.id).toList(),
    );
  }
  // Цикла нет — возвращаем темы в порядке изучения.
  return result;
}
```

ПРИЛОЖЕНИЕ Б. ХЭШИРОВАНИЕ ПАРОЛЕЙ PBKDF2-HMAC-SHA256

Файл lib/core/utils/password_hasher.dart.

```
// Файл password_hasher.dart — безопасное хранение паролей (хэширование).
// Слой: core (утилиты). Пароль никогда не хранится в открытом виде: в базу
// попадают только соль и хэш. Класс использует AuthRepositoryImpl
// (data/repositories) при регистрации, входе и смене пароля; работает он
// на сервере API (lib/server/api_server.dart), а не в приложении.
import 'dart:convert';
import 'dart:isolate';
import 'dart:math';

import 'package:crypto/crypto.dart';

/// Хэширование паролей алгоритмом PBKDF2-HMAC-SHA256 (RFC 8018).
///
/// Хэш — это «отпечаток» строки фиксированной длины, по которому нельзя
/// восстановить сам пароль, но можно проверить, совпадает ли введённый.
///
/// Соль — 16 случайных байт из криптостойкого генератора [Random.secure].
/// Благодаря соли одинаковые пароли разных пользователей дают разные хэши,
/// а готовые таблицы хэшей («радужные таблицы») бесполезны. Многократное
/// повторение HMAC ([iterations] раз) замедляет подбор пароля перебором:
/// так рекомендует OWASP вместо однократного SHA-256.
///
/// Хэш хранится в виде `pbkdf2_sha256$<итерации>$<hex>`, поэтому число
/// итераций можно увеличить позже, не ломая старые записи. Хэши версии
/// 1 семестра (однократный SHA-256 от «соль + пароль») по-прежнему
/// проверяются и заменяются новыми при входе (см. [needsRehash]).
class PasswordHasher {
  /// Создаёт хэшер. [random] можно подменить в тестах; по умолчанию —
  /// криптостойкий [Random.secure]. [iterations] должно быть больше нуля.
  PasswordHasher({Random? random, this.iterations = defaultIterations})
    : assert(iterations > 0),
      _random = random ?? Random.secure();

  /// Число итераций по умолчанию: заметно замедляет перебор и при этом
  /// не задерживает вход на телефоне дольше долей секунды.
  static const defaultIterations = 100000;

  /// Название схемы — первая часть сохранённой строки хэша.
  static const _scheme = 'pbkdf2_sha256';

  /// Сколько раз повторяется HMAC при расчёте нового хэша.
  final int iterations;

  /// Генератор случайных чисел для соли.
  final Random _random;

  /// Создаёт новую случайную соль: [length] случайных байт, записанных
  /// текстом в кодировке base64url (только безопасные для URL символы).
  String generateSalt([int length = 16]) {
    // Список из length случайных чисел от 0 до 255 (то есть байтов).
    final bytes = List<int>.generate(length, (_) => _random.nextInt(256));
    // Переводим байты в строку, чтобы хранить соль в базе как текст.
    return base64Url.encode(bytes);
  }

  /// Хэш пароля в текущем формате.
  ///
  /// Пароль и соль переводятся в байты UTF-8, прогоняются через [pbkdf2]
  /// и записываются в виде `pbkdf2_sha256$<итерации>$<hex>`.
  String hash(String password, String salt) => _format(
    iterations,
    pbkdf2(utf8.encode(password), utf8.encode(salt), iterations),
```

```
);

/// [hash] в фоновом изоляте, чтобы интерфейс не замирал на время расчёта.
///
/// Изолят — отдельный поток выполнения Dart со своей памятью; функция
/// [Isolate.run] запускает в нём вычисление и возвращает результат.
Future<String> hashAsync(String password, String salt) {
  // При малом числе итераций (например, в тестах) считаем сразу.
  if (iterations < _isolateThreshold) {
    return Future.value(hash(password, salt));
  }
  // Копируем число итераций в локальную переменную: в изолят уходит
  // только это число, а не весь текущий объект.
  final rounds = iterations;
  // В изоляте создаём новый хэшер с тем же числом итераций и считаем хэш.
  return Isolate.run(
    () => PasswordHasher(iterations: rounds).hash(password, salt),
  );
}

/// [verify] в фоновом изоляте.
Future<bool> verifyAsync(String password, String salt, String stored) {
  // Узнаём число итераций из сохранённого хэша; у старого формата
  // его нет — считаем, что итерация одна.
  final rounds = _iterationsOf(stored) ?? 1;
  // Быстрый расчёт выполняем сразу, без изолята.
  if (rounds < _isolateThreshold) {
    return Future.value(verify(password, salt, stored));
  }
  // Долгую проверку уносим в изолят. Число итераций в конструкторе здесь
  // не важно: verify берёт его из сохранённой строки хэша.
  return Isolate.run(
    () => PasswordHasher(iterations: 1).verify(password, salt, stored),
  );
}

/// При малом числе итераций запуск изолята дороже самого вычисления.
static const _isolateThreshold = 10000;

/// Хэш старого формата (версия 1 семестра): SHA-256 от «соль + пароль».
static String legacyHash(String password, String salt) =>
    sha256.convert(utf8.encode('$salt$password')).toString();

/// Проверяет, подходит ли [password] к сохранённому хэшу [stored].
///
/// Понимает оба формата: новый PBKDF2 и старый [legacyHash].
bool verify(String password, String salt, String stored) {
  // Здесь будет хэш введённого пароля, посчитанный тем же способом.
  final String actual;
  // Число итераций из сохранённой строки (null — старый формат).
  final rounds = _iterationsOf(stored);
  if (rounds != null) {
    // Новый формат: считаем PBKDF2 с тем же числом итераций.
    actual = _format(
      rounds,
      pbkdf2(utf8.encode(password), utf8.encode(salt), rounds),
    );
  } else {
    // Старый формат: однократный SHA-256.
    actual = legacyHash(password, salt);
  }
  // Сравниваем хэши за постоянное время (см. _constantTimeEquals).
  return _constantTimeEquals(actual, stored);
}

/// `true`, если хэш старого формата или посчитан с меньшим числом итераций.
bool needsRehash(String stored) {
  // Старый формат даёт null, новый — число итераций.
  final rounds = _iterationsOf(stored);
  // Пересчитать нужно, если формат старый или итераций меньше текущих.
  return rounds == null || rounds < iterations;
```

```
}

/// PBKDF2 с HMAC-SHA256. Для ключа длиной до 32 байт нужен один блок:
/// T = U1 ⊕ U2 ⊕ … ⊕ Uc, где U1 = HMAC(P, S ‖ 00000001), Uj = HMAC(P, Uj-1).
///
/// [password] и [salt] — байты, [iterations] — число повторов,
/// [length] — длина результата в байтах (по умолчанию 32).
static List<int> pbkdf2(
  List<int> password,
  List<int> salt,
  int iterations, [
  int length = 32,
]) {
  // HMAC-SHA256 с паролем в роли ключа — основная функция алгоритма.
  final prf = Hmac(sha256, password);
  // Один блок SHA-256 даёт 32 байта; считаем, сколько блоков нужно
  // (деление с округлением вверх).
  final blocks = (length + 31) ~/ 32;
  // Сюда складываем байты всех блоков подряд.
  final result = <int>[];
  // Блоки нумеруются с единицы.
  for (var block = 1; block <= blocks; block++) {
    // U1 = HMAC(соль + номер блока в виде 4 байт, старший байт первым).
    var u = prf.convert([
      ...salt,
      block >> 24 & 0xff,
      block >> 16 & 0xff,
      block >> 8 & 0xff,
      block & 0xff,
    ]).bytes;
    // T начинается с копии U1.
    final t = List<int>.of(u);
    // Остальные итерации: каждый следующий U — HMAC от предыдущего.
    for (var i = 1; i < iterations; i++) {
      u = prf.convert(u).bytes;
      // Побайтово накапливаем T через XOR (^ — «исключающее или»).
      for (var j = 0; j < t.length; j++) {
        t[j] ^= u[j];
      }
    }
    // Готовый блок добавляем к результату.
    result.addAll(t);
  }
  // Обрезаем результат до нужной длины.
  return result.sublist(0, length);
}

/// Собирает строку хэша для хранения: `pbkdf2_sha256$<итерации>$<hex>`.
static String _format(int rounds, List<int> key) =>
    '$_scheme\$$rounds\$${_hex(key)}';

/// Достаёт число итераций из сохранённой строки хэша.
///
/// Возвращает `null`, если строка не в формате PBKDF2 (старый хэш)
/// или число итераций некорректно.
static int? _iterationsOf(String stored) {
  // Делим строку по символу «$» на схему, итерации и сам хэш.
  final parts = stored.split(r'$');
  // Частей не три или схема чужая — значит, это не наш новый формат.
  if (parts.length != 3 || parts[0] != _scheme) return null;
  // Пробуем прочитать число итераций.
  final rounds = int.tryParse(parts[1]);
  // Принимаем только положительное число.
  return rounds != null && rounds > 0 ? rounds : null;
}

/// Переводит байты в шестнадцатеричную строку (по 2 символа на байт).
static String _hex(List<int> bytes) =>
    bytes.map((b) => b.toRadixString(16).padLeft(2, '0')).join();

/// Сравнение без раннего выхода, чтобы время не зависело от совпадения.
```

```
  ///
  /// Обычное сравнение строк останавливается на первом отличии, и по
  /// времени ответа можно угадывать хэш; здесь всегда проверяются все
  /// символы.
  static bool _constantTimeEquals(String a, String b) {
    // Разная длина — точно не совпадают.
    if (a.length != b.length) return false;
    // Накапливаем отличия всех символов через XOR и OR.
    var diff = 0;
    for (var i = 0; i < a.length; i++) {
      diff |= a.codeUnitAt(i) ^ b.codeUnitAt(i);
    }
    // Ноль — значит, ни один символ не отличался.
    return diff == 0;
  }
}
```

ПРИЛОЖЕНИЕ В. РАСКЛАДКА ТЕМ ПО ДНЯМ И ИНТЕРВАЛЬНЫЕ ПОВТОРЕНИЯ

Файл `lib/domain/planner/scheduler.dart`.

```
// Планировщик — раскладка тем по календарю (слой domain, папка planner).
// Берёт уже упорядоченные темы и расставляет занятия по учебным дням с
// дневным лимитом минут, добавляя интервальные повторения через 1, 3 и 7
// дней после изучения темы. Используется в PlanGenerator и Replanner.

import 'dart:collection';

import '../entities/schedule_settings.dart';
import '../entities/session.dart';
import '../entities/topic.dart';
import 'dates.dart';

/// Повторение, которое нужно провести не раньше [dueDate].
///
/// Это «заявка» на повторение: в расписание она превращается в занятие
/// типа «повторение», когда планировщик дойдёт до подходящего дня.
class PendingReview {
  /// Создаёт заявку на повторение темы; все поля обязательны.
  const PendingReview({
    required this.topicId,
    required this.topicTitle,
    required this.category,
    required this.dueDate,
  });

  /// id темы, которую нужно повторить.
  final String topicId;

  /// Название темы (копируется в занятие-повторение).
  final String topicTitle;

  /// Категория темы (тоже копируется в занятие).
  final String category;

  /// Самая ранняя дата, на которую можно поставить повторение.
  final DateTime dueDate;
}

/// Раскладывает упорядоченные темы по учебным дням.
///
/// Алгоритм моделирует календарь день за днём, начиная с даты старта:
/// 1. Невыбранные дни недели пропускаются.
/// 2. В учебный день доступно [ScheduleSettings.dailyLimitMinutes] минут.
/// 3. Сначала в день ставятся повторения, срок которых наступил
///    (по [reviewMinutes] минут каждое).
/// 4. Оставшееся время заполняется текущей темой. Тема дробится на части,
///    но не мельче [minChunkMinutes], если остаток темы больше этого порога.
/// 5. Когда тема закончена, планируются повторения через [reviewIntervals]
///    календарных дней (по кривой забывания Эббингауза: 1, 3, 7 дней).
///    Повторение, выпавшее на неучебный день, переносится на ближайший учебный.
class Scheduler {
  /// Создаёт планировщик. По умолчанию: повторения через 1, 3 и 7 дней
  /// по 15 минут, части темы не короче 30 минут и не больше 3650 шагов
  /// по календарю (около 10 лет).
  const Scheduler({
    this.reviewIntervals = const [1, 3, 7],
    this.reviewMinutes = 15,
    this.minChunkMinutes = 30,
    this.maxDays = 3650,
  });

  /// Через сколько календарных дней после завершения темы ставить
```

```
/// повторения (по умолчанию 1, 3 и 7 дней).
final List<int> reviewIntervals;

/// Длительность одного повторения в минутах.
final int reviewMinutes;

/// Минимальная длина части темы в минутах, чтобы тема не дробилась
/// на совсем короткие кусочки.
final int minChunkMinutes;

/// Защита от бесконечного цикла при некорректных настройках.
final int maxDays;

/// [remainingMinutes] позволяет передать остаток по теме, если часть уже
/// изучена (используется при перепланировании). По умолчанию —
/// полная длительность темы.
/// [initialReviews] — повторения, перенесённые из прошлого расписания.
///
/// Возвращает список занятий [PlannedSession] в порядке дат. Бросает
/// [ArgumentError], если не выбран ни один учебный день.
List<PlannedSession> schedule(
  List<Topic> orderedTopics,
  ScheduleSettings settings, {
  Map<String, int> remainingMinutes = const {},
  List<PendingReview> initialReviews = const [],
}) {
  // Сколько минут можно заниматься в один учебный день.
  final dailyLimit = settings.dailyLimitMinutes;
  // Без учебных дней (или с нулевым лимитом) план построить нельзя.
  if (settings.studyDays.isEmpty || dailyLimit <= 0) {
    throw ArgumentError('Нужно выбрать хотя бы один учебный день');
  }

  // Очередь (Queue — список «первым пришёл — первым ушёл») тем, которые
  // ещё нужно изучить, в порядке сортировки. Для каждой темы храним
  // остаток минут: полный или из remainingMinutes. Темы с нулевым
  // остатком сразу отбрасываем — они уже изучены.
  final work = Queue<_TopicWork>()
    ..addAll(
      orderedTopics
          .map((t) => _TopicWork(t, remainingMinutes[t.id] ?? t.minutes))
          .where((w) => w.remaining > 0),
    );
  // Ожидающие повторения, отсортированные по сроку (ближайшие первыми).
  final reviews = [...initialReviews]
    ..sort((a, b) => a.dueDate.compareTo(b.dueDate));
  // Сюда складываем готовые занятия.
  final sessions = <PlannedSession>[];

  // Идём по календарю день за днём, начиная с даты старта. maxDays
  // ограничивает число шагов, чтобы цикл точно закончился.
  var day = dateOnly(settings.startDate);
  for (var i = 0; i < maxDays; i++, day = addDays(day, 1)) {
    // Темы и повторения закончились — расписание готово.
    if (work.isEmpty && reviews.isEmpty) break;
    // Шаг 1: день недели не выбран как учебный — пропускаем его.
    if (!settings.studyDays.contains(day.weekday)) continue;

    // Шаг 2: свободное время на сегодня равно дневному лимиту.
    var capacity = dailyLimit;

    // Шаг 3: повторения, срок которых наступил.
    // Берём повторения со сроком сегодня или раньше. Сюда же попадают
    // повторения, выпавшие на неучебный день, — так они переносятся
    // на ближайший учебный день.
    final due = reviews.where((r) => !r.dueDate.isAfter(day)).toList();
    for (final review in due) {
      // Время на ещё одно повторение закончилось — остальные
      // повторения перейдут на следующий учебный день.
      if (capacity < reviewMinutes) break;
      // Ставим занятие-повторение на сегодня.
```

```
      sessions.add(
        PlannedSession(
          topicId: review.topicId,
          topicTitle: review.topicTitle,
          category: review.category,
          date: day,
          minutes: reviewMinutes,
          kind: SessionKind.review,
        ),
      );
      // Уменьшаем свободное время и убираем повторение из ожидающих.
      capacity -= reviewMinutes;
      reviews.remove(review);
    }

    // Шаг 4: новые темы.
    // Пока есть темы и свободное время, заполняем день текущей темой.
    while (work.isNotEmpty && capacity > 0) {
      // Текущая тема — первая в очереди.
      final current = work.first;
      // Размер части: сколько поместится сегодня, но не больше, чем
      // осталось изучить по теме.
      final chunk = capacity < current.remaining
          ? capacity
          : current.remaining;
      // Кусок короче минимума и это не конец темы — не дробим тему
      // мелко, а переносим её продолжение на следующий учебный день.
      if (chunk < minChunkMinutes && chunk < current.remaining) break;

      // Ставим занятие «изучение» на сегодня.
      sessions.add(
        PlannedSession(
          topicId: current.topic.id,
          topicTitle: current.topic.title,
          category: current.topic.category,
          date: day,
          minutes: chunk,
          kind: SessionKind.study,
        ),
      );
      // Уменьшаем свободное время дня и остаток темы.
      capacity -= chunk;
      current.remaining -= chunk;

      // Тема изучена полностью.
      if (current.remaining == 0) {
        // Убираем её из очереди — дальше пойдёт следующая тема.
        work.removeFirst();
        // Шаг 5: повторения после завершения темы.
        // Для каждого интервала (1, 3, 7 дней) создаём заявку на
        // повторение со сроком «день завершения + интервал».
        for (final interval in reviewIntervals) {
          reviews.add(
            PendingReview(
              topicId: current.topic.id,
              topicTitle: current.topic.title,
              category: current.topic.category,
              dueDate: addDays(day, interval),
            ),
          );
        }
        // Снова сортируем повторения по сроку, чтобы ближайшие
        // оставались в начале списка.
        reviews.sort((a, b) => a.dueDate.compareTo(b.dueDate));
      }
    }
  }

  // Проставляем номера частей («часть 2 из 3») и отдаём результат.
  return _numberParts(sessions);
}
```

```
  /// Проставляет номер части для тем, разбитых на несколько занятий.
  List<PlannedSession> _numberParts(List<PlannedSession> sessions) {
    // Сначала считаем, на сколько занятий разбита каждая тема.
    final totals = <String, int>{};
    for (final s in sessions) {
      if (s.kind == SessionKind.study) {
        totals[s.topicId] = (totals[s.topicId] ?? 0) + 1;
      }
    }
    // Счётчики уже пронумерованных частей по каждой теме.
    final counters = <String, int>{};
    // Проходим занятия по порядку: у занятия «изучение» увеличиваем
    // счётчик его темы и записываем «номер части / всего частей»;
    // повторения оставляем без изменений.
    return [
      for (final s in sessions)
        if (s.kind == SessionKind.study)
          s.copyWith(
            part: counters[s.topicId] = (counters[s.topicId] ?? 0) + 1,
            totalParts: totals[s.topicId],
          )
        else
          s,
    ];
  }
}

/// Служебная запись планировщика: тема и сколько минут по ней ещё осталось
/// распланировать. Поле [remaining] уменьшается по ходу раскладки.
/// Класс приватный (имя начинается с «_»), виден только в этом файле.
class _TopicWork {
  /// Создаёт запись для темы [topic] с остатком [remaining] минут.
  _TopicWork(this.topic, this.remaining);

  /// Тема, которую нужно разложить по дням.
  final Topic topic;

  /// Сколько минут темы ещё не поставлено в расписание.
  int remaining;
}
```

ПРИЛОЖЕНИЕ Г. ГЕНЕРАТОР ПЛАНА И ПРОВЕРКА ДЕДЛАЙНА

Файл `lib/domain/planner/plan_generator.dart`.

```
// Генератор персонального плана обучения (слой domain, папка planner).
// Собирает вместе все шаги построения плана: исключение известных тем,
// сортировку Кана, раскладку по дням (Scheduler) и проверку дедлайна.
// Используется в PlanService (предпросмотр и создание плана) и в Replanner.

import '../entities/plan.dart';
import '../entities/schedule_settings.dart';
import '../entities/session.dart';
import '../entities/topic.dart';
import 'scheduler.dart';
import 'topological_sort.dart';

/// Генератор персонального плана обучения.
///
/// Конвейер из пяти шагов:
/// 1. Направление — граф тем с зависимостями (вход [topics]).
/// 2. Темы, известные по результатам теста ([knownTopicIds]), исключаются.
/// 3. Оставшиеся темы упорядочиваются топологической сортировкой (Кан).
/// 4. Темы раскладываются по учебным дням с лимитом часов ([Scheduler]).
/// 5. Проверяется дедлайн; если он не выполняется, подбирается минимальное
///    число часов в неделю, при котором план в него укладывается.
///    Интервальные повторения добавляет [Scheduler] на шаге 4.
class PlanGenerator {
  /// Создаёт генератор. [scheduler] можно передать свой (например, в
  /// тестах); по умолчанию — стандартный планировщик с повторениями
  /// через 1, 3 и 7 дней.
  const PlanGenerator({this.scheduler = const Scheduler()});

  /// Планировщик, который раскладывает темы по дням (шаг 4).
  final Scheduler scheduler;

  /// Максимум часов в неделю, который имеет смысл предлагать (12 ч × 7 дней).
  static const double maxHoursPerWeek = 84;

  /// Строит план по темам направления [topics] и расписанию [settings].
  ///
  /// [knownTopicIds] — id тем, которые пользователь уже знает по тесту; они
  /// не попадают в план. Возвращает [PlanResult]: порядок тем, занятия по
  /// датам, дату окончания и признак того, успевает ли план к дедлайну.
  PlanResult generate({
    required List<Topic> topics,
    required ScheduleSettings settings,
    Set<String> knownTopicIds = const {},
  }) {
    // Шаг 2.
    // Темы, которые пользователь уже знает, откладываем отдельно, чтобы
    // показать их на экране как исключённые.
    final excluded = topics.where((t) => knownTopicIds.contains(t.id)).toList();
    // Все остальные темы — их и нужно изучать.
    final remaining = topics
        .where((t) => !knownTopicIds.contains(t.id))
        .toList();

    // Шаг 3.
    // Упорядочиваем темы алгоритмом Кана: каждая тема встаёт после всех
    // тем, от которых она зависит.
    final ordered = topologicalSort(remaining);

    // Шаг 4.
    // Раскладываем темы по учебным дням (повторения добавятся там же).
    final sessions = scheduler.schedule(ordered, settings);
    // День последнего занятия по новой теме — дата окончания плана.
    final finish = lastStudyDate(sessions);

    // Шаг 5.
```

```
  // Дедлайн выполнен, если его нет, если занятий нет вообще или если
  // план заканчивается не позже дедлайна.
  final deadline = settings.deadline;
  final met = deadline == null || finish == null || !finish.isAfter(deadline);

  // Собираем результат. Если дедлайн не выполнен, дополнительно считаем,
  // сколько часов в неделю нужно, чтобы успеть.
  return PlanResult(
    orderedTopics: ordered,
    excludedTopics: excluded,
    sessions: sessions,
    finishDate: finish,
    deadlineMet: met,
    requiredHoursPerWeek: met
        ? null
        : requiredHoursPerWeek(ordered, settings),
  );
}

/// Подбирает минимальное число часов в неделю (с шагом 0,5 ч), при котором
/// все темы будут изучены не позже дедлайна. Возвращает `null`, если даже
/// [maxHoursPerWeek] не хватает.
///
/// [remainingMinutes] — остаток минут по темам; его передаёт Replanner,
/// когда часть тем уже изучена.
double? requiredHoursPerWeek(
  List<Topic> orderedTopics,
  ScheduleSettings settings, {
  Map<String, int> remainingMinutes = const {},
}) {
  // Без дедлайна подбирать нечего.
  final deadline = settings.deadline;
  if (deadline == null) return null;

  // Простой перебор: начинаем с текущих часов + 0,5 и прибавляем
  // по 0,5 ч, пока не дойдём до максимума.
  for (
    var hours = settings.hoursPerWeek + 0.5;
    hours <= maxHoursPerWeek;
    hours += 0.5
  ) {
    // Копия настроек, в которой изменено только число часов в неделю.
    final candidate = settings.copyWith(hoursPerWeek: hours);
    // Пробно раскладываем те же темы с новым дневным лимитом.
    final sessions = scheduler.schedule(
      orderedTopics,
      candidate,
      remainingMinutes: remainingMinutes,
    );
    // Смотрим, когда закончится такой пробный план.
    final finish = lastStudyDate(sessions);
    // Успеваем к дедлайну — это и есть минимальное подходящее значение.
    if (finish == null || !finish.isAfter(deadline)) return hours;
  }
  // Даже максимума не хватает — дедлайн невыполним.
  return null;
}

/// Находит дату последнего занятия типа «изучение»; повторения не
/// учитываются. Возвращает `null`, если таких занятий нет.
/// Метод статический (не требует объекта), его вызывает и Replanner.
static DateTime? lastStudyDate(List<PlannedSession> sessions) {
  // Самая поздняя найденная дата (пока ничего не нашли).
  DateTime? last;
  // Просматриваем все занятия по очереди.
  for (final s in sessions) {
    // Повторения пропускаем: план закончен, когда изучена последняя тема.
    if (s.kind != SessionKind.study) continue;
    // Запоминаем дату, если она позже уже найденной.
    if (last == null || s.date.isAfter(last)) last = s.date;
  }
```

```
    // Возвращаем самую позднюю дату изучения.
    return last;
  }
}
```

ПРИЛОЖЕНИЕ Д. ПЕРЕПЛАНИРОВАНИЕ

Файл `lib/domain/planner/replanner.dart`.

```
// Перепланирование — перестройка плана, когда пользователь отстал или
// поменял расписание (слой domain, папка planner). Выполненные занятия не
// трогаются, а остаток тем заново раскладывается с сегодняшнего дня.
// Вызывается из PlanService.replan().

import '../entities/schedule_settings.dart';
import '../entities/session.dart';
import '../entities/topic.dart';
import 'dates.dart';
import 'plan_generator.dart';
import 'scheduler.dart';

/// Результат перепланирования.
class ReplanResult {
  /// Создаёт результат; все поля, кроме [requiredHoursPerWeek], обязательны.
  const ReplanResult({
    required this.sessions,
    required this.finishDate,
    required this.deadlineMet,
    this.requiredHoursPerWeek,
  });

  /// Новые невыполненные занятия, начиная с сегодняшнего дня.
  final List<PlannedSession> sessions;

  /// Новая дата окончания — день последнего занятия по изучению темы
  /// (`null`, если изучать больше нечего).
  final DateTime? finishDate;

  /// `true`, если план укладывается в дедлайн или дедлайн не задан.
  final bool deadlineMet;

  /// Сколько часов в неделю нужно, чтобы успеть к дедлайну. Заполняется,
  /// только если [deadlineMet] равно `false`; `null` в этом случае значит,
  /// что не успеть даже при максимуме часов.
  final double? requiredHoursPerWeek;
}

/// Перестраивает план, если пользователь отстал или изменил расписание.
///
/// Выполненные занятия остаются на своих местах. Для каждой темы считается,
/// сколько минут уже изучено; остаток заново раскладывается начиная с
/// [today] теми же правилами, что и при генерации. Невыполненные повторения
/// по уже изученным темам переносятся на ближайшие учебные дни.
class Replanner {
  /// Создаёт перепланировщик. [scheduler] можно подменить (например, в
  /// тестах); по умолчанию — стандартный планировщик.
  const Replanner({this.scheduler = const Scheduler()});

  /// Планировщик, который заново раскладывает остаток тем по дням.
  final Scheduler scheduler;

  /// Перестраивает план.
  ///
  /// [orderedTopics] — темы плана в порядке изучения; [sessions] — все
  /// сохранённые занятия плана (выполненные и нет); [settings] — расписание;
  /// [today] — сегодняшняя дата, с неё начинается новое расписание.
  ReplanResult replan({
    required List<Topic> orderedTopics,
    required List<StudySession> sessions,
    required ScheduleSettings settings,
    required DateTime today,
  }) {
    // Новое расписание начинается с сегодняшнего дня (без времени суток).
    final start = dateOnly(today);
```

```
// Считаем, сколько минут по каждой теме уже изучено: складываем
// минуты выполненных занятий типа «изучение».
final doneMinutes = <String, int>{};
for (final s in sessions) {
  if (s.isDone && s.kind == SessionKind.study) {
    doneMinutes[s.topicId] = (doneMinutes[s.topicId] ?? 0) + s.minutes;
  }
}

// Остаток по каждой теме = длительность темы − уже изучено.
// clamp() не даёт остатку уйти ниже 0 и выше полной длительности.
final remaining = <String, int>{
  for (final t in orderedTopics)
    t.id: (t.minutes - (doneMinutes[t.id] ?? 0)).clamp(0, t.minutes),
};

// Невыполненные повторения тем, которые уже изучены полностью.
// Их не теряем, а передаём планировщику как начальные повторения.
final pendingReviews = <PendingReview>[
  for (final s in sessions)
    if (!s.isDone &&
        s.kind == SessionKind.review &&
        remaining[s.topicId] == 0)
      PendingReview(
        topicId: s.topicId,
        topicTitle: s.topicTitle,
        category: s.category,
        // Просроченное повторение переносим на сегодня, будущее
        // оставляем на его дату.
        dueDate: s.date.isBefore(start) ? start : s.date,
      ),
];

// Копия настроек, у которой дата старта заменена на сегодня.
final newSettings = settings.copyWith(startDate: start);
// Раскладываем остаток тем тем же планировщиком, что и при создании
// плана: передаём остатки минут и перенесённые повторения.
final planned = scheduler.schedule(
  orderedTopics,
  newSettings,
  remainingMinutes: remaining,
  initialReviews: pendingReviews,
);

// Номера частей продолжают уже выполненные части темы.
// Сначала считаем, сколько частей каждой темы уже выполнено.
final doneParts = <String, int>{};
for (final s in sessions) {
  if (s.isDone && s.kind == SessionKind.study) {
    doneParts[s.topicId] = (doneParts[s.topicId] ?? 0) + 1;
  }
}
// Сдвигаем номера: если 2 части темы уже пройдены, новая «часть 1»
// становится «частью 3», а общее число частей тоже растёт на 2.
// Повторения и темы без выполненных частей остаются как есть.
final renumbered = [
  for (final s in planned)
    if (s.kind == SessionKind.study && (doneParts[s.topicId] ?? 0) > 0)
      s.copyWith(
        part: s.part + doneParts[s.topicId]!,
        totalParts: s.totalParts + doneParts[s.topicId]!,
      )
    else
      s,
];

// Новая дата окончания — день последнего занятия по изучению.
final finish = PlanGenerator.lastStudyDate(renumbered);
// Проверяем дедлайн так же, как при генерации плана.
final deadline = settings.deadline;
```

```
    final met = deadline == null || finish == null || !finish.isAfter(deadline);
    // Собираем результат. Если не успеваем, подбираем нужное число часов
    // в неделю с учётом того, что часть тем уже изучена.
    return ReplanResult(
      sessions: renumbered,
      finishDate: finish,
      deadlineMet: met,
      requiredHoursPerWeek: met
          ? null
          : PlanGenerator(scheduler: scheduler).requiredHoursPerWeek(
              orderedTopics,
              newSettings,
              remainingMinutes: remaining,
            ),
    );
  }
}
```

ПРИЛОЖЕНИЕ Е. ТАБЛИЦЫ БАЗЫ ДАННЫХ

Файл `lib/data/db/app_database.dart`.

```
// Описание базы данных SQLite через библиотеку drift (слой data, папка db).
// drift — пакет для работы с SQLite: по таблицам, описанным классами Dart,
// он генерирует код запросов (файл app_database.g.dart, вручную не правится).
// Базу открывает сервер API (bin/marshrut_server.dart), а работают с ней
// репозитории lib/data/repositories. Приложение к базе напрямую не обращается:
// все данные оно получает от сервера через API.

import 'package:drift/drift.dart';

// Подключаем сгенерированную часть этого файла (код запросов drift).
part 'app_database.g.dart';

// Таблицы базы данных SQLite. Код запросов генерирует drift:
//   dart run build_runner build
// Сгенерированный файл app_database.g.dart хранится в репозитории, поэтому
// для запуска приложения генерация не нужна.

/// Пользователи. Пароль хранится только в виде хэша PBKDF2-HMAC-SHA256 с солью
/// (записи версии 1 семестра с хэшем SHA-256 пересчитываются при входе).
///
/// Каждый геттер класса — столбец таблицы. Аннотация `@DataClassName`
/// задаёт имя класса одной строки таблицы (здесь UserRow), который
/// сгенерирует drift.
@DataClassName('UserRow')
class Users extends Table {
  /// Уникальный номер пользователя; база назначает его сама
  /// (autoIncrement — каждая новая запись получает следующий номер).
  IntColumn get id => integer().autoIncrement()();

  /// Имя пользователя (от 2 до 40 символов).
  TextColumn get name => text().withLength(min: 2, max: 40)();

  /// Email — уникальный (два аккаунта с одним email создать нельзя).
  TextColumn get email => text().unique()();

  /// Хэш пароля — результат необратимого преобразования пароля.
  /// Сам пароль в базе не хранится.
  TextColumn get passwordHash => text()();

  /// Соль — случайная строка, которая добавляется к паролю перед
  /// хэшированием, чтобы одинаковые пароли давали разные хэши.
  TextColumn get salt => text()();

  /// Цвет аватара, записанный числом.
  IntColumn get avatarColor => integer()();

  /// Включено ли ежедневное напоминание (по умолчанию выключено).
  BoolColumn get reminderEnabled =>
      boolean().withDefault(const Constant(false))();

  /// Час ежедневного напоминания (по умолчанию 19).
  IntColumn get reminderHour => integer().withDefault(const Constant(19))();

  /// Минуты ежедневного напоминания (по умолчанию 0).
  IntColumn get reminderMinute => integer().withDefault(const Constant(0))();

  /// Дата и время регистрации.
  DateTimeColumn get createdAt => dateTime()();
}

/// Планы обучения. У пользователя может быть несколько планов, один активный.
@DataClassName('PlanRow')
class Plans extends Table {
  /// Уникальный номер плана (назначается автоматически).
  IntColumn get id => integer().autoIncrement()();
```

```
  /// Владелец плана — ссылка (внешний ключ) на таблицу Users. При удалении
  /// пользователя его планы удаляются автоматически (каскадно).
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();

  /// id направления из каталога, по которому построен план.
  TextColumn get directionId => text()();

  /// Название плана.
  TextColumn get title => text()();

  /// Сколько часов в неделю пользователь готов учиться.
  RealColumn get hoursPerWeek => real()();

  /// Учебные дни через запятую: «1,3,5» (1 — понедельник).
  TextColumn get studyDays => text()();

  /// Дата начала обучения.
  DateTimeColumn get startDate => dateTime()();

  /// Дедлайн. nullable — значение может отсутствовать (план без дедлайна).
  DateTimeColumn get deadline => dateTime().nullable()();

  /// Темы, исключённые по тесту, через запятую.
  TextColumn get knownTopicIds => text().withDefault(const Constant(''))();

  /// Активный ли это план (по умолчанию — да).
  BoolColumn get isActive => boolean().withDefault(const Constant(true))();

  /// Дата и время создания плана.
  DateTimeColumn get createdAt => dateTime()();
}

/// Темы в составе плана — снимок, чтобы план не зависел от изменений каталога.
@DataClassName('PlanTopicRow')
class PlanTopics extends Table {
  /// Уникальный номер записи (назначается автоматически).
  IntColumn get id => integer().autoIncrement()();

  /// План, к которому относится тема; удаляется вместе с планом.
  IntColumn get planId =>
      integer().references(Plans, #id, onDelete: KeyAction.cascade)();

  /// id темы (из каталога или собственной темы пользователя).
  TextColumn get topicId => text()();

  /// Название темы.
  TextColumn get title => text()();

  /// Категория темы.
  TextColumn get category => text()();

  /// Длительность изучения темы в минутах.
  IntColumn get minutes => integer()();

  /// Уровень сложности темы (влияет на порядок при сортировке Кана).
  IntColumn get level => integer()();

  /// id тем-зависимостей через запятую (пустая строка — зависимостей нет).
  TextColumn get prerequisites => text().withDefault(const Constant(''))();

  /// Позиция темы в порядке изучения.
  IntColumn get position => integer()();
}

/// Занятия плана (изучение темы или повторение) на конкретную дату.
@DataClassName('SessionRow')
class Sessions extends Table {
  /// Уникальный номер занятия (назначается автоматически).
  IntColumn get id => integer().autoIncrement()();
```

```
  /// План, к которому относится занятие; удаляется вместе с планом.
  IntColumn get planId =>
      integer().references(Plans, #id, onDelete: KeyAction.cascade)();

  /// id темы занятия.
  TextColumn get topicId => text()();

  /// Название темы (копия, чтобы не искать тему при показе списка).
  TextColumn get topicTitle => text()();

  /// Категория темы.
  TextColumn get category => text()();

  /// Дата занятия.
  DateTimeColumn get date => dateTime()();

  /// Длительность занятия в минутах.
  IntColumn get minutes => integer()();

  /// «study» или «review».
  TextColumn get kind => text()();

  /// Номер части темы (по умолчанию 1).
  IntColumn get part => integer().withDefault(const Constant(1))();

  /// Сколько всего частей у темы (по умолчанию 1).
  IntColumn get totalParts => integer().withDefault(const Constant(1))();

  /// Выполнено ли занятие (по умолчанию — нет).
  BoolColumn get isDone => boolean().withDefault(const Constant(false))();

  /// Когда занятие отмечено выполненным; пусто, если ещё не выполнено.
  DateTimeColumn get completedAt => dateTime().nullable()();
}

/// Собственные темы пользователя.
@DataClassName('CustomTopicRow')
class CustomTopics extends Table {
  /// Уникальный номер темы (назначается автоматически).
  IntColumn get id => integer().autoIncrement()();

  /// Автор темы; темы удаляются вместе с пользователем.
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();

  /// Направление, в которое пользователь добавил тему.
  TextColumn get directionId => text()();

  /// Название темы.
  TextColumn get title => text()();

  /// Описание темы (по умолчанию пустое).
  TextColumn get description => text().withDefault(const Constant(''))();

  /// Длительность изучения в минутах.
  IntColumn get minutes => integer()();

  /// Категория темы.
  TextColumn get category => text()();

  /// id тем-зависимостей через запятую.
  TextColumn get prerequisites => text().withDefault(const Constant(''))();

  /// Дата и время создания темы.
  DateTimeColumn get createdAt => dateTime()();
}

/// Заметки пользователя.
@DataClassName('NoteRow')
class Notes extends Table {
```

```
  /// Уникальный номер заметки (назначается автоматически).
  IntColumn get id => integer().autoIncrement()();

  /// Автор заметки; заметки удаляются вместе с пользователем.
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();

  /// Заголовок заметки.
  TextColumn get title => text()();

  /// Текст заметки.
  TextColumn get body => text()();

  /// id темы, к которой привязана заметка (может отсутствовать).
  TextColumn get topicId => text().nullable()();

  /// Название этой темы (может отсутствовать).
  TextColumn get topicTitle => text().nullable()();

  /// Когда заметка создана.
  DateTimeColumn get createdAt => dateTime()();

  /// Когда заметку изменяли последний раз.
  DateTimeColumn get updatedAt => dateTime()();
}

/// Избранные темы.
@DataClassName('FavoriteRow')
class Favorites extends Table {
  /// Пользователь; записи удаляются вместе с ним.
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();

  /// id избранной темы.
  TextColumn get topicId => text()();

  /// Когда тема добавлена в избранное.
  DateTimeColumn get createdAt => dateTime()();

  /// Составной первичный ключ: пара (пользователь, тема) уникальна,
  /// поэтому одну тему нельзя добавить в избранное дважды.
  @override
  Set<Column> get primaryKey => {userId, topicId};
}

/// Завершённые интервалы Pomodoro.
@DataClassName('PomodoroLogRow')
class PomodoroLogs extends Table {
  /// Уникальный номер записи (назначается автоматически).
  IntColumn get id => integer().autoIncrement()();

  /// Пользователь; записи удаляются вместе с ним.
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();

  /// Длительность интервала в минутах.
  IntColumn get minutes => integer()();

  /// Тема, над которой шла работа (может отсутствовать).
  TextColumn get topicTitle => text().nullable()();

  /// Когда интервал завершён.
  DateTimeColumn get completedAt => dateTime()();
}

/// Токены входа в API. После входа сервер выдаёт приложению случайный
/// токен, а приложение отправляет его с каждым запросом. В таблице хранится
/// только хэш SHA-256 токена: даже по копии базы войти не получится.
@DataClassName('ApiTokenRow')
class ApiTokens extends Table {
  /// Хэш SHA-256 токена (64 шестнадцатеричных символа).
```

```
  TextColumn get tokenHash => text()();

  /// Пользователь, которому выдан токен; токены удаляются вместе с ним.
  IntColumn get userId =>
      integer().references(Users, #id, onDelete: KeyAction.cascade)();

  /// Когда токен выдан: по этой дате сервер отбрасывает старые токены.
  DateTimeColumn get createdAt => dateTime()();

  /// Первичный ключ — сам хэш токена.
  @override
  Set<Column> get primaryKey => {tokenHash};
}

/// База данных приложения.
///
/// Аннотация `@DriftDatabase` перечисляет все таблицы; по ней drift
/// генерирует базовый класс `_$AppDatabase` с доступом к таблицам.
@DriftDatabase(
  tables: [
    Users,
    Plans,
    PlanTopics,
    Sessions,
    CustomTopics,
    Notes,
    Favorites,
    PomodoroLogs,
    ApiTokens,
  ],
)
class AppDatabase extends _$AppDatabase {
  /// Принимает executor — «движок», через который drift работает с SQLite.
  /// На сервере это файл базы на компьютере (создаётся в
  /// bin/marshrut_server.dart), в тестах — база в памяти.
  AppDatabase(super.executor);

  /// Версия схемы базы. Её увеличивают при изменении таблиц, чтобы drift
  /// запустил миграцию (обновление структуры старой базы).
  /// Версия 2 добавила таблицу токенов API.
  @override
  int get schemaVersion => 2;

  /// Стратегия миграции: что делать при создании и открытии базы.
  @override
  MigrationStrategy get migration => MigrationStrategy(
    // При первом запуске создаём все таблицы.
    onCreate: (m) => m.createAll(),
    // База старой версии: добавляем таблицы, которых в ней ещё нет.
    onUpgrade: (m, from, to) async {
      // В версии 1 не было таблицы токенов API.
      if (from < 2) await m.createTable(apiTokens);
    },
    // Выполняется перед каждым открытием базы.
    beforeOpen: (details) async {
      // Включаем проверку внешних ключей (каскадное удаление планов и занятий).
      await customStatement('PRAGMA foreign_keys = ON');
    },
  );
}
```

ПРИЛОЖЕНИЕ Ж. ПРАВИЛО ДОСТУПА К ЭКРАНАМ И МАРШРУТЫ

Файл `lib/core/router/app_router.dart`.

```
/// Маршруты, доступные без входа в систему.
const publicRoutes = {'/login', '/register'};

/// Правило доступа к экранам (вынесено в функцию, чтобы его можно было
/// проверить юнит-тестом без запуска интерфейса).
///
/// * пока сессия не проверена — показываем только Splash;
/// * неавторизованный пользователь видит только вход и регистрацию;
/// * авторизованного пользователя с экранов входа и Splash ведём на главную.
///
/// Возвращает адрес, куда нужно перенаправить, или `null`, если можно
/// остаться на запрошенном экране [location].
@visibleForTesting
String? authRedirect(AuthStatus status, String location) {
  // Статус ещё неизвестен (сессия проверяется): держим пользователя
  // на заставке, любой другой адрес перенаправляем на '/splash'.
  if (status == AuthStatus.unknown) {
    return location == '/splash' ? null : '/splash';
  }
  // Вошёл ли пользователь и открыт ли сейчас публичный экран.
  final loggedIn = status == AuthStatus.authenticated;
  final isPublic = publicRoutes.contains(location);
  // Гостя пускаем только на публичные экраны, остальное — на вход.
  if (!loggedIn) return isPublic ? null : '/login';
  // Вошедшему незачем видеть вход, регистрацию и заставку — ведём на главную.
  if (isPublic || location == '/splash') return '/home';
  // Во всех остальных случаях перенаправление не нужно.
  return null;
}

/// Ключ корневого навигатора: через него роутер управляет стеком экранов.
final _rootKey = GlobalKey<NavigatorState>(debugLabel: 'root');

/// Провайдер Riverpod (объект, который создаёт и раздаёт зависимость
/// виджетам), хранящий единственный экземпляр роутера [GoRouter].
///
/// Содержит полную карту экранов приложения и правило доступа
/// [authRedirect].
final routerProvider = Provider<GoRouter>((ref) {
  // Роутер пересчитывает redirect каждый раз, когда меняется состояние входа.
  // ValueNotifier — простой «сигнал»: увеличиваем число, роутер это слышит.
  final refresh = ValueNotifier<int>(0);
  // Подписываемся на контроллер авторизации: любое изменение -> сигнал.
  ref.listen(authControllerProvider, (_, _) => refresh.value++);
  // Освобождаем сигнал, когда провайдер уничтожается.
  ref.onDispose(refresh.dispose);

  // Создаём сам роутер с настройками и списком маршрутов.
  final router = GoRouter(
    // Корневой навигатор приложения.
    navigatorKey: _rootKey,
    // При запуске всегда открываем заставку.
    initialLocation: '/splash',
    // Перепроверять redirect при каждом сигнале refresh.
    refreshListenable: refresh,
    // Перед каждым переходом спрашиваем authRedirect, пускать ли на экран.
    redirect: (context, state) => authRedirect(
      ref.read(authControllerProvider).status,
      state.matchedLocation,
    ),
    // Список всех маршрутов (путь -> экран).
    routes: [
```

```
// Заставка при запуске.
GoRoute(path: '/splash', builder: (_, _) => const SplashScreen()),
// Экран входа. Параметры адреса: email — подставить в поле,
// registered=1 — показать сообщение об успешной регистрации.
GoRoute(
  path: '/login',
  builder: (_, state) => LoginScreen(
    initialEmail: state.uri.queryParameters['email'],
    justRegistered: state.uri.queryParameters['registered'] == '1',
  ),
),
// Экран регистрации.
GoRoute(path: '/register', builder: (_, _) => const RegisterScreen()),

// Онбординг: создание нового плана.
// Шаги: выбор направления -> тест -> расписание -> предпросмотр.
GoRoute(
  path: '/onboarding',
  builder: (_, _) => const DirectionScreen(),
  // Вложенные пути: /onboarding/test, /onboarding/schedule и т. д.
  routes: [
    GoRoute(path: 'test', builder: (_, _) => const DiagnosticScreen()),
    GoRoute(path: 'schedule', builder: (_, _) => const ScheduleScreen()),
    GoRoute(path: 'preview', builder: (_, _) => const PreviewScreen()),
  ],
),

// Основные вкладки с нижней навигацией.
// indexedStack хранит все вкладки в памяти, поэтому при переключении
// каждая сохраняет своё состояние (прокрутку, введённые данные).
StatefulShellRoute.indexedStack(
  // MainShell — оболочка с панелью навигации вокруг текущей вкладки.
  builder: (_, _, shell) => MainShell(navigationShell: shell),
  // Каждая ветка — отдельная вкладка со своей историей переходов.
  branches: [
    // Вкладка «Главная».
    StatefulShellBranch(
      routes: [
        GoRoute(path: '/home', builder: (_, _) => const HomeScreen()),
      ],
    ),
    // Вкладка «План».
    StatefulShellBranch(
      routes: [
        GoRoute(path: '/plan', builder: (_, _) => const PlanScreen()),
      ],
    ),
    // Вкладка «Темы».
    StatefulShellBranch(
      routes: [
        GoRoute(path: '/topics', builder: (_, _) => const TopicsScreen()),
      ],
    ),
    // Вкладка «Статистика».
    StatefulShellBranch(
      routes: [
        GoRoute(path: '/stats', builder: (_, _) => const StatsScreen()),
      ],
    ),
    // Вкладка «Профиль».
    StatefulShellBranch(
      routes: [
        GoRoute(
          path: '/profile',
          builder: (_, _) => const ProfileScreen(),
        ),
      ],
    ),
  ],
),
```

```
// Экраны поверх вкладок.
// Занятие по его номеру id из адреса (например, /session/5).
GoRoute(
  path: '/session/:id',
  builder: (_, state) =>
      SessionScreen(sessionId: int.parse(state.pathParameters['id']!)),
),
// Список всех планов пользователя.
GoRoute(path: '/plans', builder: (_, _) => const PlansScreen()),
// Настройки текущего плана.
GoRoute(
  path: '/plan/settings',
  builder: (_, _) => const PlanSettingsScreen(),
),
// Таймер Помодоро; параметр topic — название темы для подписи на экране.
GoRoute(
  path: '/pomodoro',
  builder: (_, state) =>
      PomodoroScreen(topicTitle: state.uri.queryParameters['topic']),
),
// Карточка одной темы по её строковому id.
GoRoute(
  path: '/topic/:id',
  builder: (_, state) =>
      TopicScreen(topicId: state.pathParameters['id']!),
),
// Создание своей темы; direction — направление, куда её добавить.
// Этот маршрут стоит выше '/topics/custom/:id', чтобы слово 'new'
// не приняли за id.
GoRoute(
  path: '/topics/custom/new',
  builder: (_, state) => CustomTopicScreen(
    directionId: state.uri.queryParameters['direction'],
  ),
),
// Редактирование своей темы по её id.
GoRoute(
  path: '/topics/custom/:id',
  builder: (_, state) =>
      CustomTopicScreen(topicId: state.pathParameters['id']),
),
// Список заметок.
GoRoute(path: '/notes', builder: (_, _) => const NotesScreen()),
// Создание или редактирование заметки.
GoRoute(
  path: '/notes/edit',
  builder: (_, state) {
    // Параметры адреса: id заметки (если редактируем) и тема,
    // к которой заметка привязана.
    final q = state.uri.queryParameters;
    return NoteEditScreen(
      // Новый ключ для каждой заметки: иначе при переходе между
      // заметками сохранялось бы состояние предыдущей.
      key: ValueKey(state.uri.toString()),
      // Нет id или он не число — значит, создаётся новая заметка.
      noteId: int.tryParse(q['id'] ?? ''),
      topicId: q['topicId'],
      topicTitle: q['topicTitle'],
    );
  },
),
// Редактирование профиля (имя, email).
GoRoute(
  path: '/profile/edit',
  builder: (_, _) => const EditProfileScreen(),
),
// Смена пароля.
GoRoute(
  path: '/profile/password',
  builder: (_, _) => const ChangePasswordScreen(),
),
```

```
    ],
  );
  // Освобождаем ресурсы роутера, когда провайдер уничтожается.
  ref.onDispose(router.dispose);
  // Отдаём готовый роутер всем, кто его читает.
  return router;
});
```

ПРИЛОЖЕНИЕ И. МОДУЛЬНЫЕ ТЕСТЫ ТОПОЛОГИЧЕСКОЙ СОРТИРОВКИ

Файл `test/domain/topological_sort_test.dart`.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:marshrut/domain/entities/topic.dart';
import 'package:marshrut/domain/planner/topological_sort.dart';

Topic t(String id, {List<String> deps = const [], int level = 1}) => Topic(
  id: id,
  title: id,
  minutes: 60,
  category: 'Тест',
  level: level,
  prerequisites: deps,
);

List<String> ids(List<Topic> topics) => topics.map((t) => t.id).toList();

void main() {
  group('topologicalSort (алгоритм Кана)', () {
    test('каждая тема идёт после всех своих зависимостей', () {
      final topics = [
        t('riverpod', deps: ['widgets', 'async']),
        t('widgets', deps: ['dart']),
        t('async', deps: ['dart']),
        t('dart'),
      ];

      final order = ids(topologicalSort(topics));

      expect(order.first, 'dart');
      expect(order.last, 'riverpod');
      for (final topic in topics) {
        for (final dep in topic.prerequisites) {
          expect(
            order.indexOf(dep) < order.indexOf(topic.id),
            isTrue,
            reason: '$dep должна идти раньше ${topic.id}',
          );
        }
      }
    });

    test(
      'независимые темы упорядочены по уровню, затем по исходной позиции',
      () {
        final topics = [
          t('hard', level: 3),
          t('easy_b', level: 1),
          t('medium', level: 2),
          t('easy_a', level: 1),
        ];

        expect(ids(topologicalSort(topics)), [
          'easy_b',
          'easy_a',
          'medium',
          'hard',
        ]);
      },
    );

    test('зависимость, которой нет в наборе, считается выполненной', () {
      final topics = [
        t('b', deps: ['known_topic']),
```

```
        t('a'),
      ];

      expect(ids(topologicalSort(topics)), ['b', 'a']);
    });

    test('повторяющаяся зависимость не ломает подсчёт степени', () {
      final topics = [
        t('b', deps: ['a', 'a']),
        t('a'),
      ];

      expect(ids(topologicalSort(topics)), ['a', 'b']);
    });

    test('пустой список даёт пустой результат', () {
      expect(topologicalSort(const []), isEmpty);
    });

    test('цикл в графе обнаруживается', () {
      final topics = [
        t('a', deps: ['c']),
        t('b', deps: ['a']),
        t('c', deps: ['b']),
        t('free'),
      ];

      expect(
        () => topologicalSort(topics),
        throwsA(
          isA<CycleDetectedException>().having(
            (e) => e.topicIds,
            'topicIds',
            unorderedEquals(['a', 'b', 'c']),
          ),
        ),
      );
    });
  });
}
```

ПРИЛОЖЕНИЕ К. СЕРВЕР API: АДРЕСА, МАРШРУТЫ, ПРОВЕРКА ТОКЕНА И ОБРАБОТКА ОШИБОК

Все адреса, кроме проверки связи, каталога, регистрации и входа, требуют заголовок `Authorization: Bearer <токен>`. Тела запросов и ответов передаются в формате JSON.

Таблица К.1 — Адреса API сервера

Метод	**Адрес**	**Назначение**
GET	`/api/health`	Проверка связи
GET	`/api/directions`	Каталог направлений и тем
POST	`/api/auth/register`	Регистрация
POST	`/api/auth/login`	Вход, выдача токена
POST	`/api/auth/logout`	Выход, удаление токена
GET	`/api/me`	Данные своего профиля
PUT	`/api/me`	Изменение имени, email и цвета аватара
PUT	`/api/me/password`	Смена пароля
PUT	`/api/me/reminder`	Время ежедневного напоминания
DELETE	`/api/me`	Удаление аккаунта со всеми данными
GET	`/api/plans`	Список планов
POST	`/api/plans`	Сохранение нового плана
GET	`/api/plans/active`	Активный план
GET	`/api/plans/{id}`	Один план
PATCH	`/api/plans/{id}`	Переименование плана
DELETE	`/api/plans/{id}`	Удаление плана с темами и занятиями
POST	`/api/plans/{id}/activate`	Сделать план активным
PUT	`/api/plans/{id}/settings`	Новые настройки расписания
GET	`/api/plans/{id}/topics`	Темы плана по порядку
PUT	`/api/plans/{id}/topics`	Новый список тем плана
GET	`/api/plans/{id}/sessions`	Занятия плана
PUT	`/api/plans/{id}/sessions/open`	Новое расписание невыполненных занятий
GET	`/api/sessions/completed`	Выполненные занятия
GET	`/api/sessions/{id}`	Одно занятие
PUT	`/api/sessions/{id}/done`	Отметка о выполнении
GET	`/api/custom-topics`	Свои темы
POST	`/api/custom-topics`	Новая своя тема
GET	`/api/custom-topics/{id}`	Одна своя тема
PUT	`/api/custom-topics/{id}`	Изменение своей темы
DELETE	`/api/custom-topics/{id}`	Удаление своей темы
GET	`/api/favorites`	Избранные темы
POST	`/api/favorites/{id}/toggle`	Добавить в избранное или убрать
GET	`/api/notes`	Заметки (все или к одной теме)
POST	`/api/notes`	Новая заметка
GET	`/api/notes/{id}`	Одна заметка
PUT	`/api/notes/{id}`	Изменение заметки
DELETE	`/api/notes/{id}`	Удаление заметки

Метод	Адрес	Назначение
GET	`/api/pomodoros`	Журнал Pomodoro
POST	`/api/pomodoros`	Запись завершённого интервала

Файл `lib/server/api_server.dart` (фрагмент).

```
/// Сервер API: маршруты (какой адрес что делает) и их обработчики.
class ApiServer {
  /// [db] — база SQLite; [catalog] — направления и темы в виде JSON (как в
  /// файлах assets/catalog); [hasher] — хэширование паролей (в тестах —
  /// быстрое); [log] — куда писать журнал запросов; [clock] — текущее время.
  ApiServer({
    required this.db,
    required this.catalog,
    PasswordHasher? hasher,
    this.log,
    this.tokenLifetime = const Duration(days: 30),
    DateTime Function()? clock,
  }) : _auth = AuthRepositoryImpl(db, hasher: hasher),
       _plans = PlanRepositoryImpl(db),
       _topics = TopicRepositoryImpl(db),
       _notes = NoteRepositoryImpl(db),
       _pomodoros = PomodoroRepositoryImpl(db),
       _clock = clock ?? DateTime.now;

  /// Версия API: приложение может проверить её через /api/health.
  static const apiVersion = 1;

  /// База данных сервера.
  final AppDatabase db;

  /// Каталог направлений (JSON-объекты из файлов assets/catalog).
  final List<Object?> catalog;

  /// Журнал: сюда пишется строка о каждом запросе. `null` — без журнала.
  final void Function(String line)? log;

  /// Сколько действует токен входа.
  final Duration tokenLifetime;

  /// Репозиторий пользователей (регистрация, вход, профиль).
  final AuthRepositoryImpl _auth;

  /// Репозиторий планов, тем плана и занятий.
  final PlanRepositoryImpl _plans;

  /// Репозиторий своих тем и избранного.
  final TopicRepositoryImpl _topics;

  /// Репозиторий заметок.
  final NoteRepositoryImpl _notes;

  /// Репозиторий журнала Pomodoro.
  final PomodoroRepositoryImpl _pomodoros;

  /// Источник текущего времени.
  final DateTime Function() _clock;

  /// Криптостойкий генератор случайных чисел для токенов.
  final _random = Random.secure();

  /// Обработчик всех запросов. Pipeline — цепочка «прослоек» (middleware):
  /// сначала журнал, затем перехват ошибок, затем нужный маршрут.
  late final Handler handler = const Pipeline()
      .addMiddleware(_logRequests)
      .addMiddleware(_handleErrors)
      .addHandler(_buildRouter().call);

  /// Таблица маршрутов: метод HTTP + адрес → функция-обработчик.
```

```
/// `<id|[0-9]+>` — часть адреса, которая передаётся в обработчик (только
/// цифры).
Router _buildRouter() {
  final r = Router(notFoundHandler: (_) => _error(404, 'Адрес не найден'));

  // Служебные адреса: страница сервера, проверка связи, каталог.
  r.get('/', _home);
  r.get('/api/health', _health);
  r.get('/api/directions', (Request _) => _json(catalog));

  // Регистрация, вход, выход и профиль.
  r.post('/api/auth/register', _register);
  r.post('/api/auth/login', _login);
  r.post('/api/auth/logout', _logout);
  r.get('/api/me', _me);
  r.put('/api/me', _updateProfile);
  r.delete('/api/me', _deleteAccount);
  r.put('/api/me/password', _changePassword);
  r.put('/api/me/reminder', _updateReminder);

  // Планы обучения.
  r.get('/api/plans', _listPlans);
  r.post('/api/plans', _createPlan);
  r.get('/api/plans/active', _activePlan);
  r.get('/api/plans/<id|[0-9]+>', _getPlan);
  r.patch('/api/plans/<id|[0-9]+>', _renamePlan);
  r.delete('/api/plans/<id|[0-9]+>', _deletePlan);
  r.post('/api/plans/<id|[0-9]+>/activate', _activatePlan);
  r.put('/api/plans/<id|[0-9]+>/settings', _updateSettings);
  r.get('/api/plans/<id|[0-9]+>/topics', _planTopics);
  r.put('/api/plans/<id|[0-9]+>/topics', _replacePlanTopics);
  r.get('/api/plans/<id|[0-9]+>/sessions', _planSessions);
  r.put('/api/plans/<id|[0-9]+>/sessions/open', _replaceOpenSessions);

  // Занятия.
  r.get('/api/sessions/completed', _completedSessions);
  r.get('/api/sessions/<id|[0-9]+>', _getSession);
  r.put('/api/sessions/<id|[0-9]+>/done', _setSessionDone);

  // Свои темы и избранное.
  r.get('/api/custom-topics', _listCustomTopics);
  r.post('/api/custom-topics', _createCustomTopic);
  r.get('/api/custom-topics/<id>', _getCustomTopic);
  r.put('/api/custom-topics/<id>', _updateCustomTopic);
  r.delete('/api/custom-topics/<id>', _deleteCustomTopic);
  r.get('/api/favorites', _favorites);
  r.post('/api/favorites/<topicId>/toggle', _toggleFavorite);

  // Заметки.
  r.get('/api/notes', _listNotes);
  r.post('/api/notes', _createNote);
  r.get('/api/notes/<id|[0-9]+>', _getNote);
  r.put('/api/notes/<id|[0-9]+>', _updateNote);
  r.delete('/api/notes/<id|[0-9]+>', _deleteNote);

  // Журнал Pomodoro.
  r.get('/api/pomodoros', _listPomodoros);
  r.post('/api/pomodoros', _logPomodoro);
  return r;
}

// ---------------------------------------------------------------------
// Прослойки (middleware)
// ---------------------------------------------------------------------

/// Пишет в журнал строку о каждом запросе: время, метод, адрес, код
/// ответа и сколько миллисекунд занял ответ.
Handler _logRequests(Handler inner) => (request) async {
  // Засекаем время обработки.
  final watch = Stopwatch()..start();
  final response = await inner(request);
```

```
  // Время в виде ЧЧ:ММ:СС.
  final now = _clock().toIso8601String().substring(11, 19);
  final query = request.url.hasQuery ? '?${request.url.query}' : '';
  log?.call(
    '$now  ${request.method.padRight(6)} /${request.url.path}$query'
    '  →  ${response.statusCode}  (${watch.elapsedMilliseconds} мс)',
  );
  return response;
};

/// Перехватывает ошибки обработчиков и превращает их в ответ JSON с
/// нужным кодом: так приложение всегда получает понятный текст ошибки.
Handler _handleErrors(Handler inner) => (request) async {
  try {
    return await inner(request);
  } on HttpError catch (e) {
    // Ошибка, которую обработчик выбросил сам (401, 404 …).
    return _error(e.statusCode, e.message);
  } on AppException catch (e) {
    // Ошибка правил приложения: «email уже занят», «неверный пароль» …
    return _error(400, e.message);
  } on FormatException {
    // Неверный формат данных (например, дата не в формате ISO 8601).
    return _error(400, 'Некорректные данные в запросе');
  } on TypeError {
    // В JSON поле другого типа, чем ожидалось (строка вместо числа …).
    return _error(400, 'Некорректные данные в запросе');
  } on Object catch (e, stack) {
    // Непредвиденная ошибка: подробности — только в журнал сервера.
    log?.call('Ошибка сервера: $e\n$stack');
    return _error(500, 'Внутренняя ошибка сервера');
  }
};

// ---------------------------------------------------------------------
// Вспомогательные функции
// ---------------------------------------------------------------------

/// Ответ с телом JSON. [status] — код ответа (по умолчанию 200 «OK»).
Response _json(Object? body, {int status = 200}) => Response(
  status,
  body: jsonEncode(body),
  headers: {'content-type': 'application/json; charset=utf-8'},
);

/// Ответ 201 «Создано» с идентификатором новой записи.
Response _created(Object id) => _json({'id': id}, status: 201);

/// Ответ 204 «Нет содержимого»: действие выполнено, данных в ответе нет.
Response _noContent() => Response(204);

/// Ответ с ошибкой: код [status] и JSON `{"error": message}`.
Response _error(int status, String message) =>
    _json({'error': message}, status: status);

/// Читает тело запроса как JSON-объект.
Future<Json> _body(Request request) async {
  final text = await request.readAsString();
  try {
    final data = jsonDecode(text);
    if (data is Map<String, dynamic>) return data;
  } on FormatException {
    // Ниже ответим той же ошибкой, что и для не-объекта.
  }
  throw const HttpError(400, 'Тело запроса должно быть объектом JSON');
}

/// Достаёт из JSON обязательное поле типа [T] или отвечает ошибкой 400.
T _field<T>(Json body, String name) {
  final value = body[name];
  if (value is T) return value;
```

```
  throw HttpError(400, 'Поле «$name» не заполнено или имеет неверный тип');
}

/// Необязательное поле типа [T]: `null`, если его нет.
T? _optional<T>(Json body, String name) {
  final value = body[name];
  if (value == null || value is T) return value as T?;
  throw HttpError(400, 'Поле «$name» имеет неверный тип');
}

/// Список JSON-объектов из поля [name].
List<Json> _objects(Json body, String name) =>
    _field<List<dynamic>>(body, name).cast<Json>();

/// Повторная проверка данных на сервере теми же правилами, что и в
/// формах приложения: первая найденная ошибка уходит ответом 400.
void _validate(List<String?> errors) {
  for (final error in errors) {
    if (error != null) throw HttpError(400, error);
  }
}

/// Хэш токена SHA-256: в базе хранится он, а не сам токен.
static String _hashToken(String token) =>
    sha256.convert(utf8.encode(token)).toString();

/// Выдаёт новый токен входа пользователю [userId] и возвращает его.
/// Токен — 32 случайных байта в виде строки base64url.
Future<String> issueToken(int userId) async {
  final bytes = List<int>.generate(32, (_) => _random.nextInt(256));
  final token = base64Url.encode(bytes).replaceAll('=', '');
  await db
      .into(db.apiTokens)
      .insert(
        ApiTokensCompanion.insert(
          tokenHash: _hashToken(token),
          userId: userId,
          createdAt: _clock(),
        ),
      );
  return token;
}

/// Токен из заголовка `Authorization: Bearer <токен>` или `null`.
String? _tokenOf(Request request) {
  final header = request.headers['authorization'];
  if (header == null || !header.startsWith('Bearer ')) return null;
  return header.substring('Bearer '.length).trim();
}

/// Определяет по токену, какой пользователь отправил запрос.
/// Без токена или с устаревшим токеном — ошибка 401.
Future<int> _userId(Request request) async {
  final token = _tokenOf(request);
  if (token == null) throw const HttpError(401, 'Требуется вход в аккаунт');
  // Ищем в базе запись с хэшем этого токена.
  final row = await (db.select(
    db.apiTokens,
  )..where((t) => t.tokenHash.equals(_hashToken(token)))).getSingleOrNull();
  // Токена нет или он старше срока действия.
  if (row == null || _clock().difference(row.createdAt) > tokenLifetime) {
    throw const HttpError(401, 'Сессия истекла, войдите снова');
  }
  return row.userId;
}

/// Проверяет, что план [planId] принадлежит пользователю [userId].
Future<void> _ownPlan(int userId, int planId) async {
  final plan = await _plans.planById(planId);
  if (plan == null || plan.userId != userId) {
    throw const HttpError(404, 'План не найден');
```

```
    }
  }

  /// Возвращает занятие [sessionId], если оно из плана пользователя.
  Future<StudySession> _ownSession(int userId, int sessionId) async {
    final session = await _plans.sessionById(sessionId);
    final plan = session == null ? null : await _plans.planById(session.planId);
    if (session == null || plan == null || plan.userId != userId) {
      throw const HttpError(404, 'Занятие не найдено');
    }
    return session;
  }

  /// Возвращает строку своей темы [topicId], если тема принадлежит
  /// пользователю [userId]. id своей темы выглядит как `custom_12`.
  Future<CustomTopicRow> _ownCustomTopic(int userId, String topicId) async {
    final id = TopicRepositoryImpl.isCustomId(topicId)
        ? int.tryParse(
            topicId.substring(TopicRepositoryImpl.customPrefix.length),
          )
        : null;
    final row = id == null
        ? null
        : await (db.select(db.customTopics)
                ..where((t) => t.id.equals(id) & t.userId.equals(userId)))
              .getSingleOrNull();
    if (row == null) throw const HttpError(404, 'Тема не найдена');
    return row;
  }

  /// Проверяет, что заметка [noteId] принадлежит пользователю [userId].
  Future<void> _ownNote(int userId, int noteId) async {
    final note = await _notes.noteById(noteId);
    if (note == null || note.userId != userId) {
      throw const HttpError(404, 'Заметка не найдена');
    }
  }

  // … обработчики запросов …
}
```

ПРИЛОЖЕНИЕ Л. КЛИЕНТ API В ПРИЛОЖЕНИИ

Файл `lib/data/api/api_client.dart`.

```
/// Ошибка, которую вернул сервер (код ответа 4xx или 5xx).
class ApiException extends AppException {
  /// [statusCode] — код ответа HTTP, [message] — текст для пользователя.
  const ApiException(this.statusCode, super.message);

  /// Код ответа HTTP, например 400, 401, 404 или 500.
  final int statusCode;
}

/// Клиент API: отправляет запросы и разбирает ответы сервера.
class ApiClient {
  /// [baseUrl] — адрес сервера, например `http://192.168.1.5:8080`;
  /// [token] — токен входа, если пользователь уже вошёл;
  /// [httpClient] можно подменить в тестах;
  /// [timeout] — сколько ждать ответа сервера.
  ApiClient({
    required String baseUrl,
    this.token,
    http.Client? httpClient,
    this.timeout = const Duration(seconds: 10),
  }) : _baseUrl = normalizeUrl(baseUrl),
       _ownsHttp = httpClient == null,
       _http = httpClient ?? http.Client();

  /// Объект пакета http, который физически отправляет запросы.
  final http.Client _http;

  /// true, если объект http создан этим клиентом (тогда его закрывает
  /// [close]); переданный снаружи объект закрывает его владелец.
  final bool _ownsHttp;

  /// Предельное время ожидания ответа.
  final Duration timeout;

  /// Адрес сервера без косой черты в конце.
  String _baseUrl;

  /// Токен входа: отправляется в заголовке `Authorization`.
  String? token;

  /// Вызывается, когда сервер ответил 401 («нужно войти»): так приложение
  /// узнаёт, что токен устарел, и возвращает пользователя на экран входа.
  void Function()? onUnauthorized;

  /// Сигнал «данные на сервере изменились»: после каждого успешного запроса
  /// на изменение (POST, PUT, PATCH, DELETE) сюда приходит событие, и
  /// открытые экраны заново загружают свои данные.
  final _changes = StreamController<void>.broadcast();

  /// Поток событий об изменении данных.
  Stream<void> get changes => _changes.stream;

  /// Текущий адрес сервера.
  String get baseUrl => _baseUrl;

  /// Меняет адрес сервера (например, после правки в настройках входа).
  set baseUrl(String value) => _baseUrl = normalizeUrl(value);

  /// Приводит адрес к виду `http://хост:порт`: убирает пробелы и косую
  /// черту в конце, а если схема не указана — добавляет `http://`.
  static String normalizeUrl(String url) {
    // Убираем пробелы по краям и косые черты в конце адреса.
    var result = url.trim().replaceAll(RegExp(r'/+$'), '');
    // Пользователь мог ввести просто «192.168.1.5:8080».
    if (result.isNotEmpty && !result.contains('://')) {
```

```
    result = 'http://$result';
  }
  return result;
}

/// GET-запрос: получение данных.
Future<dynamic> get(String path, {Map<String, String>? query}) =>
    _send('GET', path, query: query);

/// POST-запрос: создание записи или действие.
Future<dynamic> post(String path, [Object? body]) =>
    _send('POST', path, body: body);

/// PUT-запрос: изменение записи целиком.
Future<dynamic> put(String path, [Object? body]) =>
    _send('PUT', path, body: body);

/// PATCH-запрос: изменение части записи.
Future<dynamic> patch(String path, [Object? body]) =>
    _send('PATCH', path, body: body);

/// DELETE-запрос: удаление записи.
Future<dynamic> delete(String path, [Object? body]) =>
    _send('DELETE', path, body: body);

/// Отправляет запрос и возвращает разобранный JSON ответа
/// (`null`, если у ответа нет тела, например код 204).
Future<dynamic> _send(
  String method,
  String path, {
  Object? body,
  Map<String, String>? query,
}) async {
  // Собираем полный адрес: сервер + путь + параметры запроса.
  final Uri uri;
  try {
    uri = Uri.parse('$_baseUrl$path').replace(queryParameters: query);
  } on FormatException {
    throw ConnectionException('Неверный адрес сервера: $_baseUrl');
  }
  // Адрес без хоста (например, пустой) отправить нельзя.
  if (!uri.hasAuthority || !uri.scheme.startsWith('http')) {
    throw ConnectionException(
      'Неверный адрес сервера: «$_baseUrl». Пример: http://192.168.1.5:8080',
    );
  }
  // Готовим запрос: метод, адрес, заголовки и тело в JSON.
  final request = http.Request(method, uri)
    ..headers['Accept'] = 'application/json';
  // Токен входа передаётся в стандартном заголовке Authorization.
  final currentToken = token;
  if (currentToken != null) {
    request.headers['Authorization'] = 'Bearer $currentToken';
  }
  // Тело есть только у запросов с данными.
  if (body != null) {
    request.headers['Content-Type'] = 'application/json; charset=utf-8';
    request.body = jsonEncode(body);
  }

  final http.Response response;
  try {
    // Отправляем запрос и ждём ответ не дольше timeout.
    final streamed = await _http.send(request).timeout(timeout);
    response = await http.Response.fromStream(streamed).timeout(timeout);
  } on TimeoutException {
    throw ConnectionException(
      'Сервер $_baseUrl не отвечает. Проверьте, что сервер запущен.',
    );
  } on http.ClientException {
    // Нет сети, сервер выключен или адрес неверный.
```

```
      throw ConnectionException(
        'Нет связи с сервером $_baseUrl. Проверьте, что сервер запущен и '
        'телефон в той же сети Wi-Fi.',
      );
    }

    // Разбираем тело ответа (оно в UTF-8, чтобы не испортить кириллицу).
    final text = utf8.decode(response.bodyBytes);
    final Object? data;
    try {
      data = text.isEmpty ? null : jsonDecode(text);
    } on FormatException {
      throw ApiException(
        response.statusCode,
        'Сервер вернул ответ не в формате JSON (код ${response.statusCode})',
      );
    }

    // Успешный ответ (коды 200–299).
    if (response.statusCode >= 200 && response.statusCode < 300) {
      // Запрос на изменение прошёл — сообщаем экранам, что данные другие.
      if (method != 'GET') _changes.add(null);
      return data;
    }
    // Текст ошибки сервер присылает в поле error.
    final message = data is Map && data['error'] is String
        ? data['error'] as String
        : 'Ошибка сервера (код ${response.statusCode})';
    // 401 — токен недействителен: просим войти заново.
    if (response.statusCode == 401) onUnauthorized?.call();
    throw ApiException(response.statusCode, message);
  }

  /// GET /api/health — проверяет, что по адресу [baseUrl] отвечает именно
  /// сервер «Маршрута». При ошибке выбрасывает [AppException] с причиной.
  Future<void> checkHealth() async {
    final data = await get('/api/health');
    if (data is! Map || data['service'] != 'Маршрут') {
      throw const ConnectionException(
        'По этому адресу отвечает не сервер «Маршрута»',
      );
    }
  }

  /// Поток данных, который обновляется сам: сразу загружает значение через
  /// [load], а потом загружает заново после каждого изменения данных
  /// (событие из [changes]). Если загрузка не удалась, поток передаёт
  /// ошибку экрану и через [retryDelay] пробует ещё раз.
  Stream<T> watch<T>(
    Future<T> Function() load, {
    Duration retryDelay = const Duration(seconds: 5),
  }) {
    // Поток, который получит экран.
    late final StreamController<T> controller;
    // Подписка на события об изменении данных.
    StreamSubscription<void>? subscription;
    // Таймер повторной попытки после ошибки.
    Timer? retry;
    // Номер последнего запроса: ответ на устаревший запрос отбрасывается.
    var generation = 0;
    // true, когда экран закрыт и данные больше не нужны.
    var cancelled = false;

    // Загружает данные и передаёт их в поток.
    Future<void> refresh() async {
      retry?.cancel();
      final current = ++generation;
      try {
        final value = await load();
        // Пока ждали ответ, мог уйти более новый запрос или экран закрылся.
        if (current == generation && !cancelled) controller.add(value);
```

```
      } on Object catch (error, stack) {
        if (current != generation || cancelled) return;
        // Показываем ошибку и через несколько секунд пробуем снова.
        controller.addError(error, stack);
        retry = Timer(retryDelay, refresh);
      }
    }

    controller = StreamController<T>(
      // Экран подписался: следим за изменениями и загружаем данные.
      onListen: () {
        subscription = changes.listen((_) => refresh());
        refresh();
      },
      // Экран закрыт: отписываемся и отменяем повтор.
      onCancel: () async {
        cancelled = true;
        retry?.cancel();
        await subscription?.cancel();
      },
    );
    return controller.stream;
  }

  /// Закрывает соединения (при завершении работы).
  void close() {
    if (_ownsHttp) _http.close();
    _changes.close();
  }
}
```